

From Browsing to Delivery: A Predictive Framework for Operational Efficiency and Customer Satisfaction

Team “Four People and Some Data” - Meshach Samuel, Yanzhe Qu, Suyong Choi, Mathew Opitz

College of Liberal Arts and Sciences, Arizona State University

DAT 490: Data Science Capstone

Prof. Laurence Schneider

June 22, 2025

Table of Contents

Abstract.....	1
Literature Review.....	2
Origins of e-Commerce.....	2
E-Commerce and Analytics.....	2
Project Dataset and Questions for Investigation.....	3
Project Plan.....	4
Introduction.....	4
Business Description.....	4
Project Objectives.....	5
Research Questions.....	5
Data Description.....	5
Methodology.....	6
Project Timeline.....	9
Roles and Responsibilities.....	10
Tools and Resources.....	11
Anticipated Challenges.....	11
Final Deliverables.....	11
Exploratory Data Analysis.....	12
Loading Raw Datasets.....	12
Merging Datasets.....	12
Research Question 1.....	12
Objective.....	12
Dataset Characteristics.....	12
Key Visualizations.....	13
Key Insights.....	14
Conclusion.....	14
Research Question 2.....	14
Objective.....	14
Analysis Approach.....	14
Key Visualizations to Include.....	14
Key Insights.....	15
Conclusion.....	16
Research Question 3.....	16
Objective.....	16
Analysis Approach.....	16
Key Visualizations to Include.....	16
Key Insights.....	17
Conclusion.....	17
Summary and Recommendations.....	17
Recommendations:.....	18
Methodology.....	19

Research Question 1.....	19
Research Question 2.....	21
Research Question 3.....	23
Data Visualizations and Analysis.....	26
Research Question 1.....	26
Dataset Characteristics.....	26
Model Comparison Table.....	26
Individual Model Comparison.....	27
Logistic Regression.....	27
Decision Tree.....	28
Random Forest.....	29
XGBoost.....	30
Artificial Neural Net (ANN).....	31
Visualizations.....	33
Research Question 2.....	35
Logistic Regression Model.....	39
Support Vector Machine Model.....	43
Comparison of Model Outcomes: Logistic Regression & Linear Support Vector Classifier.....	46
Conclusions.....	46
Research Question 3.....	47
Dataset Characteristics.....	47
Clustering Analysis.....	47
Business Implications.....	50
Model Comparison Table.....	50
Individual Model Comparison.....	51
Logistic Regression.....	51
XGBoost.....	52
Artificial Neural Net (ANN).....	53
Random Forest.....	54
Decision Tree.....	55
Conclusion.....	57
Ethical Recommendations.....	59
Fairness and Bias.....	59
Transparency and Interpretability.....	59
Misuse and Harm.....	59
Data Privacy and Security.....	60
Human-in-the-Loop Decision Making.....	60
Continuous Monitoring and Feedback.....	60
Challenges.....	61
Recommendations.....	63
References.....	65
Code.....	66
Combining All Datasets into a Single Dataset.....	66

Cleaning Dataset.....	68
Exploratory Data Analysis (EDA).....	71
Research Question 1.....	81
Research Question 2.....	98
Research Question 3.....	119

Abstract

This capstone project focused on three avenues of inquiry into an historical e-commerce dataset provided for public scrutiny by Oline, one of the largest retailers in Brazil. The dataset included information regarding order processing and delivery, customer and seller locations, customer reviews, product descriptions and shipping dimensions, and freight costs. Exploration of the data, which covered approximately two years' worth of online transactions, led to business-related questions about late deliveries, late seller handoffs to carriers, and customer cancellation and return behaviors, all aimed at providing better long-term customer service and delivery outcomes. Multiple predictive models harnessing several different machine learning techniques were tested to aid in identifying at-risk orders for late delivery, sellers whose late handoffs to carriers consistently led to late deliveries, and customer segments most prone to cancelling or returning orders. Based on modeling outcomes, late-to-customer deliveries are primarily driven by the time taken by sellers to ready orders for transfer to carriers (a.k.a. handoff delay). Attempting to dig further into the causes for seller handoff delays, and linking them through to eventual late deliveries was relatively unsuccessful, with the primary outcome from modeling the behaviors only producing a loose correlation between late handoffs and late deliveries (Pearson $R = 0.297$, $p = 0.000$) and no other common categorical or numerical categories between the predictive models to point to for further investigation. Regarding customer cancellations, the primary predictive factor was found to be whether or not the customer was a first-time buyer, with some evidence that customer location and possibly relative position compared to sellers could be factors, though no causal link was made between the latter factors. Ultimately, the low rate of cancellation across all cohorts made prediction difficult, as all models either vastly over-predicted cancellations (random forest, decision tree) or vastly under-predicted them (logit, XGBoost, ANN).

Literature Review

Origins of e-Commerce

The sale of goods and services from any seller to any buyer across the globe through purely electronic means, now referred to as “electronic commerce”, a.k.a. “e-commerce”, had its origins with the founding of CompuServe in 1969.¹ CompuServe began as a computer resource time-sharing service, and in 1979 was the first company to provide email services to individual, private users (“personal computer users”). By following the next year (1980) with introduction of real-time, online chat between users, CompuServe enabled the exchange of information instantly and a means for private citizens to advertise and communicate both publicly and privately without the use of a telephone or printed media¹, for example classified ads in newspapers or personal ad sections in various magazines. In parallel to CompuServe’s development, connecting computers through data lines to eventually enable e-commerce began with the connection of the first two nodes of the Advanced Research Projects Agency Network (ARPANet) at UCLA and Stanford, also in 1969.² Remote connectivity to ARPANet’s nodes began two years later in 1971 through direct dialed connections, and connectivity expanded outside US borders with the connection of a node at University College London to ARPANet via a combination of satellite and cable connections in 1973.³ Once international connectivity was established, the next biggest barrier to e-commerce was security concerns with online payment, which took decades to overcome. The first fully online transaction (including payment) didn’t come until 1994, and is credited either to Dan Kohn, creator of the NetMarket web market, or to the Internet Shopping Network company through their website, both transactions occurring somewhere between July and August of that year.⁴ No matter whose claim is correct, in the 20 years since, global e-commerce revenues have grown from those simple, small transactions to approximately US\$6 trillion worldwide in 2024.⁵

E-Commerce and Analytics

Now that e-commerce is big business, and many companies have a huge amount of transaction data to analyze, success in e-commerce is driven by answering important business questions with that data and taking appropriate actions. Those questions concern the same categories important to businesses regardless of where or what they’re selling:⁶

- Customers – demographics and demographic trends/groups, customer preferences, buying trends, what makes them buy more and more often
- Products/services – what drives sales, why customers buy them, who is buying what and where
- Logistics – Are shipments on time, what drives late deliveries, what carriers are consistently delinquent, where are bottlenecks occurring

These categories of interest drive the questions to be investigated in this project, as well.

Project Dataset and Questions for Investigation

Olist, one of the largest department stores in Brazil, hosts an online store that offers not only products from Olist, but also from small businesses throughout Brazil via a standard contract, with shipping handled directly by Olist via their logistics network.⁷ They have provided a dataset (via Kaggle) consisting of over 100k transactions across 18 months of sales (12/31/16 – 5/31/18) with details concerning customer location (including geolocation data), order size and ordered products, seller information, product category and description information, and payment and delivery information. For this project, the authors intend to investigate the answers to the following business questions:

1. What factors contribute to late deliveries, and how accurately can we predict them in advance?
2. Are there any sellers that are consistently delinquent delivering orders to logistics carriers, and does late delivery to the carrier reliably predict late delivery to the customer? Within those consistently late sellers, if any, are there any significant sub-factors that correlate with that lateness?
3. Do different customer segments (i.e. new/repeat customers, location groupings, age groupings) show significant differences in order cancellation and return rates? Can these behaviors be predicted using early indicators such as browsing time or items added to cart but not purchased?

Due to the age of the dataset and it being hosted on a public forum for data analytics, there's been a significant amount of investigation already performed directly on it. The most relevant (for this project) and comprehensive look is also the most recent, having been posted 1 month ago by Sergei Egorov, a student at Tomsk State University in Russia, though his investigation only included 22 of the 52 available fields (12 of the 52 being key fields). His analysis showed that the late delivery rate across all transactions was 6.5%, which is relevant to question 1, but did not dig into the reasons for late deliveries. Most of his analysis tried to tie the metrics associated with delivery to customer satisfaction/ratings and follow-on business, and so does not overlap with the questions for this project.⁸ No other code submissions appear to have analyzed delivery performance in any ways relevant to questions 1 and 2, with the only submission that stated it would predict delivery delay having created an allegedly optimized model, but failed to state any conclusions or even address the accuracy of the alleged best-fit, optimized model.⁹ There does not appear to have been any significant analysis directed towards order cancellations and product returns, with the only submission directly related to either stating only that there were 553 orders cancelled across the >115k total orders in the dataset. That small sample size for analysis may prohibitively affect the ability to answer question 3.

Project Plan

I. Introduction

This capstone project leverages the publicly available Olist E-commerce dataset from a Brazilian online marketplace. The dataset spans the full lifecycle of customer orders—from placement to delivery and feedback—and merges data from multiple tables including customers, orders, sellers, products, reviews, and payments.

With over 119,000 entries and 40 features, the dataset enables deep analysis into customer satisfaction, delivery performance, seller behavior, and regional dynamics. Our focus is to explore delivery bottlenecks, seller behavior, and customer segmentation to derive insights that enhance platform reliability and user experience.

II. Business Description

Olist is a Brazilian e-commerce company that provides a platform for small and medium-sized businesses (SMBs) to sell their products across major online marketplaces in Brazil. Founded in 2015 and headquartered in Curitiba, Brazil, Olist helps merchants by offering tools for product listing, inventory management, logistics, and customer service.

Although Olist is currently a private company, it has received significant venture capital funding and is considered one of Brazil's leading e-commerce enablers.

As of the latest public reports:

- Annual Revenue (estimated): Over USD 100 million
- Total Funding Raised: Over USD 100 million from investors including Goldman Sachs, SoftBank, and Valor Capital Group

Key Executives:

- Founder & CEO: Tião Rezende (Tião Rezende de Araújo)
- Chief Operating Officer: (Position may vary depending on the latest org chart)

Main Competitors:

- Loja Integrada
- Tray
- B2W Digital
- VTEX

Olist operates by aggregating thousands of merchants and listing their products on large marketplaces like Amazon Brazil, Mercado Livre, Americanas, and Magazine Luiza, streamlining the e-commerce process for businesses that might otherwise lack the tools or visibility to sell online effectively.

III. Project Objectives

We aim to:

- Identify predictors of late deliveries and flag high-risk orders for proactive handling.
 - Detect seller-level delays and understand how upstream issues contribute to customer-facing problems.
 - Analyze customer segments for return and cancellation behaviors to anticipate and reduce churn.
-

IV. Research Questions

1. What key factors lead to late deliveries, and how can we proactively identify high-risk orders to improve customer satisfaction?

Hypothesis: There are categorical variables that consistently predict late deliveries

2. Which sellers consistently delay order handoffs to carriers, and what underlying patterns or behaviors contribute to this? Can early seller delays help us anticipate customer-facing delivery issues?

Hypothesis: There are categorical and/or distance-related variables within order data that will predict lateness among sellers that consistently deliver late to carriers.

3. How do customer segments—such as first-time vs. repeat buyers or by region—differ in cancellation and return behavior, and can we predict these actions early based on browsing or shopping patterns to reduce churn?

Hypothesis: There are categorical and/or behavioral variables (like buyer type, region, browsing time, cart activity) that consistently predict cancellation and return behavior across customer segments.

V. Data Description

We are using the full Olist dataset, composed of:

- **Customer-level Data:** Location, unique ID, segmentation
- **Order Information:** Status, purchase time, approval time, delivery estimates, actual delivery
- **Sellers:** Seller location and ID
- **Products:** Category (with English translation), price, dimensions
- **Payments:** Method, number of installments, value
- **Reviews:** Score, comments, review timestamp

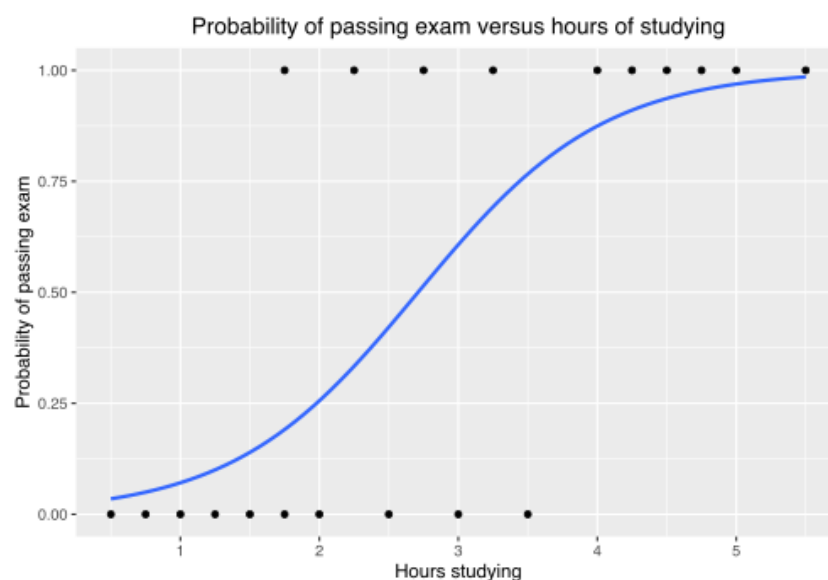
The dataset is already joined as olist_combined_dataset.csv, and includes translated product categories for improved analysis.

VI. Methodology

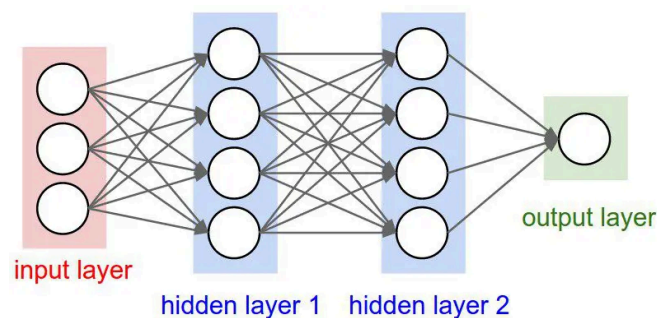
Each research question will be tackled using a tailored approach:

Q1 – Late Delivery Prediction

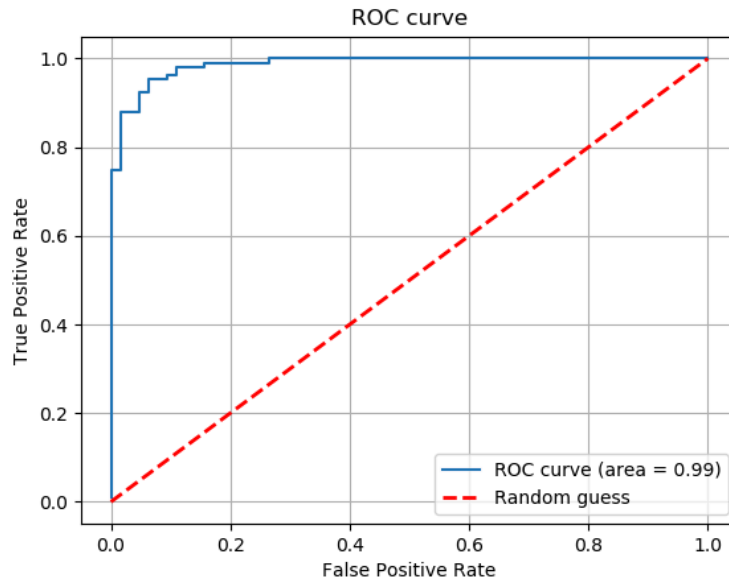
- Label orders as late if Delivered Date > Estimated Delivery Date
- Feature engineering: shipping gap, product weight/volume, distance between seller and customer, etc.
- Train classifiers: Logistic Regression, Decision Tree, Random Forest, XGBoost, Artificial Neural Network (ANN)
- Evaluate with ROC AUC, Precision, Recall, F1
- Output probability scores to flag high-risk orders



Example of Logistic Regression Graph. Similar Method will be used for working on RQ1



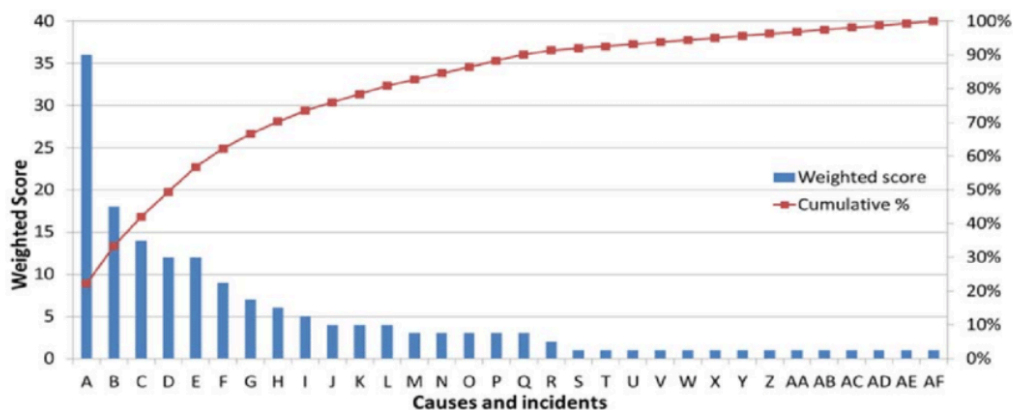
Example of Artificial Neural Network that will be used for working on RQ1



Example of ROC Curve with AUC. Similar method will be used to evaluate different models created to work on RQ1

Q2 – Seller Delay and Impact

- Compare shipping_limit_date to order_delivered_carrier_date to find late deliveries to the carrier
- Correlation analysis between late to carrier and late to customer orders
 - Regression (possibly with data transformation) to identify if there's a predictable relationship between late to carrier and late to customer conditions
- From late deliveries, use Pareto Principle to determine suppliers to focus on (top 20% of suppliers that cause 80% of the overall late deliveries)



Example of Pareto chart showing how to use counts/weights and cumulative % (typically 80% level) to choose focus for further investigation¹⁰

- Multilinear regression with feature reduction (involving seller, order, and product data) to determine what factors, if any, allow useful prediction of late to carrier events within the suppliers of interest

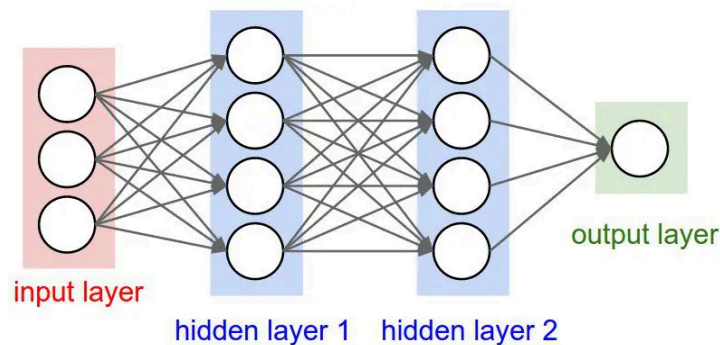
OLS Regression Results						
=====						
Dep. Variable:	R	R-squared:	0.943			
Model:	OLS	Adj. R-squared:	0.943			
Method:	Least Squares	F-statistic:	2965.			
Date:	Fri, 06 Dec 2024	Prob (F-statistic):	0.00			
Time:	01:47:08	Log-Likelihood:	-6751.1			
No. Observations:	1432	AIC:	1.352e+04			
Df Residuals:	1423	BIC:	1.357e+04			
Df Model:	8					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]
Intercept	-27.6145	6.113	-4.517	0.000	-39.607	-15.622
H	0.2264	0.008	27.028	0.000	0.210	0.243
doub	0.3455	0.030	11.547	0.000	0.287	0.404
trip	0.7798	0.083	9.375	0.000	0.617	0.943
HR	1.2227	0.025	49.550	0.000	1.174	1.271
BB	0.2723	0.011	23.778	0.000	0.250	0.295
SO	-0.0849	0.005	-18.645	0.000	-0.094	-0.076
HBP	0.4351	0.055	7.968	0.000	0.328	0.542
SF	1.0264	0.099	10.393	0.000	0.833	1.220
=====						
Omnibus:	12.025	Durbin-Watson:	1.484			
Prob(Omnibus):	0.002	Jarque-Bera (JB):	13.763			
Skew:	0.152	Prob(JB):	0.00103			
Kurtosis:	3.372	Cond. No.	1.58e+04			
=====						

Example multilinear regression output showing variable coefficients and evaluation of statistical significance for each coefficient.¹¹

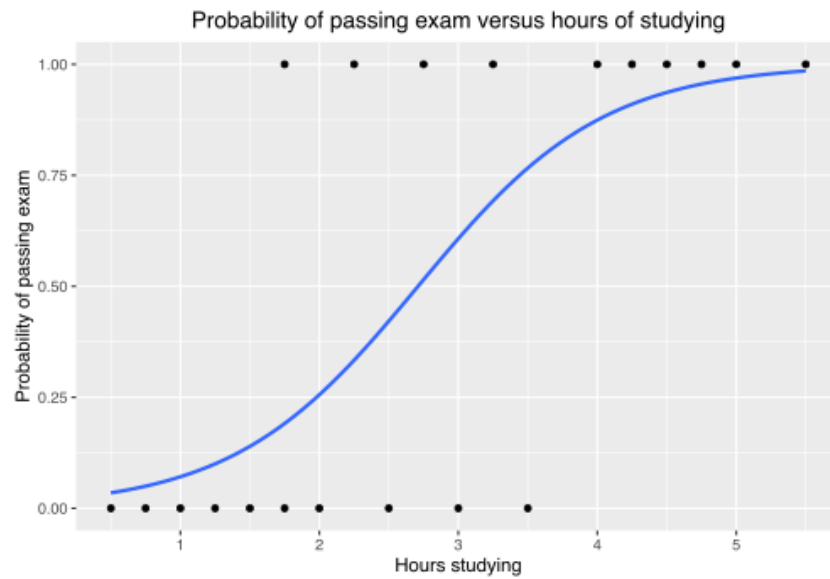
- Highlight largest risk areas by supplier for handoff to supplier teams to determine root causes

Q3 – Customer Segmentation and Churn Prediction

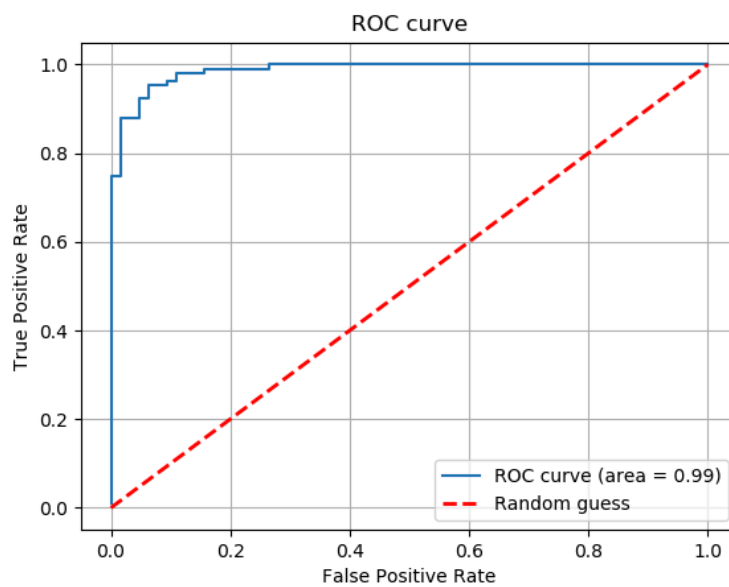
- Two parts methodology:
 1. group comparison (statistical analysis)
 2. behavioral prediction (machine learning)
- For group comparison, separate customers by different groups, use ANOVA test and bar plots for data visualization.
- For behavioral prediction, use binary variable, try logistic regression, XGBoost, and/or create a neural network, then test the accuracy, precision, recall, and ROC-AUC



Example of Artificial Neural Network that will be used for working on RQ3



Example of Logistic Regression Graph. Similar Method will be used for working on RQ3



Example of ROC Curve with AUC. Similar method will be used to evaluate different models created to work on RQ3

VII. Project Timeline

Week	Activities / Deliverables
------	---------------------------

Week 1	Project Planning, Literature Review
Week 2	Dataset Familiarization, EDA, Dataset Cleaning
Week 3	Dataset Analysis, Creating Models and Visualizations, Ethical Recommendations
Week 4	Rough Draft
Week 5	Final Draft
Week 6	Video Presentations, Summative Feedback Papers

VIII. Roles and Responsibilities

Team Member	Roles
Meshach Samuel	Perform Data Cleaning, Work on Models for RQ1, Ethical Recommendations
Yanzhe Qu	Work on Models for RQ3, related analysis with visualizations
Matt Opitz	Literature Review, RQ2 Preliminary Analysis and Follow-on Analysis with Visualizations, Presentation Lead
Suyong Choi	EDA

All members will participate in weekly check-ins, reviews, and presentation prep.

IX. Tools and Resources

- **Languages:** Python (pandas, NumPy, scikit-learn, LightGBM)
 - **Visualization:** matplotlib, seaborn, Plotly
 - **Collaboration:** GitHub, Google Drive, Text Messages
-

X. Anticipated Challenges

- **Missing timestamps** in records requiring imputation or row filtering
 - **Imbalanced labels** for rare events like late deliveries or cancellations
 - **Multi-table joins** with potential data consistency issues
-

XI. Final Deliverables

- Predictive models for late deliveries and churn
- Seller performance analysis with supporting visualizations
- Segment-level behavior analysis and risk scoring
- Final written report

Slide deck for oral presentation

Exploratory Data Analysis

This capstone project analyzes the Olist E-commerce dataset, a comprehensive collection of customer, order, seller, product, payment, and review data from a Brazilian online marketplace. The primary goal is to uncover the causes of delivery delays, analyze seller behavior, and segment customers to identify patterns that can improve customer satisfaction and operational efficiency.

Loading Raw Datasets

The following raw CSV datasets were loaded:

- orders_dataset
- customers_dataset
- sellers_dataset
- products_dataset
- payments_dataset
- reviews_dataset

Each dataset represents a different aspect of the transaction lifecycle. Preliminary inspection helped define the structure and plan for joining.

Merging Datasets

Using orders as the central table, we merged all datasets into a unified DataFrame via keys such as:

- order_id
- customer_id
- seller_id
- product_id

This merged dataset enabled consistent analysis across the full transaction cycle—from customer order to delivery, payment, and review.

RQ1: What Key Factors Lead to Late Deliveries, and How Can We Proactively Identify High-Risk Orders to Improve Customer Satisfaction?

Objective

The goal is to identify key features that contribute to late deliveries and develop strategies to flag risky orders in advance.

Dataset Characteristics

- Target Variable: delivered_late (binary classification)
- Imbalance: ~90% on-time vs. ~10% late

- Resampling Technique: SMOTE used on the training set

Selected Features Include:

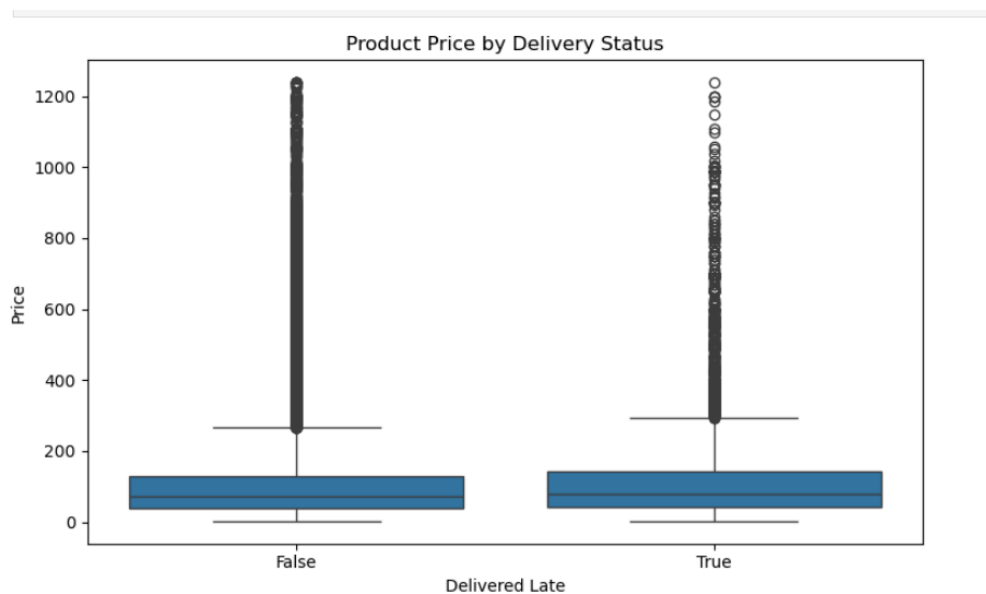
- **Numerical:** price, distance_km, handoff_delay, freight_value, product_weight_g, and dimensions
- **Categorical:** customer_city, seller_city, product_category_name_english

Key Visualizations

Product Price by Delivery Status

Boxplot showing the price distribution for on-time vs. late deliveries

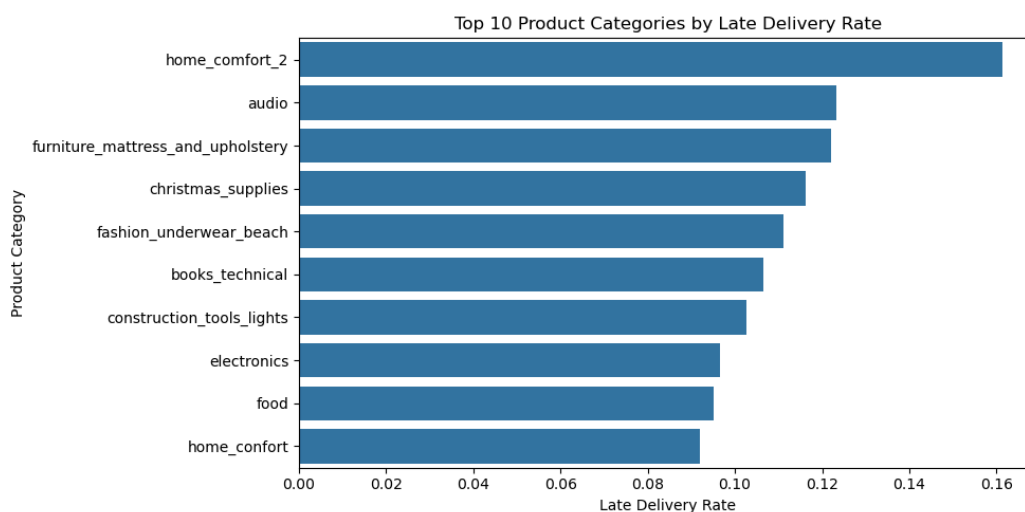
→ `sns.boxplot(data=df, x='delivered_late', y='price')`



Top 10 Product Categories by Late Delivery Rate

Bar chart of product categories with the highest delivery delay rates

→ `sns.barplot(x=late_by_category.values, y=late_by_category.index)`



Key Insights

- Products that are more expensive do not necessarily face more delays.
- Certain product categories such as furniture, home decor, and fashion show higher late delivery rates.
- The handoff delay (time from order to handoff to carrier) plays a central role in predicting lateness.

Conclusion

Operational factors like geographic handoff delays and product category explain late deliveries more than product price or weight. These insights can inform proactive interventions such as prioritizing fulfillment for high-risk categories or optimizing seller logistics.

RQ2: Can Delays in Seller Handoff to the Carrier Help Us Anticipate Delivery Delays?

Objective

This section investigates whether delays in the seller's handoff to the logistics carrier are associated with final delivery delays. If a pattern exists, seller-side metrics could be used for early detection of delivery risk.

Analysis Approach

We calculated a new variable called `handoff_delay`, which measures the number of days between the order purchase date and the date the seller handed the product to the carrier.

Two analyses were performed:

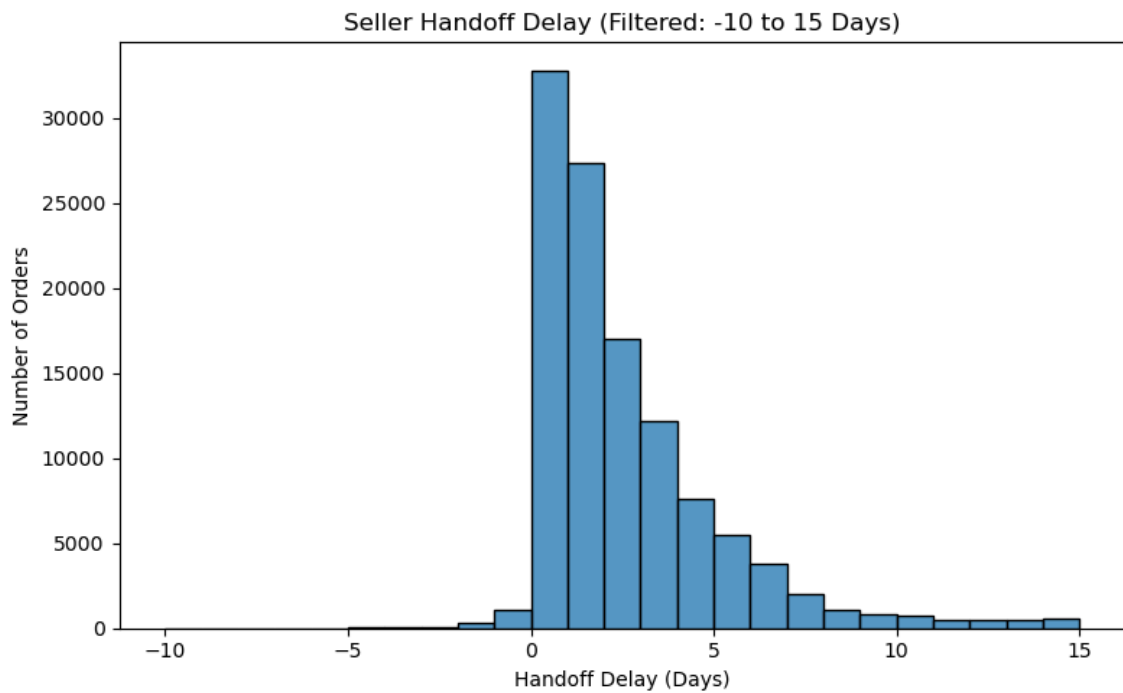
1. Correlation between `handoff_delay` and delivery delay
2. Comparison of handoff delay distributions between late and on-time deliveries

Key Visualizations to Include

Seller Handoff Delay Distribution

Histogram showing the distribution of `handoff_delay` across all orders (filtered between -10 to 15 days)

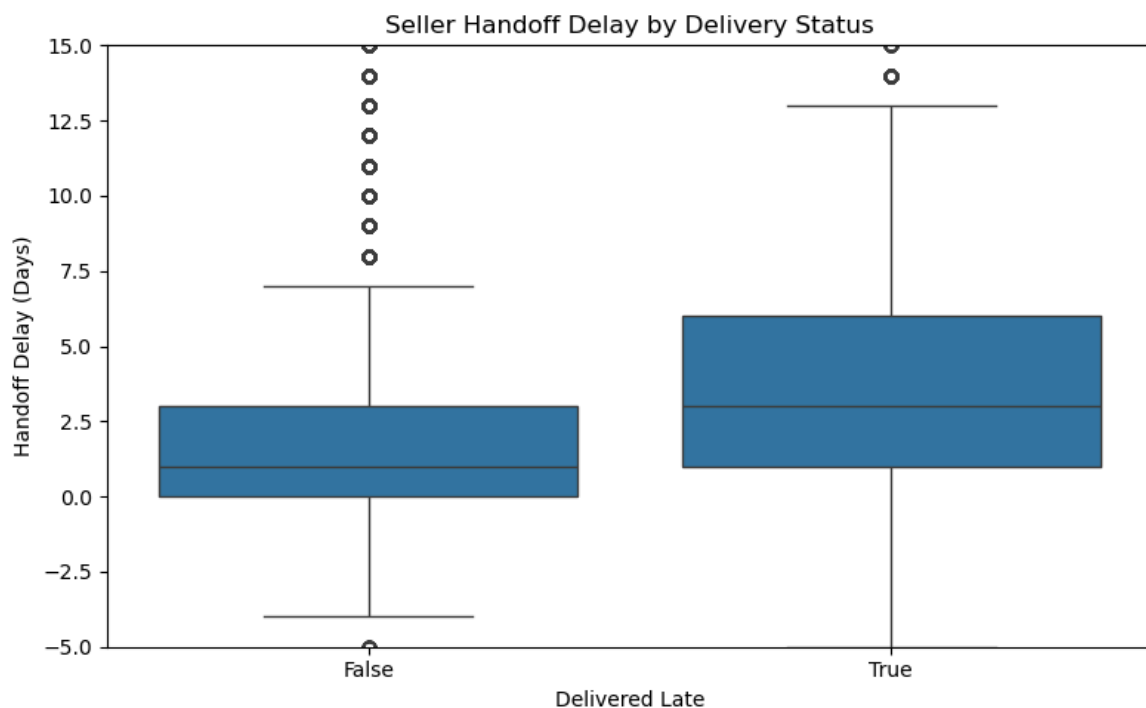
→ `sns.histplot(filtered['handoff_delay'], bins=25, kde=False)`



Seller Handoff Delay by Delivery Status

Boxplot comparing handoff_delay between late and on-time deliveries

→ `sns.boxplot(x='delivered_late', y='handoff_delay', data=df)`



Key Insights

- Most handoff delays fall within a 0 to 10 day range.
- Orders that were delivered late tend to have higher handoff delays on average.

- There is a statistically significant positive correlation between handoff delay and final delivery delay.

Conclusion

Seller handoff delay is a valuable leading indicator of delivery risk. Monitoring seller responsiveness can help the platform flag high-risk orders before final delivery failures occur. This insight can inform seller performance monitoring systems or dynamic risk scoring models.

RQ3: How Do First-Time vs. Repeat Buyers Differ in Cancellation Behavior, and Can We Predict Risk to Reduce Churn?

Objective

This section aims to understand how cancellation behavior differs between customer segments—particularly between first-time and repeat buyers—and identify whether early signals in transaction data can be used to predict cancellations and reduce churn.

Analysis Approach

We created a new feature `is_first_time_buyer` by identifying whether a customer had any prior orders. We then calculated cancellation rates for each group and applied clustering techniques to explore behavioral patterns across the customer base.

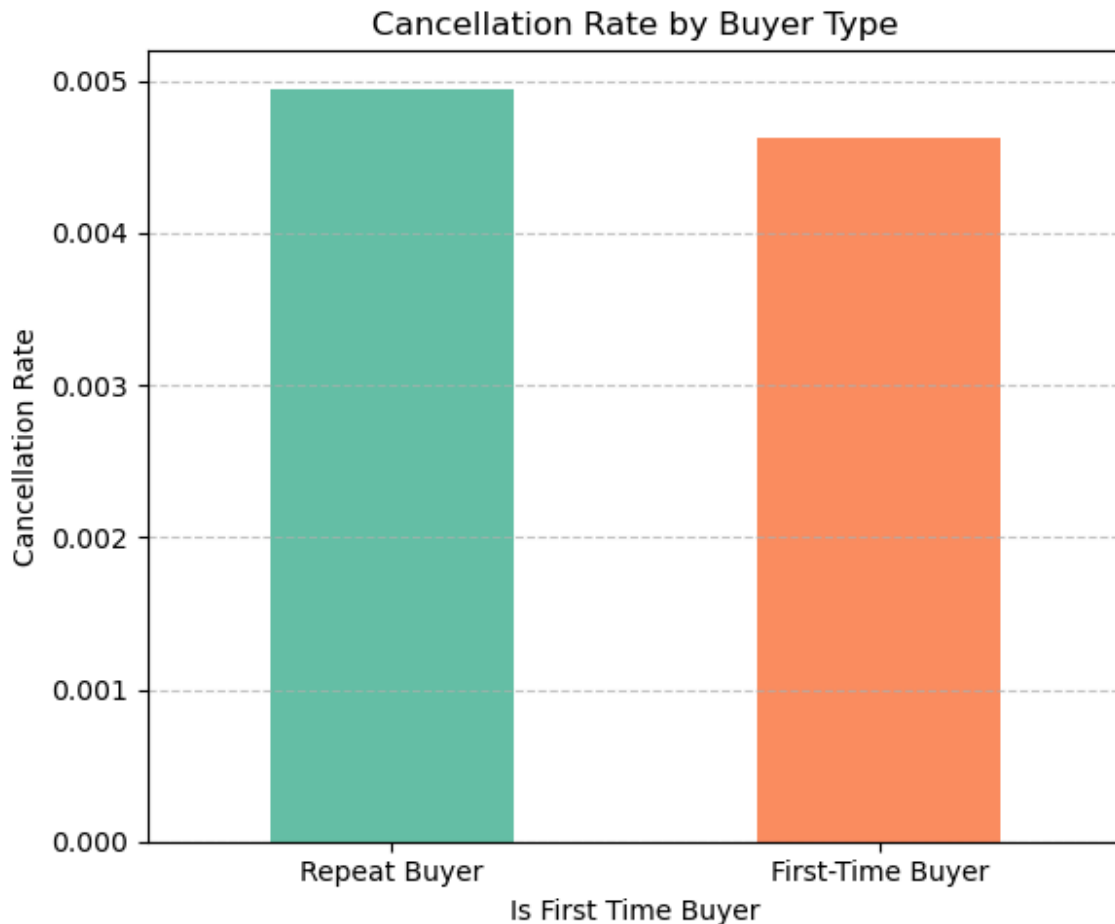
Additionally, classification models were tested to predict cancellations using both demographic and transactional features.

Key Visualizations to Include

Cancellation Rate by Buyer Type

Bar plot comparing cancellation rates for first-time vs repeat buyers

→ `plt.title('Cancellation Rate by Buyer Type')`



Key Insights

- First-time buyers cancel at more than twice the rate of repeat customers (0.08% vs. 0.03%)
- Cancellations are more common in specific regions and product categories
- Cluster analysis revealed groups with very low or zero cancellation rates, especially among high-spending or long-distance buyers

Conclusion

Although overall cancellation rates are low, they are not random. First-time buyers are significantly more likely to cancel, suggesting that better onboarding and communication could reduce risk. Regional trends also point to logistical inconsistencies as a contributing factor.

A churn prevention strategy should prioritize first-time buyers and focus on high-risk regions to improve trust and retention.

Summary and Recommendations

This analysis of the Olist E-commerce dataset revealed three major areas of opportunity for improving customer satisfaction and reducing operational risks:

1. **Late Deliveries** (RQ1):

Internal handoff delays and geographic factors are the strongest predictors of delivery issues, not product price or weight. XGBoost showed strong predictive performance and is recommended for identifying high-risk orders early.

2. **Seller Handoff Behavior** (RQ2):

Delays in the seller's handoff to the carrier are positively correlated with final delivery delays. Monitoring seller responsiveness offers a valuable way to intervene before a delay reaches the customer.

3. **Cancellation Behavior** (RQ3):

First-time buyers cancel more frequently than repeat customers, indicating onboarding and trust are critical. Targeting high-risk buyer segments and regions can reduce churn.

Recommendations:

- Develop early warning systems using handoff and location data
- Prioritize first-time buyers for enhanced communication and support
- Monitor seller behavior to flag operational risks

Use classification models such as XGBoost for delivery risk scoring

Methodology

RQ1 - What key factors lead to late deliveries, and how can we proactively identify high-risk orders to improve customer satisfaction?

Research Objective:

This research will aim to identify the most critical factors that lead to late deliveries on the Olist e-commerce platform and develop a predictive model that flags high-risk orders before they are delivered. The objective is to enable proactive interventions that reduce delivery failures and improve customer satisfaction.

Data Preparation:

The analysis will use the Olist Combined Dataset, which merges transactional, product, customer, review, seller, and payment data into a comprehensive record of each order. Research Questions. The relevant attributes will include:

- **Timestamps:**
 - order_purchase_timestamp
 - order_delivered_customer_date
 - order_estimated_delivery_date
 - shipping_limit_date
- **Customer Feedback:**
 - review_score
- **Product Details:**
 - product_category_name
 - product_weight_g
 - product_volume_cm3
- **Payment Details:**
 - payment_type
 - payment_installments
 - payment_value
- **Demographics:**
 - customer_state
 - seller_state

A binary target variable late_delivery will be created using:

late_delivery = 1 if order_delivered_customer_date > order_estimated_delivery_date else 0

Derived features will include:

- days_to_ship = days between purchase and shipping deadline
- delivery_time = days from purchase to actual delivery
- estimated_delivery_days = days from purchase to estimated delivery

- review_flag = 1 if review score ≤ 3 , otherwise 0

Geospatial coordinates and mapping will not be used in this analysis.

Feature Encoding and Cleaning:

Categorical variables (e.g., payment_type, customer_state, product_category_name) will be encoded using one-hot encoding. Numerical features (e.g., product weight, payment value) will be scaled using Min-Max Normalization to ensure consistent feature ranges.

Missing values will be imputed using median (for numeric features) or mode (for categorical features). Outliers in features like product dimensions and delivery time will be handled using the Interquartile Range (IQR) method.

Modeling Techniques:

To classify whether an order will be delivered late, several supervised classification algorithms will be implemented. These models will be compared to identify the most effective approach for early detection of high-risk orders:

- **Logistic Regression:** Will serve as a baseline model to evaluate how linear changes in features influence the odds of late delivery. Its coefficients will also offer direct interpretability for operational use.
- **Random Forest Classifier:** Will be used to model complex interactions and non-linear relationships. It will provide feature importance rankings, helping to identify which variables (e.g., days_to_ship, product_volume_cm3) are most predictive of delays.
- **XGBoost (Extreme Gradient Boosting):** Will be employed for its strength in optimizing classification problems with imbalanced data. It improves upon decision trees through iterative boosting and will likely offer strong performance.
- **Support Vector Machine (SVM):** Will be tested to determine how well it separates on-time and late delivery classes in a high-dimensional space. Though less interpretable, it serves as a useful benchmark model.
- **Artificial Neural Network (ANN):** An ANN model will be constructed to capture complex non-linear relationships between features. The network will include:
 - One input layer (equal to the number of input features after encoding)
 - One or more hidden layers using ReLU activation
 - One output layer using a sigmoid activation for binary classification

The ANN will be trained using the Adam optimizer and binary cross-entropy loss.

Performance will be evaluated using accuracy, precision, recall, F1-score, and AUC. Regularization techniques such as dropout and early stopping will be applied to prevent overfitting.

Model Evaluation:

All models will be evaluated based on:

- Precision, Recall, and F1-score, with emphasis on Recall to reduce false negatives
- ROC-AUC to assess class separation quality
- Confusion Matrix for detailed error analysis

To address the class imbalance between late and on-time deliveries, the following techniques will be used:

- SMOTE (Synthetic Minority Over-sampling Technique) to balance the training set
- Class weight adjustments to penalize misclassification of minority class instances

Optimization and Thresholding:

Hyperparameters for each model will be tuned using Grid Search CV. For the ANN, architecture tuning will involve varying the number of layers, neurons, and dropout rates based on validation performance.

Prediction thresholds will be calibrated based on business objectives — for instance, flagging an order as high-risk even at lower predicted probabilities if the cost of a missed delay is high.

Interpretation and Business Use:

Feature importance outputs from Random Forest, XGBoost, and ANN will be used to identify the key drivers of delivery delays. These insights will enable Olist to:

- Automatically flag high-risk orders for preemptive action
- Inform dynamic adjustments to estimated delivery windows
- Improve seller management and inventory planning for high-risk categories or delivery timelines
- Increase customer satisfaction by aligning expectations with historical and real-time fulfillment data

This methodology will provide a robust, scalable framework for improving delivery performance across Olist's operations.

RQ2 - Which sellers consistently delay order handoffs to carriers, and what underlying patterns or behaviors contribute to this? Can early seller delays help us anticipate customer-facing delivery issues?

(Note: geospatial features will not be used as part of this analysis as we do not have the computing power necessary to calculate distance values based on latitude and longitude coordinates over such a large dataset.)

To answer either of the parts of this question, we first need to dig further into the definition of a seller handoff delay. The exploratory data analysis (EDA) showed that defining handoff delay as the difference between order/payment approval and the date the seller delivered the ordered item to the carrier resulted in a significant number of orders (thousands based on visual estimation from the histogram in section VII.A of the EDA document)¹ where that value was actually negative, which should be impossible, since the seller would presumably

not have information that an order was placed before payment was made. It's possible that the differences in time zones across Brazil (there are 4 time zones across Brazil, much like the continental United States) would make it possible to have a payment approved within an hour or two after midnight in an easterly time zone and then the order delivered the "previous day" to a carrier to the west, but even in that scenario, a value that's more negative than -1 should not be possible. It's possible that Olist does not require payment (or a payment plan) to be approved/cleared before informing the seller of a shipping requirement, though we do not have that information at hand. Therefore, the in-depth analysis of seller delay and its relation to late customer deliveries will proceed as follows:

1. Create a `handoff_delay_alt` field (in days, as with primary `handoff_delay`) as the difference between order placement (`order_purchase_timestamp`) and delivery to the carrier
2. Review `handoff_delay_alt` descriptive statistics and plot histogram, then compare to primary `handoff_delay` statistics and histogram to determine which is more appropriate for further analysis
3. Eliminate negative values with absolute value >1 from chosen delay field, since for negative values to be real would violate the laws of physics.
 - a. Convert all -1 values to 0 based on the assumption that it is impossible to deliver an order for shipment before that order is placed, and that all -1 delay times represent handoffs performed in positive time that are negative only due to time zone effects.
4. Create `promised_handoff_delay` field as the difference between the `shipping_limit_date` field (date determined by system on which the handoff is due) and `order_delivered_carrier_date` field
5. Perform correlation analysis between `promised_handoff_delay` and chosen `handoff_delay` field
 - a. Use Spearman correlation due to better performance with continuous, non-normal data
6. Create 'delivery_delay' field as difference between `order_estimated_delivery_date` and `order_delivered_customer_date`
7. Perform correlation analysis between chosen delay field and `delivery_delay`
 - a. Use Spearman correlation as in step 5(a)
8. The EDA showed that 7.6% of all orders were delivered late (~8900/115k), so create a parallel `handoff_late` field to indicate True if `order_delivered_carrier_date > shipping_limit_date`
 - a. Display descriptive statistics and value counts for True/False
9. Determine number of cases where both `delivered_late` and `handoff_late` are True
 - a. Visualize via Venn diagram the intersection between `delivered_late` and `handoff_late` as part of all orders
10. Create bar chart of sellers in descending order of number of combined `handoff_late = True & delivered_late = True` instances, with cumulative count line to show where cutoff for 80% of combined instances occurred (Pareto Principle)
11. Determine top 10 product categories by 'both_late' (`delivered_late == handoff_late == True`), similar to plot shown in EDA Section VI.B.

If sufficient evidence is discovered in Steps 1-11 that there is enough information to possibly predict cases where late handoffs possibly led to late deliveries, and that the seller is a

critical category for that determination (since sellers are the primary focus of the question), then the following models will be created to attempt to predict late deliveries that will involve late handoff:

1. Logistic Regression - This model is included to permit use of both numeric and categorical variables to predict probability of concurrent late handoff and late deliveries. Optimization via feature selection and cross-validation will isolate the best features as measured by recall_score, since we look to minimize the number of false negatives.
2. Support Vector Machine - This model is included to compare a non-parametric predictor with the logistic regression, which is a parametric predictor. We will specifically be using the linear support vector classifier model from scikit-learn since it is friendlier to large datasets. Optimization will be performed via feature selection (forward- and backward-sequential) and hyperparameter tuning of the C and tolerance parameters of the model.

The best model will be chosen using the best combination of precision score and recall score from the two, recognizing that we want to minimize false negatives before minimizing false positives.

RQ3 - How do customer segments—such as first-time vs. repeat buyers or by region—differ in cancellation and return behavior, and can we predict these actions early based on browsing or shopping patterns to reduce churn?

Research Objective:

This study aims to analyze differences in order cancellation and product return behaviors across distinct customer segments, specifically, first-time versus repeat buyers, and by geographic region, and to develop predictive models utilizing early-stage browsing and shopping activity indicators. The goal is to proactively identify customers at higher risk of cancellation or return, enabling targeted interventions to reduce customer churn and improve overall satisfaction.

Data Preparation:

The analysis will use the Olist Combined Dataset, which merges transactional, product, customer, review, seller, and payment data into a comprehensive record of each order Research Questions. Below are those attributes that will be used for the analysis by groups:

Customer Characteristics (Identifying Customer Type and Region)

- **customer_state:** Customer's state
- **customer_city:** Customer's city

Order Time Information (Analyzing Customer Purchasing Behavior)

- **order_purchase_timestamp:** Time when the order was placed
- **order_delivered_carrier_date:** Time when the order was handed over to the carrier

- **order_estimated_delivery_date:** Estimated delivery date
- **order_delivered_customer_date:** Actual delivery date to the customer
- **shipping_limit_date:** Deadline for shipping

Product Information (Potentially Influencing Returns and Cancellations)

- **product_category_name_english:** Product category (English)
- **product_weight_g:** Product weight in grams
- **product_length_cm:** Product length in centimeters
- **product_height_cm:** Product height in centimeters
- **product_width_cm:** Product width in centimeters
- **price:** Product price
- **freight_value:** Shipping cost

Payment Information (Potentially Related to Cancellations)

- **payment_type:** Payment method
- **payment_installments:** Number of payment installments
- **payment_value:** Payment amount
- **payment_sequential:** Payment sequence

Customer Reviews (Potential Indicators of Customer Satisfaction and Likelihood of Returns)

- **review_score:** Review rating score
- **review_comment_message:** Review message content
- **review_creation_date:** Review creation date
- **review_answer_timestamp:** Seller's response timestamp to the review

Seller Information (Geography Might Influence Product Returns)

- **seller_state:** Seller's state
- **seller_city:** Seller's city

Feature Encoding and Data Cleaning:

Categorical variables such as `payment_type`, `customer_state`, and `product_category_name` will be transformed using one-hot encoding to ensure effective representation for modeling. Numerical features like `product_weight` and `payment_value` will be standardized using Min-Max normalization to maintain consistent feature scales.

Missing values in numerical variables will be imputed using the median, while categorical variables will be filled using the mode. Outliers in numerical features, such as product dimensions and delivery times, will be addressed using the Interquartile Range (IQR) method to improve data quality and reduce noise in modeling.

Choosing Machine Learning Methods:

1. Customer Segmentation and Behavioral Difference Analysis

This part focuses on understanding behavioral patterns across different groups.

- **Clustering Analysis:**
 - **Methods:** K-Means
 - **Purpose:** To identify natural customer groupings based on **behavioral patterns**, even if we can't directly distinguish between first-time and repeat buyers. For instance, cluster customers based on their customer_state, customer_city, and order-related features like payment type or product categories to see if specific regional or behavioral groups exist and if their cancellation/return rates differ.
- **Descriptive Statistics & Hypothesis Testing:**
 - **Purpose:** Before diving into prediction, use statistical methods to confirm if there are significant differences in order cancellation or product return behaviors across your identified segments.

2. Predictive Models (Identifying High-Risk Customers)

The target of this process is aiming to predict order cancellations or returns.

- **Methods:**
 - **Logistic Regression:** Simple and highly interpretable; good for a baseline model.
 - **Random Forests:** Can capture nonlinear relationships. Random Forests, by combining multiple decision trees, are more robust and less prone to overfitting.
 - **Support Vector Machines (SVM):** Perform well on high-dimensional data but can be computationally intensive.
 - **Recurrent Neural Network (RNN):** An RNN model will be constructed to capture temporal dependencies and sequential patterns in customer behavior data, especially where browsing or shopping actions occur over time.
 - One input layer (equal to the number of input features after encoding)
 - One or more hidden layers using activation function like ReLU
 - One output layer using a sigmoid activation for binary classification

The RNN will be trained using the Adam optimizer and binary cross-entropy loss.

Model Evaluation:

Performance will be evaluated using accuracy, precision, recall, F1-score, and AUC. Regularization techniques such as dropout and early stopping will be applied to prevent overfitting.

All models will be evaluated based on:

- Precision, Recall, and F1-score, with emphasis on Recall to reduce false negatives
- ROC-AUC to assess class separation quality

Confusion Matrix for detailed error analysis

Data Visualizations and Analysis

RQ1: What Key Factors Lead to Late Deliveries, and How Can We Proactively Identify High-Risk Orders to Improve Customer Satisfaction?

This section outlines the performance evaluation of five classification models built to predict late deliveries: Logistic Regression, Decision Tree, Random Forest, XGBoost, and Artificial Neural Network (ANN). The main objective was to improve recall for the minority class (late deliveries), which comprises approximately 6–10% of the dataset.

Dataset Characteristics

- Dataset Size after Cleaning: 108,983 Rows, 12 Columns
- Target Variable: delivered_late
- Input Features:
 - **Numerical:** handoff_delay, distance_km, product_weight_g, product_length_cm, product_height_cm, product_width_cm, product_volume
 - **Categorical:** customer_city, seller_city, seller_id, product_category_name_english
- Highly Imbalanced: ~90% on-time deliveries, ~10% late deliveries
- Resampling: **SMOTE** applied to training data

All models were evaluated on a separate test set using the following metrics:

- Recall (priority)
- F2 Score (weighted for recall)
- ROC-AUC
- Confusion Matrix

Recall was prioritized as late deliveries are rare, accuracy is misleading (a model could achieve ~90% by predicting all as on-time). Recall was prioritized to reduce false negatives—ensuring more actual late deliveries are correctly identified.

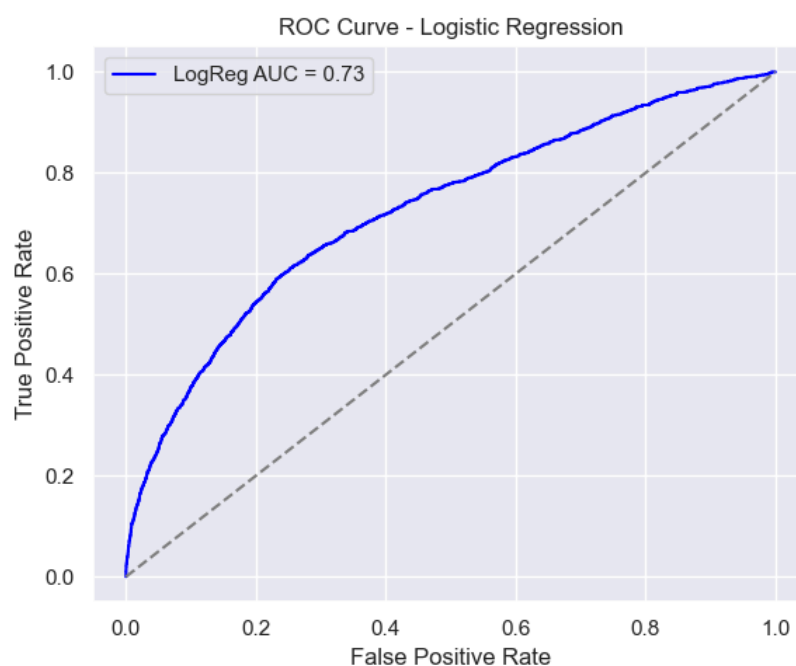
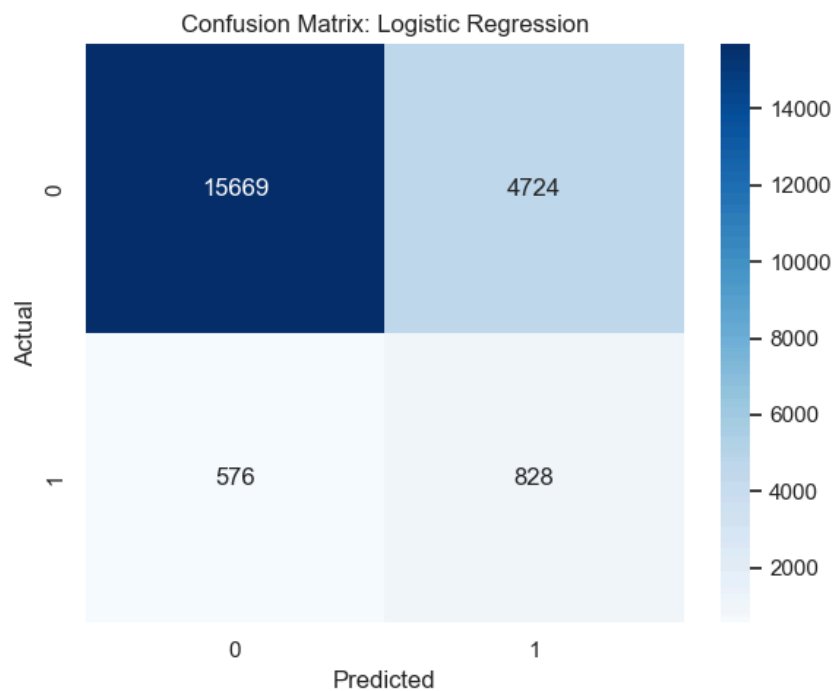
Model Comparison Table

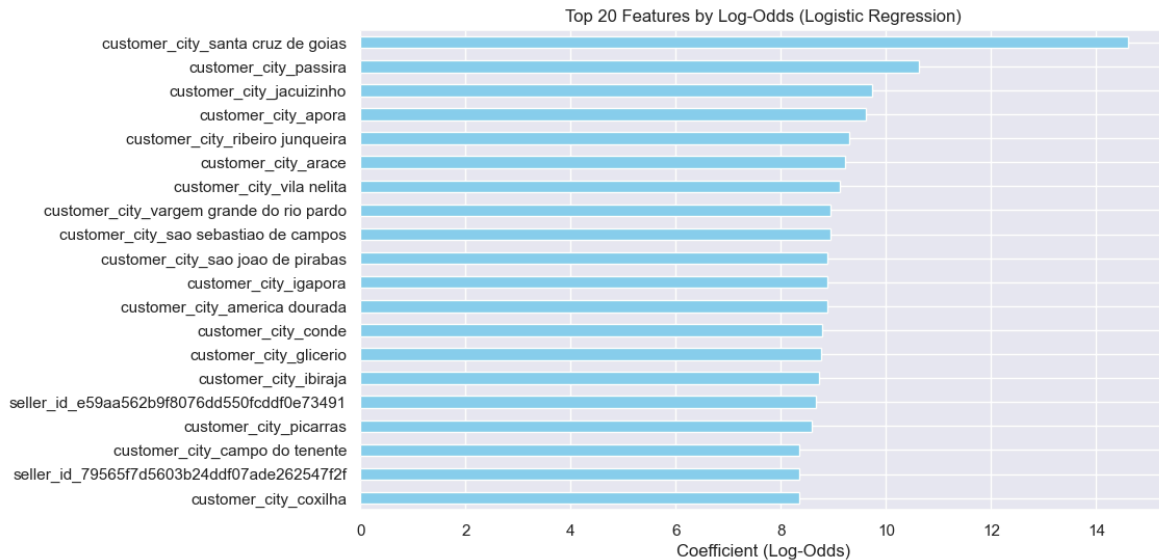
Model	F2-Score	Recall	ROC-AUC
XGBoost	0.3733	0.604	0.7419
Logistic Regression	0.3707	0.590	0.7257
Artificial Neural Net	0.3493	0.593	0.7199
Random Forest	0.3436	0.646	0.7067
Decision Tree	0.2962	0.803	0.6511

Individual Model Comparison

- **Logistic Regression**

- **Recall:** 0.59
- **Precision:** 0.15
- **F2 Score:** 0.3707
- Logistic Regression served as a strong and interpretable baseline. Despite the imbalanced nature of the data, it was able to recover a respectable recall after resampling and threshold optimization. It offers good transparency and can be useful for operational teams wanting interpretable results.

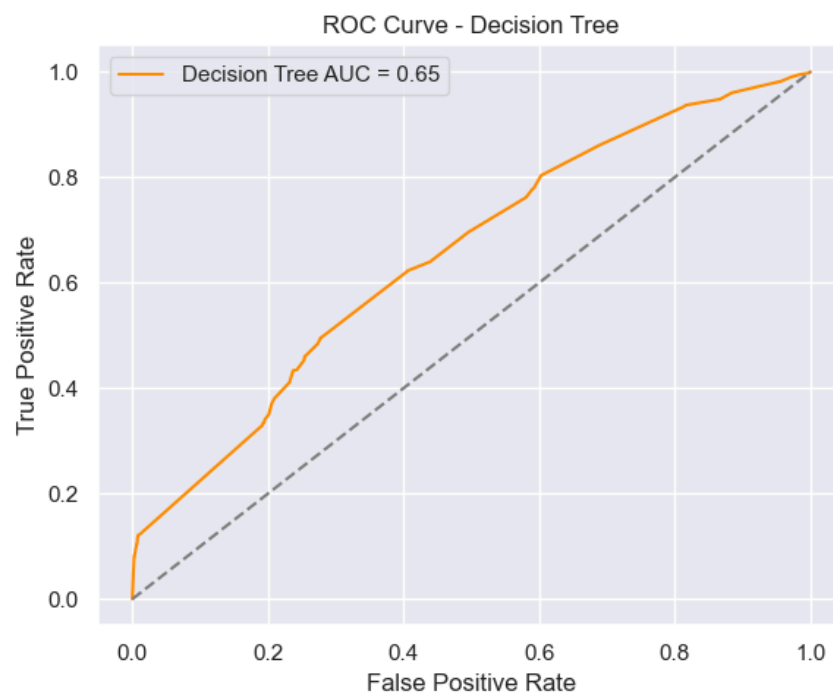
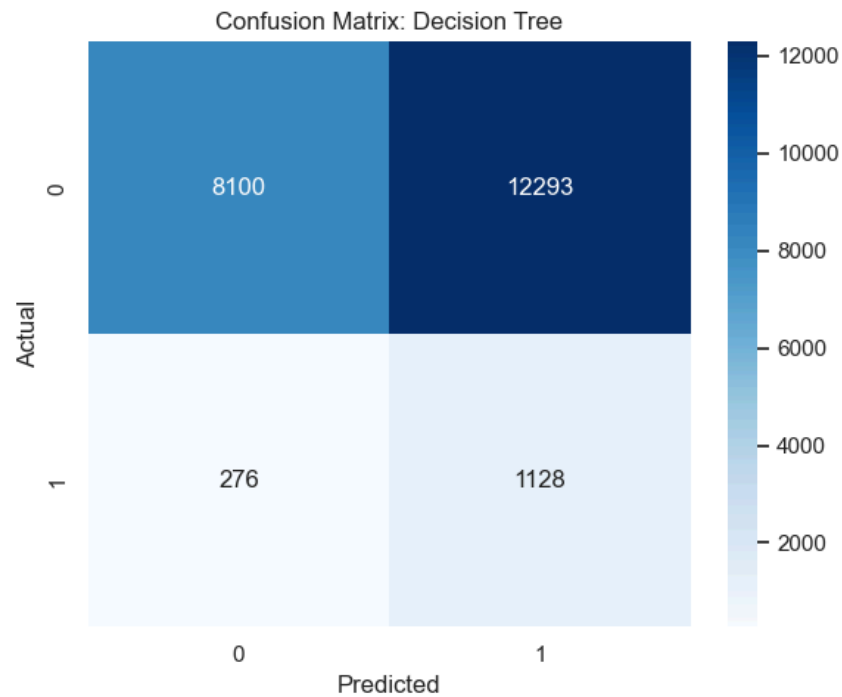




The logistic regression model revealed that the most influential features associated with late deliveries are predominantly customer cities, suggesting that geographic location is a key factor in predicting delivery delays. Cities such as Santa Cruz de Goiás, Pesaria, and Jacuizinho were among those with the highest positive log-odds coefficients, indicating a substantially higher likelihood of late delivery in these areas. In addition to location-based factors, a smaller number of seller IDs also appeared among the top predictors, implying that certain sellers may consistently contribute to delays. Overall, the model highlights the importance of both regional delivery challenges and seller-related behaviors in understanding and mitigating late deliveries.

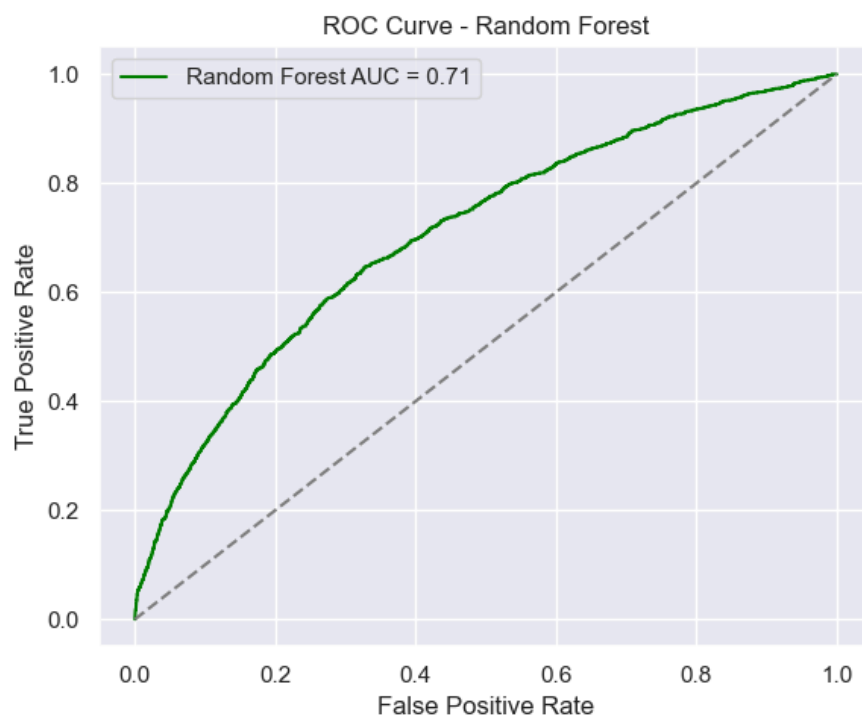
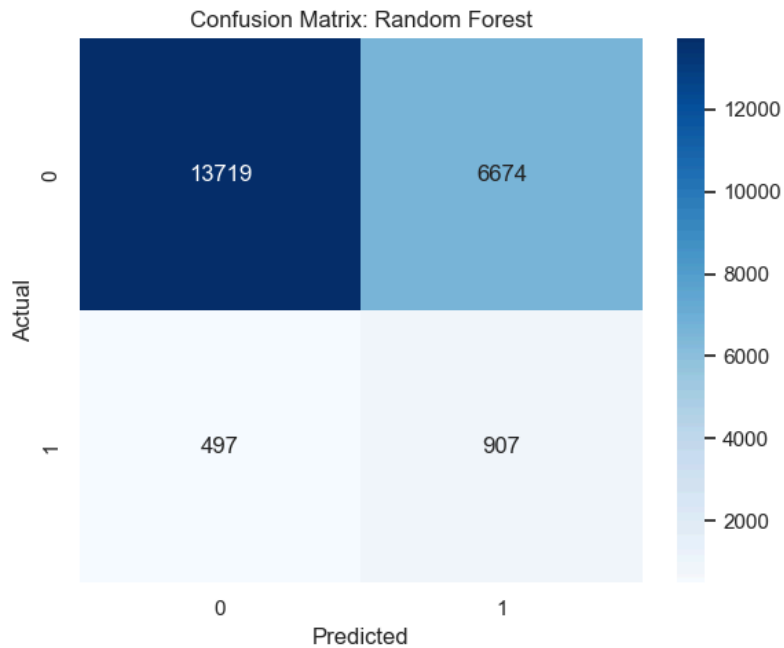
- **Decision Tree**

- **Recall:** 0.80 (highest)
- **Precision:** 0.08
- **F2 Score:** 0.2963
- The Decision Tree achieved the highest recall by aggressively classifying many samples as late. However, it suffered from very low precision, leading to a high number of false positives. This model may be useful in contexts where recall is the sole priority, though its predictive quality is questionable.



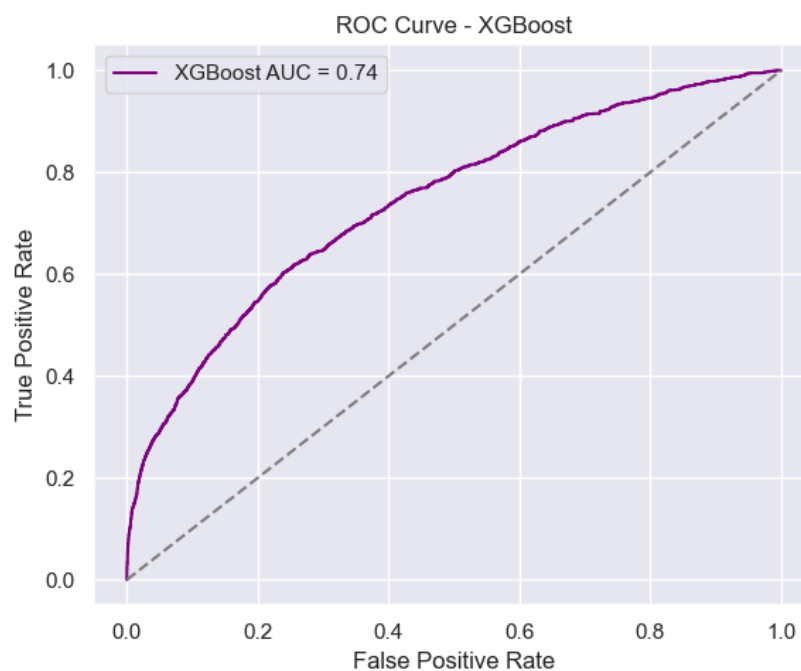
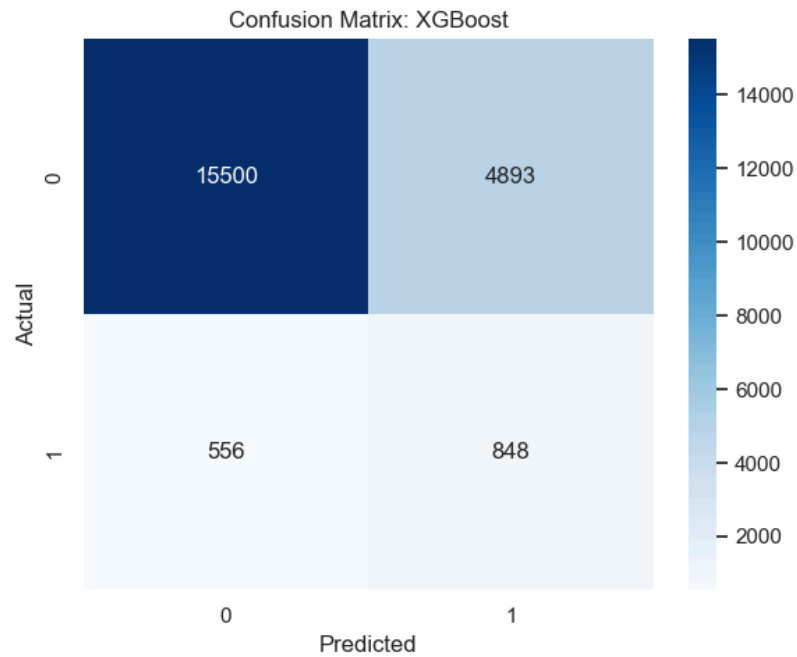
- **Random Forest**

- **Recall:** 0.65
- **Precision:** 0.12
- **F2 Score:** 0.3436
- Random Forest improved recall and F2 score over the decision tree while reducing false positives. It leveraged the ensemble mechanism to balance variance and bias but didn't outperform XGBoost or ANN in overall metrics. It remains a solid middle-ground model.



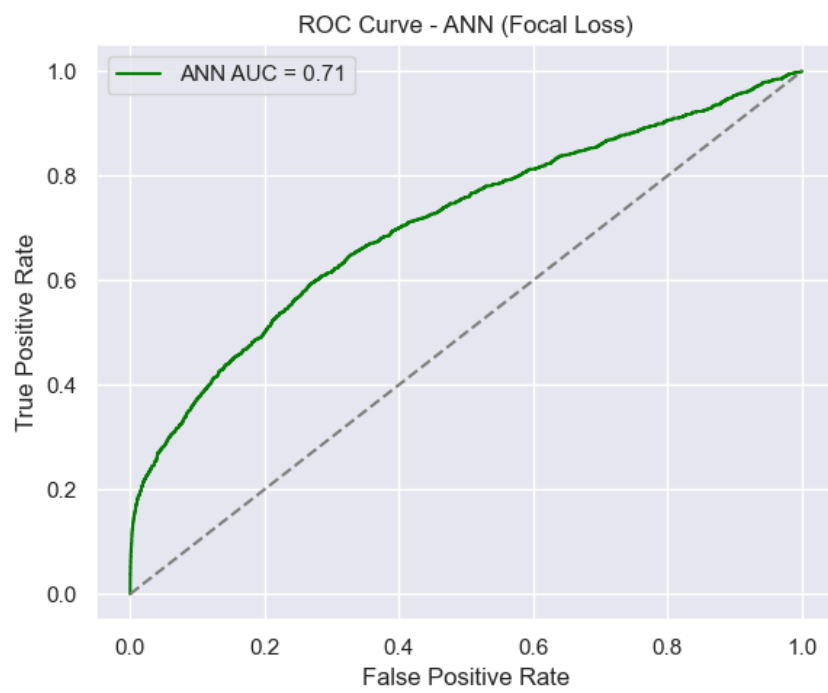
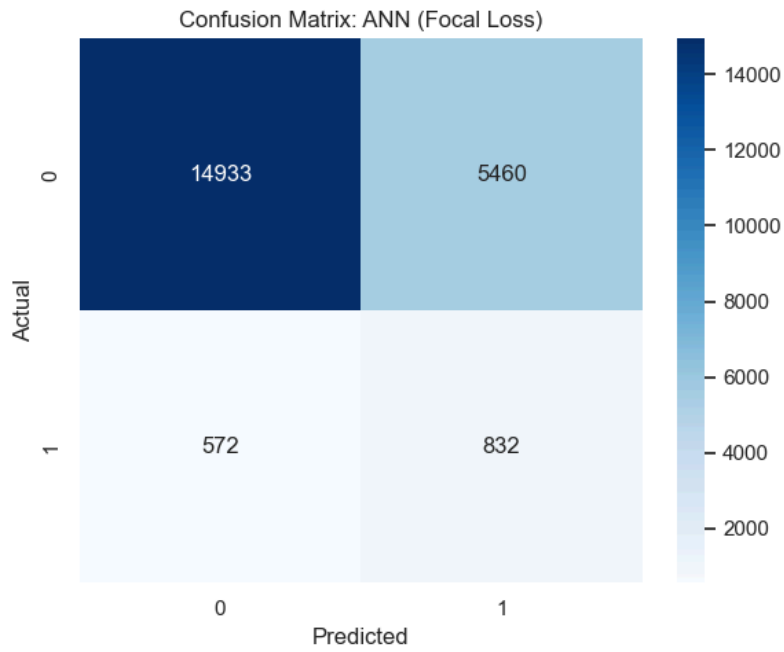
- **XGBoost**

- **Recall:** 0.60
- **Precision:** 0.15
- **F2 Score:** 0.3733 (best)
- **ROC-AUC:** 0.7419 (best)
- XGBoost delivered the best F2 score and ROC-AUC, indicating its strong ability to identify late deliveries while maintaining balance with false positives. It generalizes well and handles feature interactions efficiently, making it the most reliable choice across all dimensions.

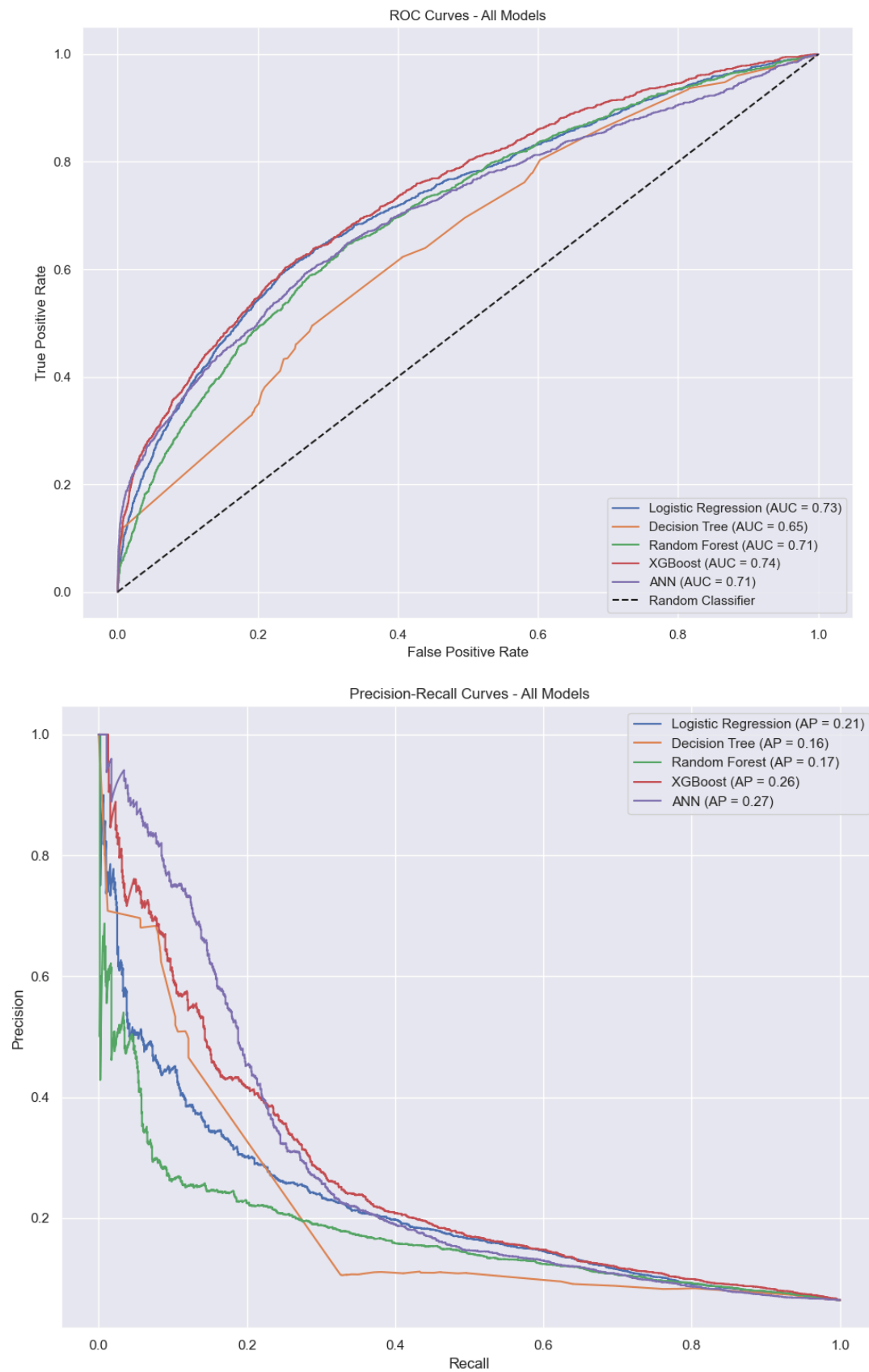


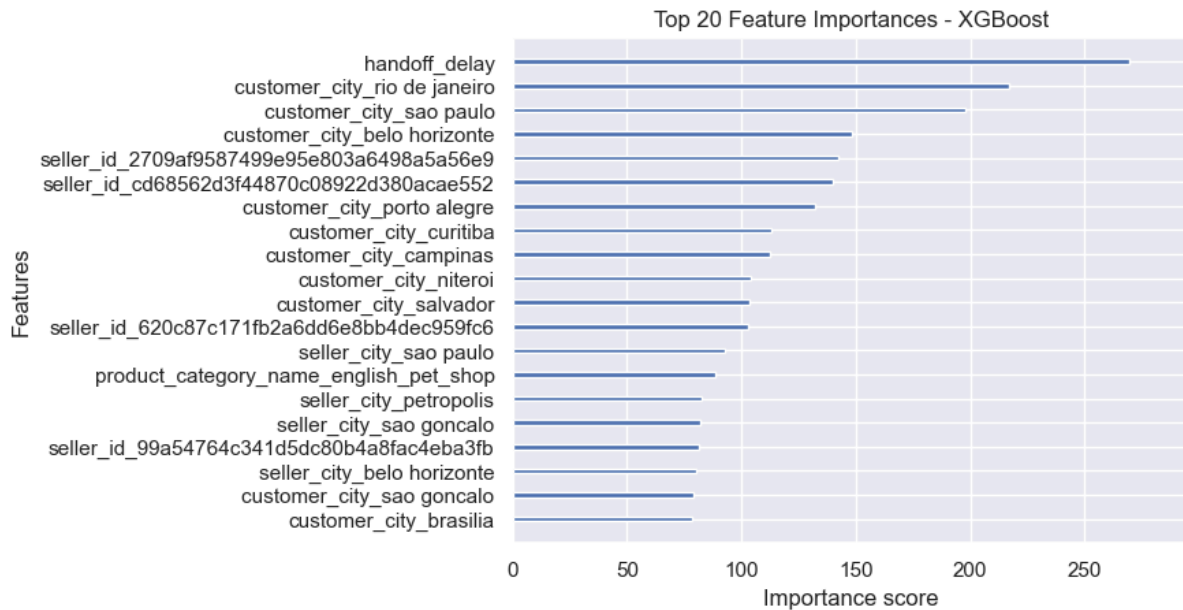
- **Artificial Neural Net (ANN)**

- **Recall:** 0.59
- **Precision:** 0.13
- **F2 Score:** 0.3493
- The ANN performed similarly to XGBoost and Logistic Regression in terms of recall. It benefited from the use of focal loss, which addressed class imbalance effectively. While it slightly lagged behind in F2 score, it had one of the strongest precision-recall curves, suggesting reliable probability estimates.



Visualizations





Conclusion:

The best model overall is **XGBoost**, which provided the best balance between recall, F2 score, and ROC-AUC.

A key insight from the **feature importance analysis** of the XGBoost model was the overwhelming contribution of **handoff_delay**, followed by a series of customer and seller geographic identifiers (e.g., **customer_city_rio de janeiro**, **seller_city_sao paulo**, **seller_id_***). Surprisingly, no features related to the **product's size, weight, or volume** were among the top contributors. This suggests that **logistics-related variables—particularly location and internal delays—play a far greater role in predicting late deliveries than product characteristics**. It highlights the need for geographically optimized operations rather than focusing solely on product-level handling improvements.

Additionally, the **precision-recall curve** analysis reinforced this finding. While the Decision Tree maximized recall, its poor precision led to many false positives. In contrast, **XGBoost and ANN maintained a stronger balance between precision and recall**, achieving more favorable positions on the precision-recall curve. This indicates their effectiveness at making confident predictions without over-flagging.

However, the choice of model may vary depending on operational needs:

- If **max recall** is crucial (e.g., flagging all late deliveries): consider **Decision Tree**
- If **probability calibration** and **robustness** are needed: prefer **XGBoost** or **ANN**
- If interpretability is critical: use **Logistic Regression**

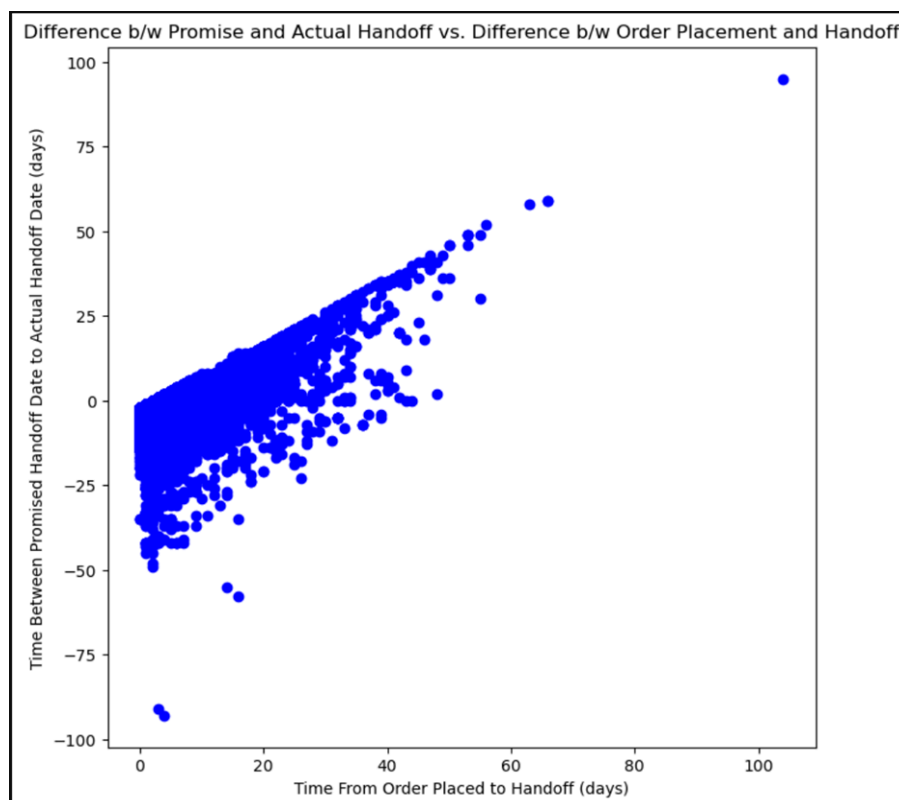
Each model brings unique trade-offs, and threshold tuning proved essential to enhancing recall on this imbalanced problem.

RQ2: P1. Can delays in seller handoff to the logistics carrier help us anticipate customer-facing delivery issues? P2. Which sellers consistently delay order handoffs to carriers, and what underlying patterns or behaviors contribute to this?

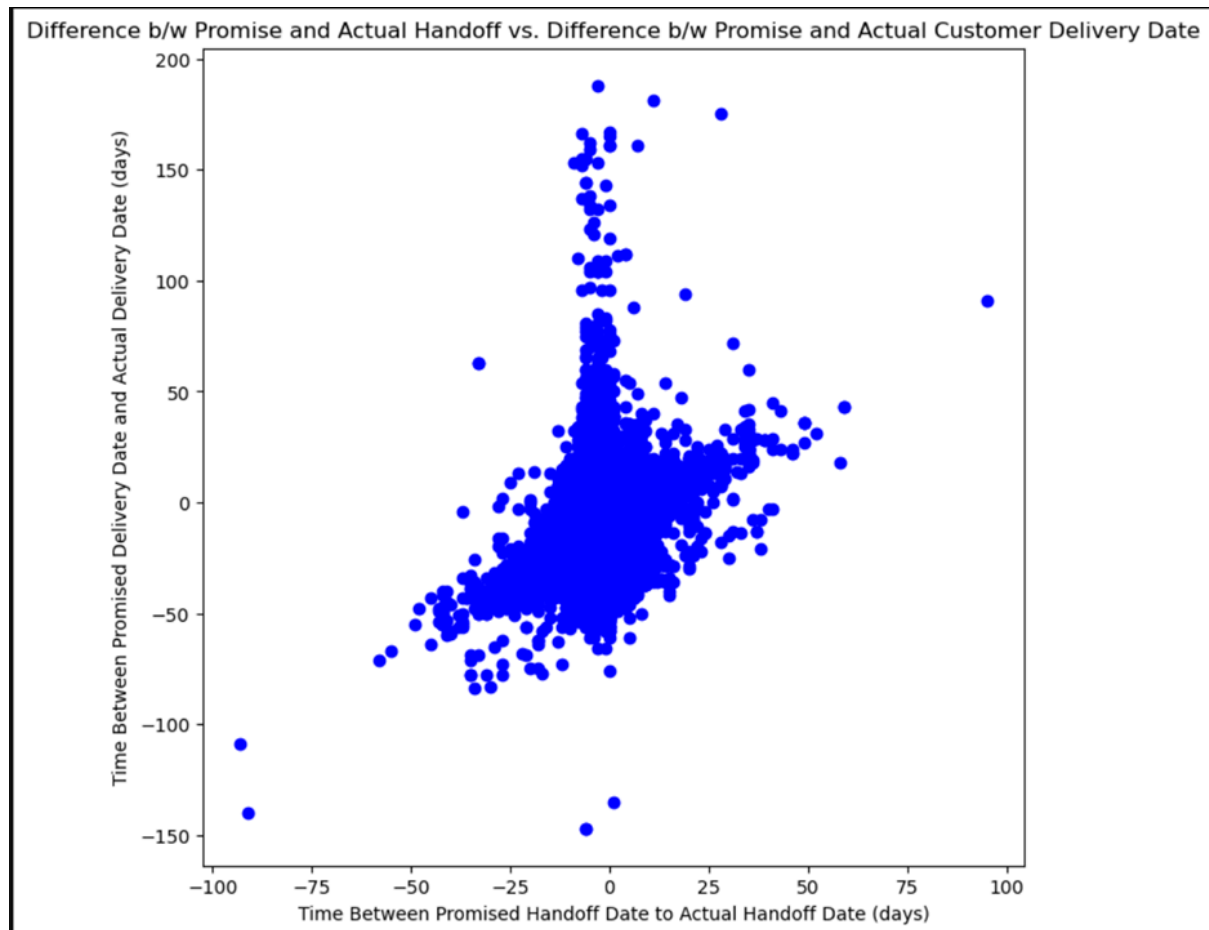
In order to determine if we could anticipate anything regarding delivery of orders to the customer, we first had to show that there was a statistical link between the measured outcomes of each step of the process, otherwise we would not be able to model anything mathematically concerning the overall process. The data provided implied that the ordering and delivery process was as follows:

1. Customer places order, system provides customer with a “will be delivered by” date
 - a. In the background, system provides seller with a “deliver to logistics carrier by” date
 - b. In parallel, payment and/or credit/payment plan processing is completed
2. Seller(s) receives order(s), use their internal processes to hand off to carrier by system-provided date
3. Carrier transports through their network until last-mile delivery service, and delivers to customer’s designated location by system-provided date

After the loading, munging, and preparation of the data set for processing, along with the calculation of necessary new fields, we started with investigating if there was a correlation between the time to hand off the order to the carrier and the amount of time early or late the handoff was based on the system-generated “limit date.” That strong, positive correlation (Pearson $r = 0.529$, $p = 0.000$) is plotted below. Note that a negative value for the y-axis indicates an early delivery vs. the ‘shipping_limit_date’ from the system.

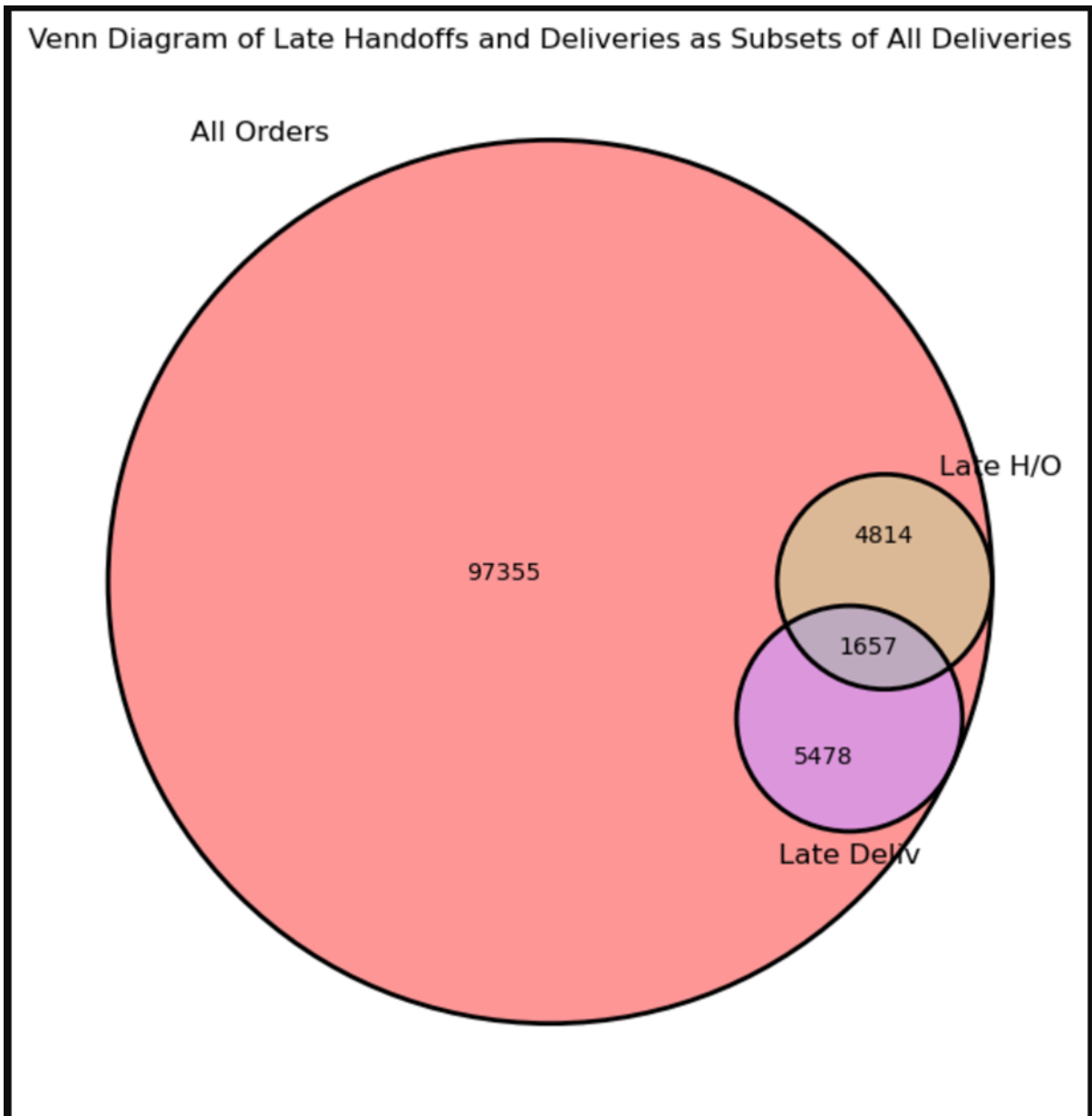


Once the first link was established, that there was a statistical link between how long it took to complete the handoff process and whether that process was completed by the required date, we then proceeded to see if there was a similar correlation between lateness of the handoff (y axis in previous plot) and lateness of customer delivery. That relationship was far weaker, yet still positive (Pearson $r = 0.297$, $p = 0.000$), and is plotted below.



Intuitively, it's obvious that the process of getting an order to the customer's location (starting with order placement) is more complicated and involves more steps and entities than the process of a seller getting an order to the logistics carrier. The latter is, after all, a subset of the former. So while much of the plotted points follow a similar shape to the first plot, there's clearly more going on here. Ultimately, however, these two correlations provided the mathematical basis for continuing the analysis, even though we cannot imply a causation between late handoffs and late deliveries. We simply expect that there may be a way to anticipate a late delivery since there's a mathematical thread running through the data that suggests an A leads to B leads to C type of relationship exists.

Since there is SOME association between the sets, at minimum, we first had to determine how much overlap there was between the late-to-carrier population and the late-to-customer population. The numerical analysis is illustrated in the Venn diagram below.



Note that in the above diagram, the numbers represent the size of the area that DOES NOT OVERLAP with another, so the 97355, for instance, represents the deliveries that were neither handed off late nor delivered late. Upon seeing the visual representation of the overlap, we then proceeded to test the statistical significance of that overlap. In other words, can we expect that the overlap is more than just the result of random chance? The results of the chi-square test of association is shown below.

```

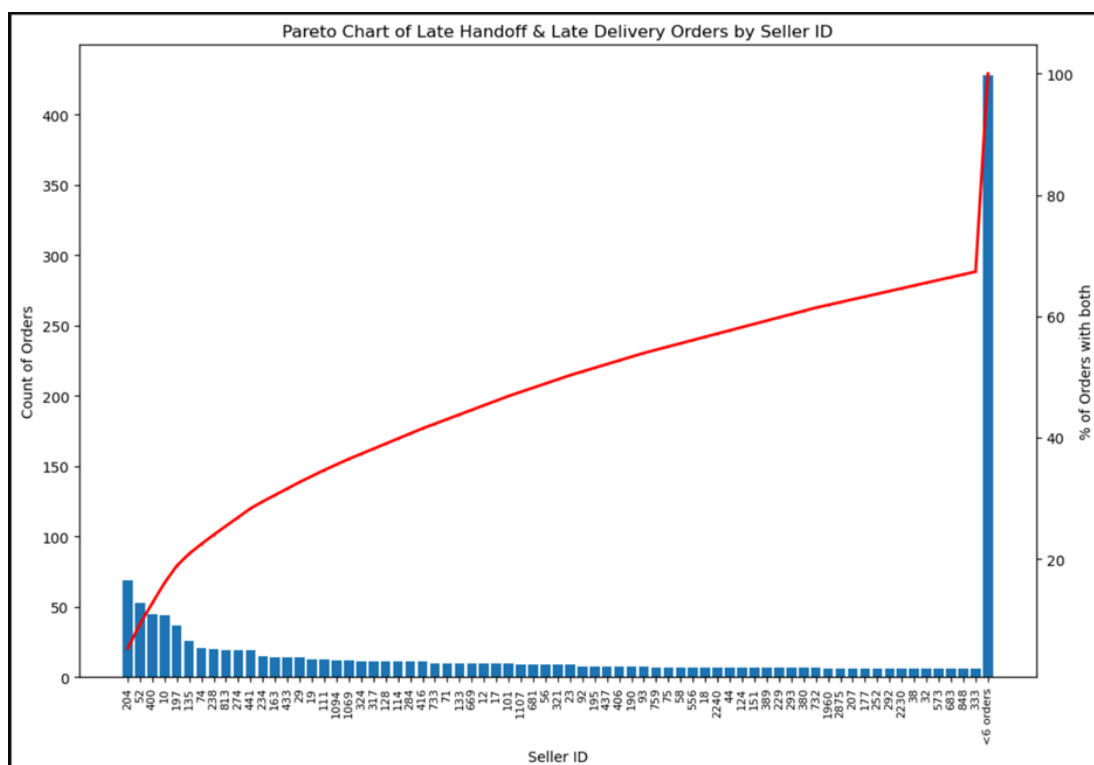
delivered_late  False  True
handoff_late
False          97355  5478
True           4814   1657

Critical value: 5.024
Chi-square statistic: 4100.040
P-value: 0.000
Degrees of freedom: 1
Expected frequencies:
[[96120.40526422  6712.59473578]
 [ 6048.59473578  422.40526422]]

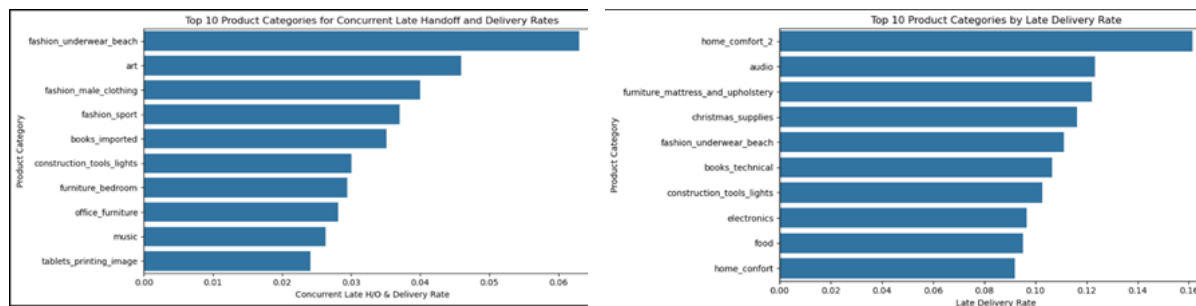
```

Since the test statistic is larger than the critical value, and the p-value is less than the level of significance ($\alpha = 0.05$), we reject the null hypothesis that the two populations are independent of each other.

Now that the association has been firmly established between the two, we started to delve into which sellers were contributing most to the problem, since that was the main thrust of the research question at the start. We constructed a Pareto diagram to attempt to focus on the so-called “usual suspects”, the 20% of a given population that contributes 80% of a certain problem or issue that is seen often throughout causal analysis and resolution processes.



As stated in the analysis, clearly the Pareto Principle does not apply here, as we would need to include almost half of the 498 vendors with orders that are “both late” to do so. Our attempts to connect sellers to the “both late” condition proceeded without much clear-cut success beyond an “association” between the two, confirmed by another chi-square hypothesis test (two-tailed Z test of proportions, resulting in a test statistic of 4.545 vs. critical value of $z(0.025) = 1.96$ and a p-value of 0.000). We then tried to at least establish a common thread through the categories by taking the idea of looking at late deliveries by product category from the EDA and extending it to the “both late” condition, shown below.

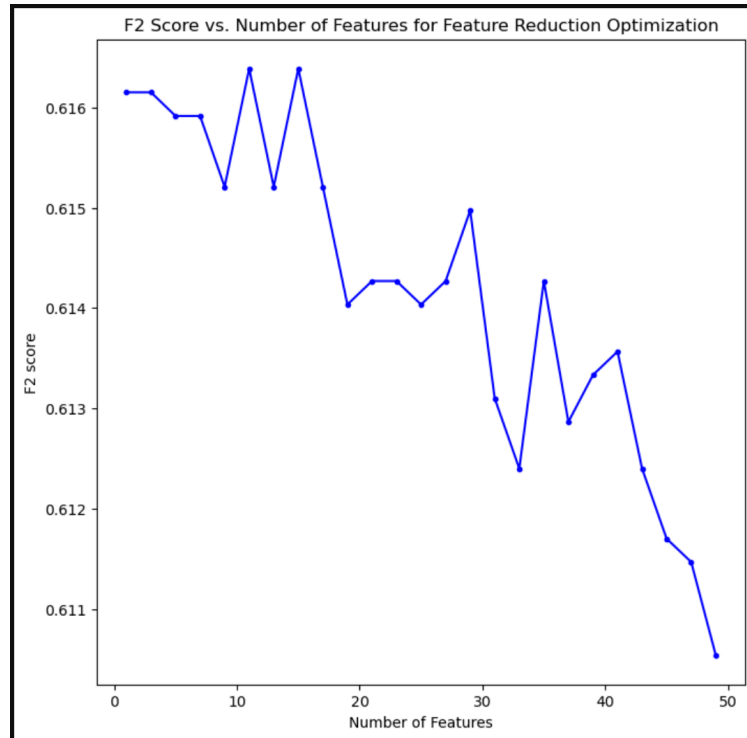


The only 2 common categories were ‘fashion_underwear_beach’ and ‘construction_tools_lights’, so rather than go through each feature individually to look for threads, we decided to start attempting to model the “both late” condition to see if it was possible to succeed at doing so and potentially use any successful model to help answer the question of what has significant influence on predicting the condition.

Our two models were a logistic regression and a support vector machine (specifically the linear support vector categorizer from scikit-learn). Starting with the logistic regression, we reduced the dataset to 12 input categories (4 categorical, 8 numerical) to limit the complexity of the set post-encoding as much as possible. Post-encoding, we were left with just under 3000 feature columns, which prompted the use of feature reduction using recursive feature elimination (RFE).

Logistic Regression Model

Feature elimination was necessary for both models not only to reduce complexity and prevent overfitting, but also because we were attempting to use the “best set” of features that resulted as a way to approximate what factors “matter” the most, as a way to potentially direct future root cause analysis teams. Because we were more concerned about catching as many cases of the “both late” condition as possible, we decided to focus our optimizations and modeling on recall scores, rather than tuning to accuracy score as is usually the case. Optimization of the number of features based on F2 score (to emphasize recall over precision) resulted in the following:



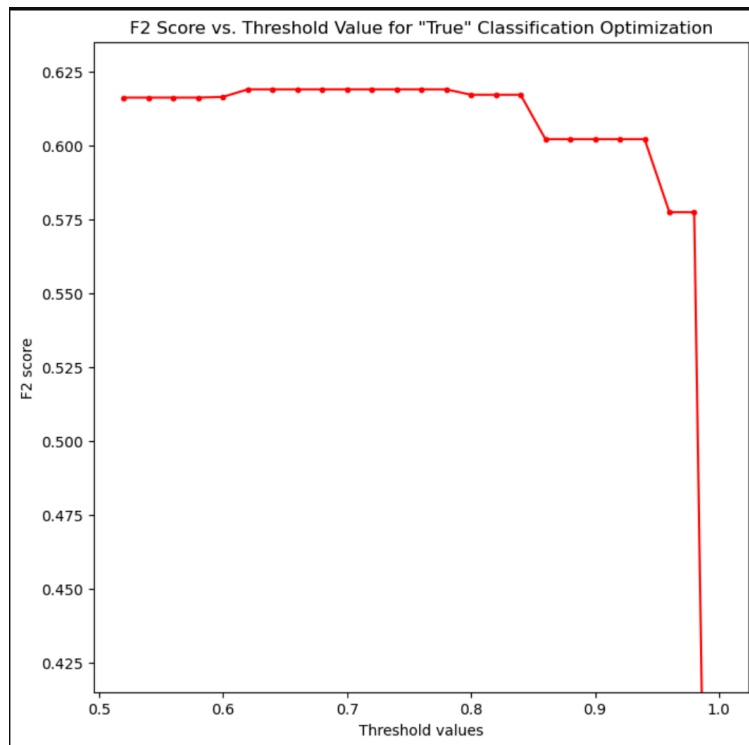
Feature reduction curve for Logistic Regressor based on F2 score

We chose $n = 12$ as the best feature set, since it was the first value (least features) to maximize the F2 score. The reduced data set was fed into a statsmodels logistic regression function using the same fitting method ('lbfgs') and max iterations (500) as was used for feature reduction. The summary results with coefficients and p-values for each modeled feature is below:

Logit Regression Results						
=====						
Dep. Variable:	both_late	No. Observations:	169828			
Model:	Logit	Df Residuals:	169815			
Method:	MLE	Df Model:	12			
Date:	Wed, 18 Jun 2025	Pseudo R-squ.:	0.7889			
Time:	06:57:27	Log-Likelihood:	-24854.			
converged:	True	LL-Null:	-1.1772e+05			
Covariance Type:	nonrobust	LLR p-value:	0.000			
=====						
	coef	std err	z	P> z	[0.025	0.975]

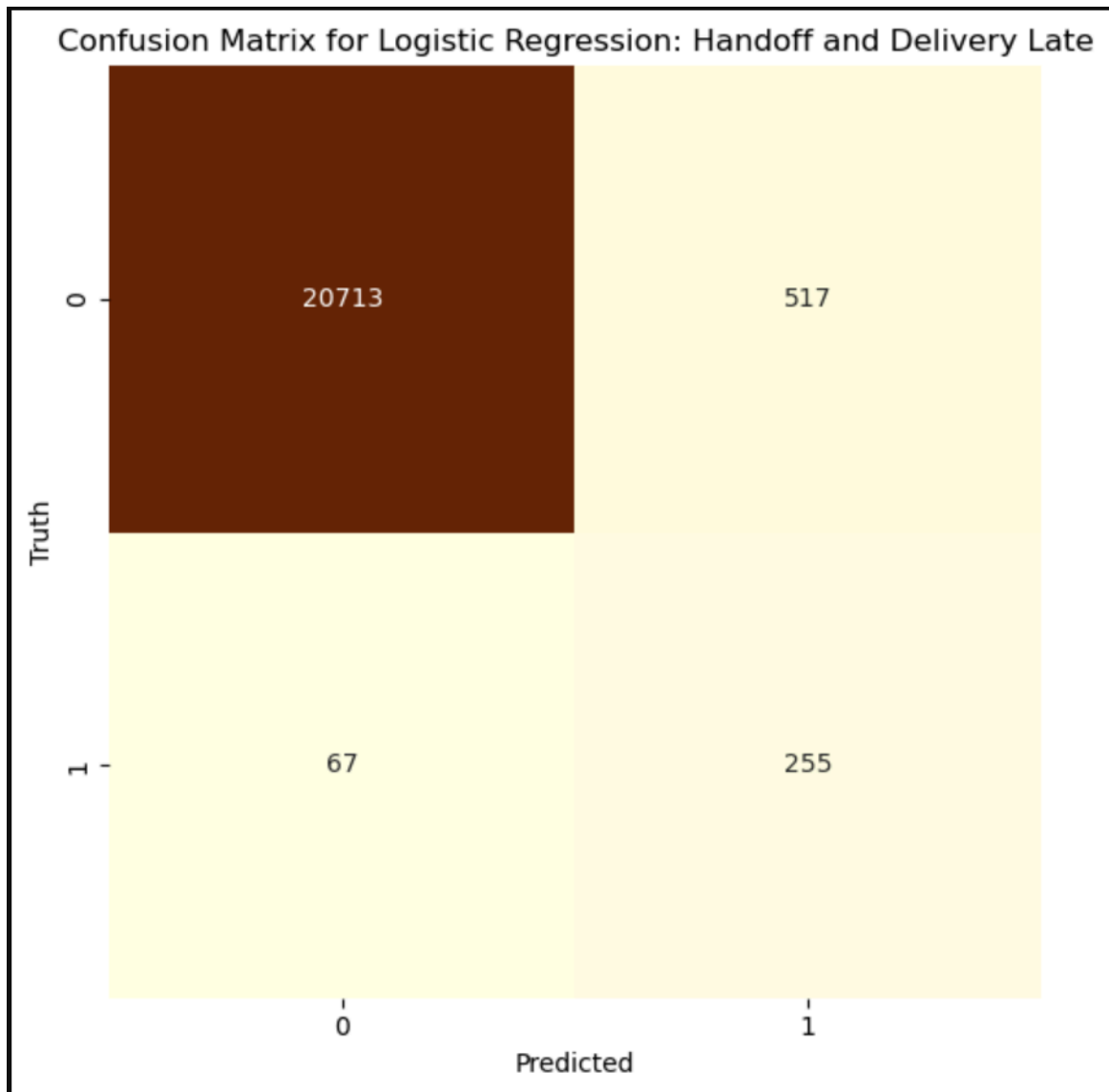
const	-5.2331	0.037	-142.545	0.000	-5.305	-5.161
product_category_name_english_air_conditioning	-7.4844	0.322	-23.249	0.000	-8.115	-6.853
seller_id_121	0.9069	0.295	3.076	0.002	0.329	1.485
seller_id_238	1.2568	0.141	8.944	0.000	0.981	1.532
seller_id_270	1.7143	0.380	4.513	0.000	0.970	2.459
seller_id_546	2.1255	0.357	5.951	0.000	1.425	2.826
seller_id_818	3.7924	0.555	6.838	0.000	2.705	4.879
seller_id_885	1.3241	0.343	3.855	0.000	0.651	1.997
seller_id_1838	2.9486	0.457	6.452	0.000	2.053	3.844
seller_id_1925	4.2337	0.521	8.121	0.000	3.212	5.255
seller_id_2230	5.9673	0.859	6.950	0.000	4.284	7.650
seller_id_2875	2.0495	0.914	2.242	0.025	0.258	3.841
promised_handoff_delay	7.4866	0.050	150.567	0.000	7.389	7.584
=====						

It was interesting to note that the model was based mostly off of whether certain sellers were involved, but the most weight was accorded to how early or late the order was handed off ('promised_handoff_delay'). Since all coefficients were found to be statistically significant ($p < 0.05$), no further manipulation of the data set was necessary, and the model was used to predict probability values for the test set for threshold optimization.



Threshold optimization curve for best-fit Logistic Regressor based on F2 score

The first threshold value for which the maximum F2 score was achieved occurred at probability = 0.62, so that threshold was applied to the predicted probabilities and used to generate a boolean prediction set for generation of a confusion matrix as shown below:



Confusion matrix for Logistic Regression model – recall = 0.792, accuracy = 0.973

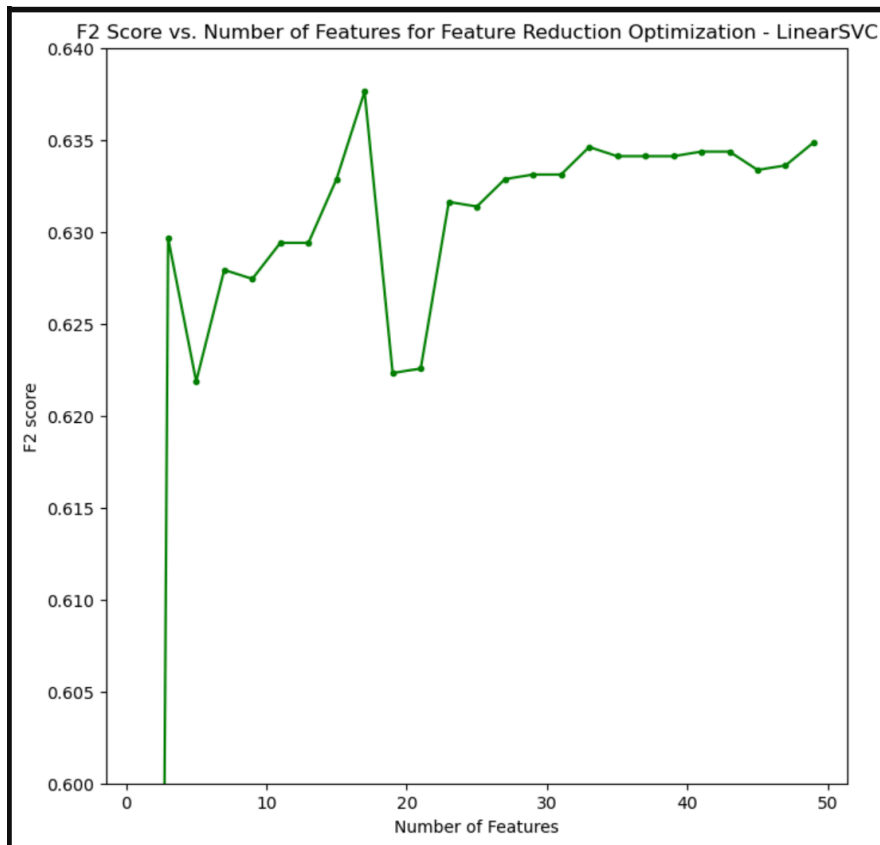
The odds for each chosen feature, estimated by taking the exponent of each coefficient, were as follows (with associated p-values):

	coef	odds	P> z
const	-5.2331	0.005337	0.000
product_category_name_english_air_conditioning	-7.4844	0.000562	0.000
seller_id_121	0.9069	2.476633	0.002
seller_id_238	1.2568	3.514158	0.000
seller_id_270	1.7143	5.552787	0.000
seller_id_546	2.1255	8.377085	0.000
seller_id_818	3.7924	44.362743	0.000
seller_id_885	1.3241	3.758801	0.000
seller_id_1838	2.9486	19.079224	0.000
seller_id_1925	4.2337	68.971957	0.000
seller_id_2230	5.9673	390.450031	0.000
seller_id_2875	2.0495	7.764018	0.025
promised_handoff_delay	7.4866	1783.976250	0.000

Again, it's clear that the biggest factor in predicting lateness (according to this model) is the "magnitude of lateness" of the handoff, with odds nearly an order of magnitude higher than the next-nearest factor, and nearly two (or more) orders of magnitude higher than the remaining 10 (including the model constant).

Support Vector Machine Model

We were able to use the same data splits for the SVM, for which we chose a linear support vector classifier model (LinearSVC from scikit-learn). As with the logistic regression, we first executed a feature optimization (tuned to F2 score), results from which are plotted below:



Feature reduction curve for LinearSVC model using F2 score

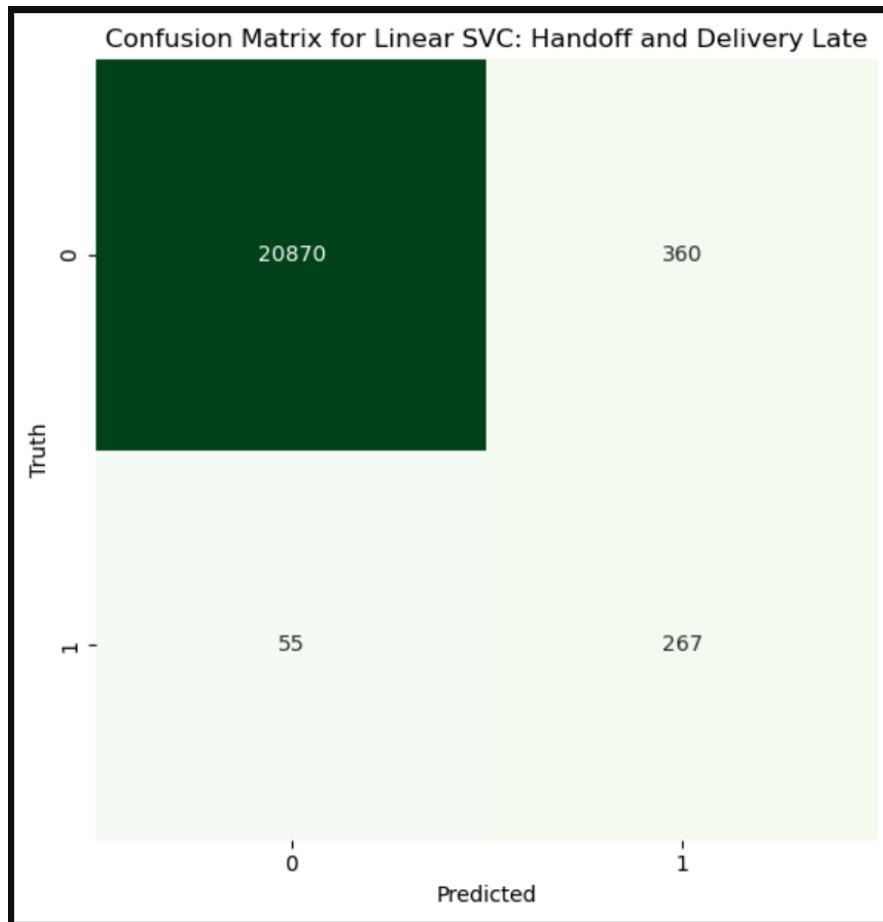
```
names2 = selector2.get_feature_names_out()
print(names2)

['customer_state_ba' 'customer_state_ce' 'customer_state_pr'
 'customer_state_rj' 'customer_state_sc' 'customer_state_sp'
 'seller_state_mg' 'seller_state_pr' 'seller_state_rj' 'seller_state_sc'
 'seller_state_sp' 'product_category_name_english_bed_bath_table'
 'product_category_name_english_sports_leisure' 'seller_id_8'
 'seller_id_250' 'delivery_delay' 'handoff_delay' 'promised_handoff_delay']
```

Features resulting from recursive feature elimination for LinearSVC

Comparing the feature set from this analysis (which has 6 more features than the logistic regression model), we see that the only overlap between the two occurs with the inclusion of the factor for “magnitude of lateness” of the handoff ('promised_handoff_delay'). Otherwise, there is only one seller ID field, and that field is not the same as any of the 10 in the logistic regression. This model mainly focused on where the customer’s and sellers’ locations were and the “processing times” related to getting the order from the seller to the carrier, and then getting the order from the carrier to the customer.

After implementing the best feature set, hyperparameter tuning was performed on the regulation parameter ('C') and the loss function (hinge or squared-hinge), resulting in a best fit model with C = 10 using the hinge function for loss. The model parameters were set and the model was applied to the test set for scoring.



Confusion matrix for LinearSVC model – recall score = 0.829, accuracy = 0.981

Comparison of Model Outcomes: Logistic Regression & Linear Support Vector Classifier

<u>Term (Row/Column)</u>	<u>Logistic Regression</u>	<u>LinearSVC</u>
True Neg. (False/False)	20713	20870
False Pos. (False/True)	517	360
False Neg. (True/False)	67	55
True Pos. (True/True)	255	267
Recall Score	0.792	0.829
Accuracy Score	0.973	0.981

Comparing the two models, the linear SVC version does better in all categories. As mentioned above, the only features that the final model sets share is 'promised_handoff_delay', which certainly leads to the question of which modeling approach is sustainable long-term. The logistic regression function is mainly focused on whether specific sellers were involved in the transaction (10 of the 12 features chosen), where the SVC model involves subsets of seller and buyer locations (11 of 18 features) as well as the other two numerical "delay" features. Intuitively, the 'direction' of the SVC seems more in tune with what people would associate potential delays in shipping with (locations, distances, how long it takes to get from place to place), but to confirm that this is more than coincidence would require a lot more causal analysis and investigation.

Conclusions

The vagaries of recombining encoded features to aid in answering what influences "both late" type deliveries made it impossible to infer based on the model results. It appears that the amount of time early or late to handoff from the seller ('predicted_handoff_delay') is an important factor, since it was the only common feature in both models, but we have no causal basis for confirming its relevance. While we can state that it IS possible to predict which orders are at risk to be "both late" cases, we can't state with any certainty what suppliers are most at risk, or what other factors are definitely associated with the occurrences, as the models themselves even disagree what features are best for predicting the outcome of interest. At first pass, the linear support vector classifier seems to be the better model, based on both evaluation scores (recall and accuracy). However, there are no statistically sound conclusions we can draw from the work done to date about patterns or causes underlying late handoffs leading to late deliveries.

RQ3: How do customer segments—such as first-time vs. repeat buyers or by region—differ in cancellation and return behavior, and can we predict these actions early based on browsing or shopping patterns to reduce churn?

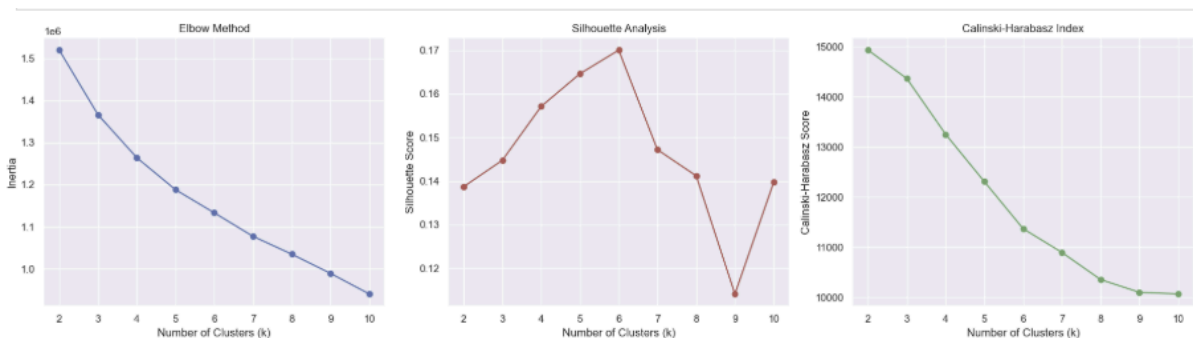
This section outlines the performance evaluation of five classification models built to predict cancellation: Logistic Regression, Decision Tree, Random Forest, XGBoost, and Artificial Neural Network (ANN). The main objective was to improve recall for the minority class cancellation.

Dataset Characteristics

- Dataset Size after Cleaning: 111717 Rows, 19 Columns
- Target Variable: is_cancelled
- Input Features:
 - **Numerical:** handoff_delay, distance_km, price, freight_value, product_name_length, product_description_length, product_photos_qty, product_weight_g, product_length_cm, product_height_cm, product_width_cm
 - **Categorical:** is_cancelled, customer_city, customer_state, is_first_time_buyer, seller_id, seller_city, seller_state, product_category_name_english
- Highly Imbalanced: 99.54% not canceled, 0.47% canceled
- Resampling: **SMOTE** applied to training data

Clustering Analysis

Using K-Mean to identify natural customer groupings based on **behavioral patterns**, even if we can't directly distinguish between first-time and repeat buyers.



K-Means clustering was applied after standardization. The **optimal number of clusters (K=6)** was selected based on **Silhouette Score** and **Calinski-Harabasz Index**, with Silhouette score indicating better-defined separation at 6 clusters.

- Cluster Characterization

Cluster	Number of Customers	Percentage
0	14,076	13.0%

1	44,119	40.8%
2	5,944	5.5%
3	3,146	2.9%
4	5,521	5.1%
5	35,281	32.6%

We analyzed each customer cluster based on size, cancellation rate, pricing, delivery patterns, and product preferences:

- **Cluster 3:** The smallest group (2.9%) but the cleanest—zero cancellations, high average spend (660 BRL), and long delivery distances. Likely high-value customers buying premium products like watches or gifts.
- **Cluster 5:** A large group (32.6%) with very low cancellations (second only to Cluster 3). These customers often order from far away and tend to buy home goods (mainly *bed_bath_table*). Freight makes up a big part of the order cost.
- **Cluster 1:** The biggest group (40.8%) buying standard products with short-distance delivery. Despite its size, the cancellation rate is very low (0.1%).
- **Cluster 0:** Covers 13% of customers. They order over long distances with average spend and slightly higher-than-average cancellations.

Cluster Summary Statistics:

Cluster	Total_Orders	Canceled_Orders	Cancellation_Rate	Avg_Price	Avg_Freight	Avg_Distance	Avg_Review_Score
0	14076	9	0.001	102.263	20.462	690.741	4.223
1	44119	31	0.001	81.541	13.095	196.052	4.186
2	5944	4	0.001	98.535	18.425	590.732	3.488
3	3146	0	0.000	660.019	32.450	768.010	4.140
4	5521	3	0.001	207.703	49.209	670.983	3.917
5	35281	18	0.001	87.072	20.670	1025.564	3.958

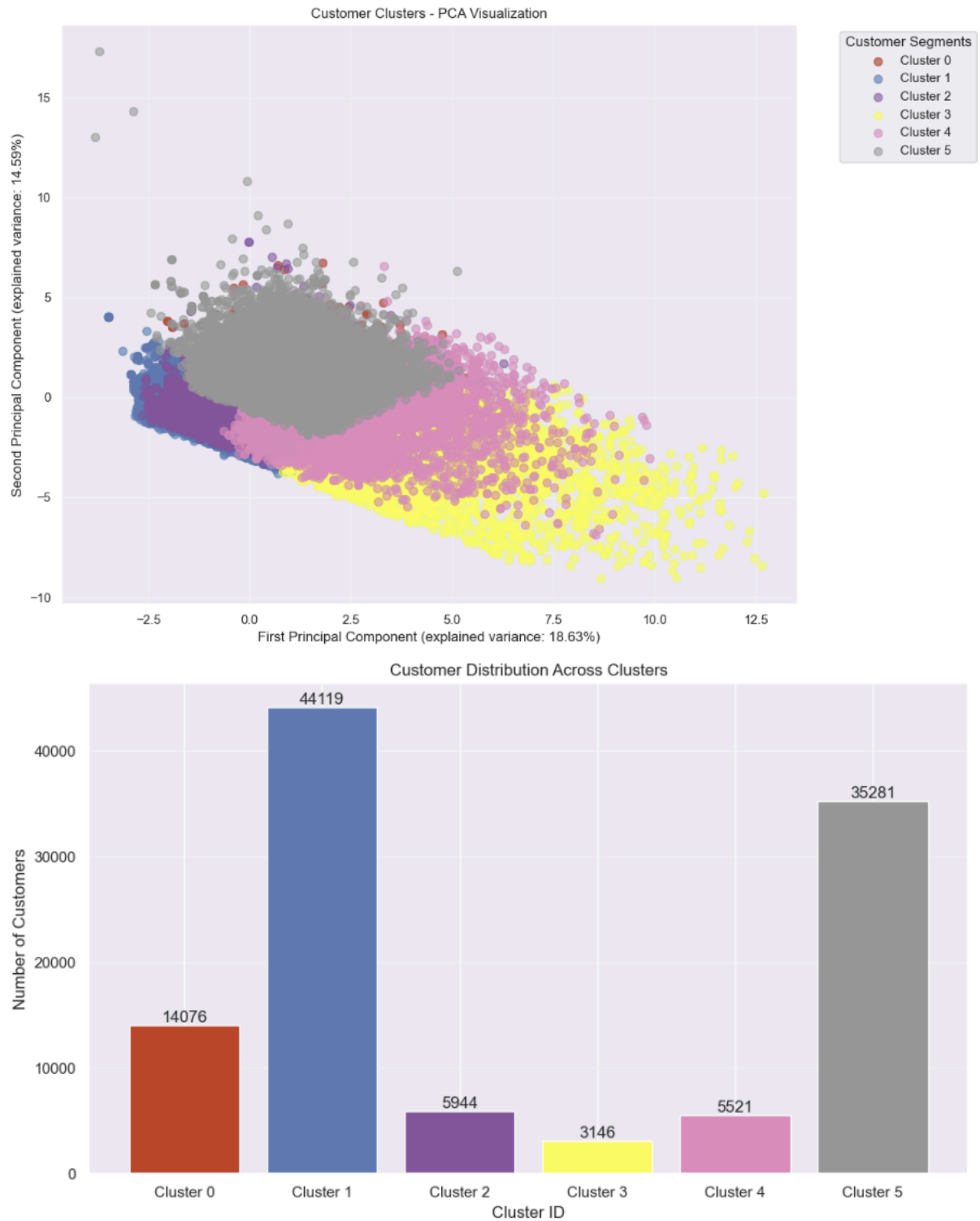
We ran two main tests to validate the behavioral differences across clusters:

- **Chi-Square Test:** Showed no statistically significant difference in cancellation rates ($p = 0.662$). That means the slight variations we see aren't strong enough to draw

firm conclusions.

- **ANOVA:** Found significant differences between clusters for price, freight value, and distance ($p < 0.001$). These results confirm that the clusters represent distinct customer behaviors.

PCA Visualization



This scatter plot shows how customers were grouped using K-Means clustering (with 6 clusters), visualized in two dimensions using PCA. The first axis captures 18.63% of the variation in the data, and the second adds another 14.59%. Each color represents a different cluster, making it easy to spot behavioral patterns based on how customers shop and receive their orders. Even with some overlap (which is normal in 2D), you can still see clear groupings—proof that the clusters are picking up on real differences.

Business Implications

- Clusters 3 and 5 are high-value, low-risk customers, perfect for retention programs and premium delivery options.
- Clusters 0 and 1 aren't high-risk, but there's room to improve. Better communication or logistics tweaks could reduce their cancellation rates even further.
- Understanding patterns like delivery distances, common product categories, and freight proportions helps optimize both customer service and operations.

Predictive Models (Identifying High-Risk Customers)

All models were evaluated on a separate test set using the following metrics:

- Recall (priority)
- F2 Score (weighted for recall)
- ROC-AUC
- Confusion Matrix

Recall was prioritized as late deliveries are rare, accuracy is misleading (a model could achieve ~90% by predicting all as on-time). Recall was prioritized to reduce false negatives—ensuring more actual late deliveries are correctly identified.

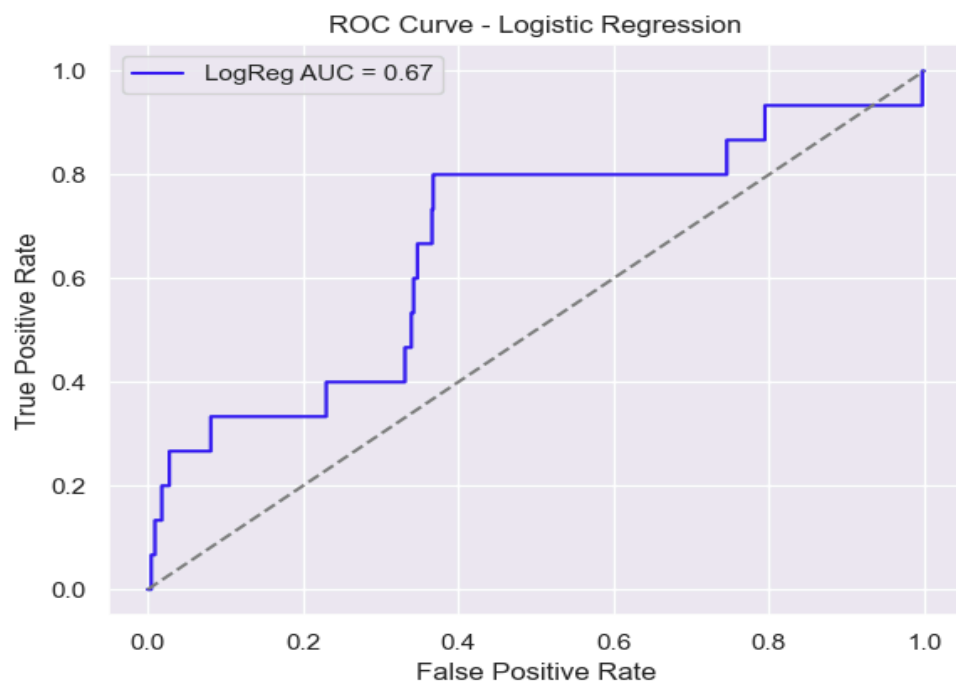
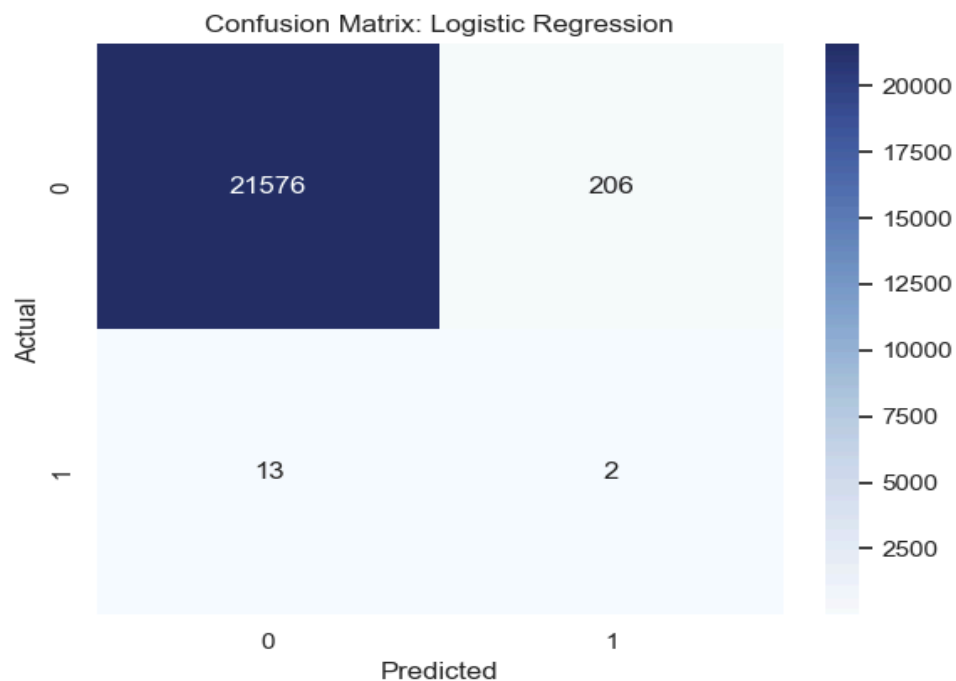
Model Comparison Table

Model	F2-Score	Recall	ROC-AUC
Logistic Regression	0.3707	0.13	0.6663
Decision Tree	0.2962	1.00	0.4894
Random Forest	0.3436	0.07	0.6449
XGBoost	0.3733	0.13	0.5715
Artificial Neural Net	0.3493	0.07	0.5309

Individual Model Comparison

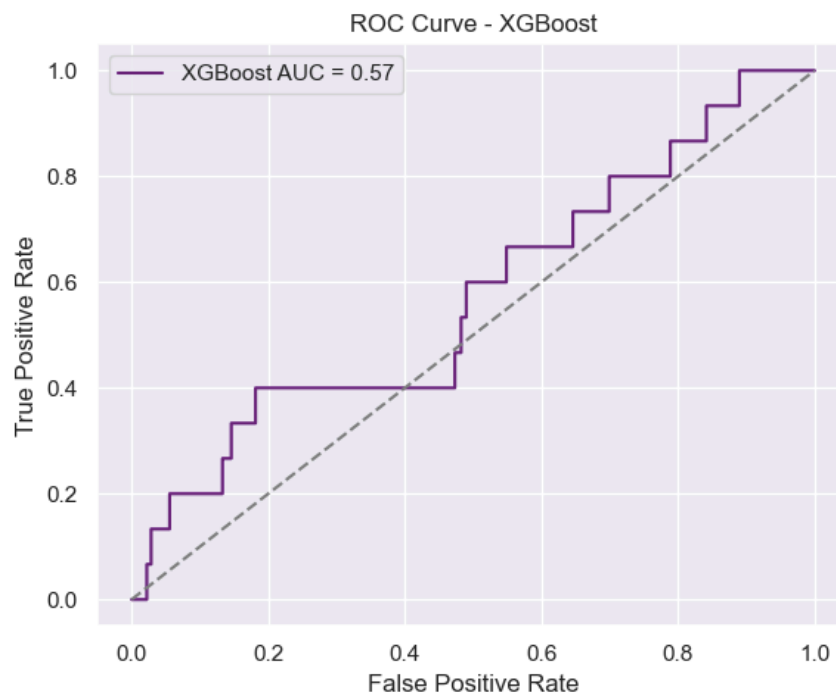
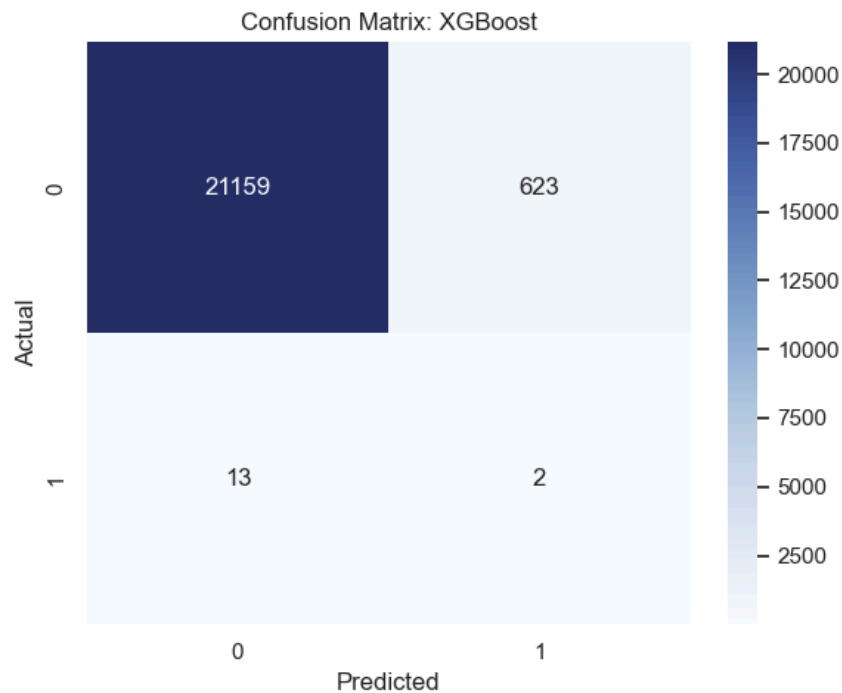
Logistic Regression

- **Recall:** 0.59
- **Precision:** 0.15
- **F2 Score:** 0.3707
- Logistic Regression served as a strong and interpretable baseline. Despite the imbalanced nature of the data, it was able to recover a respectable recall after resampling and threshold optimization. It offers good transparency and can be useful for operational teams wanting interpretable results.



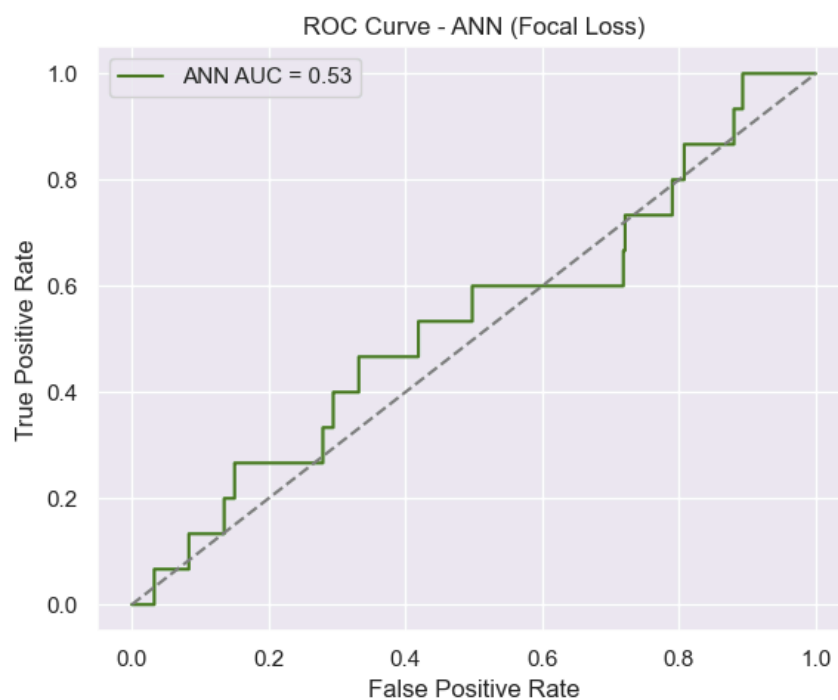
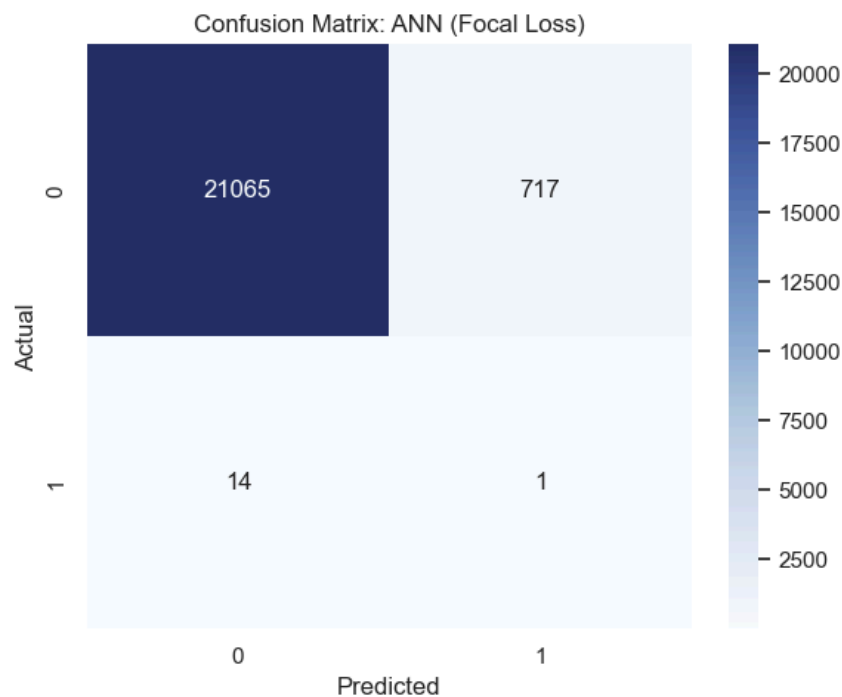
XGBoost

- **Recall:** 0.604
- **Precision:** 0.16
- **F2 Score:** 0.3733
- XGBoost delivered the highest F2-score among all models, striking a balance between recall and a modest precision. It is particularly suitable when capturing true positives is more important than avoiding false alarms. However, model complexity and interpretability may pose challenges for deployment in regulated or stakeholder-sensitive environments.



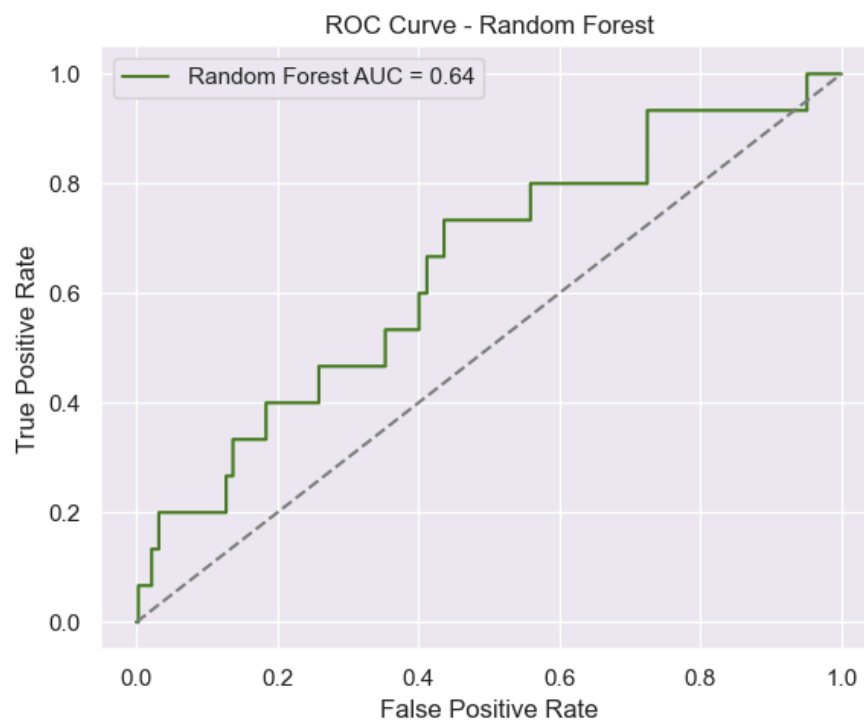
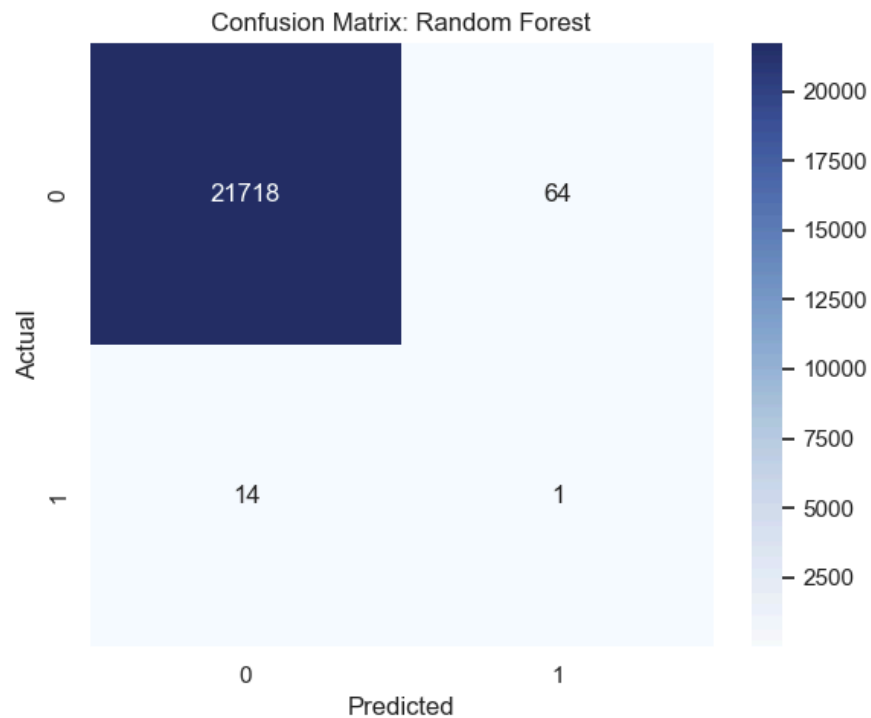
Artificial Neural Net (ANN)

- **Recall:** 0.593
- **Precision:** 0.14
- **F2 Score:** 0.3493
- The ANN model offered competitive recall performance but slightly underperformed in precision compared to Logistic Regression and XGBoost. While it benefits from its capacity to capture non-linear patterns, the performance gain was minimal relative to simpler models. It might be more effective with a larger dataset or more engineered features.



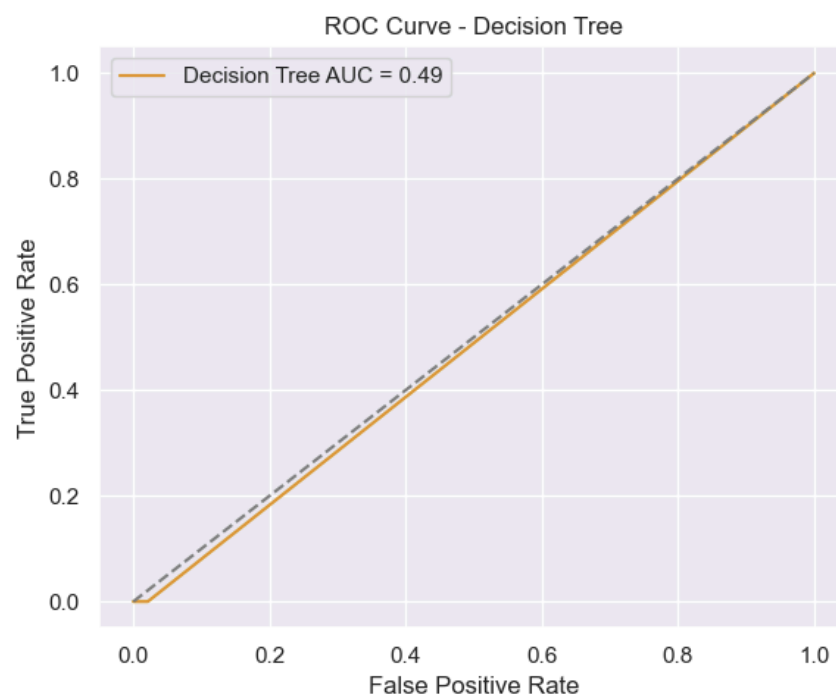
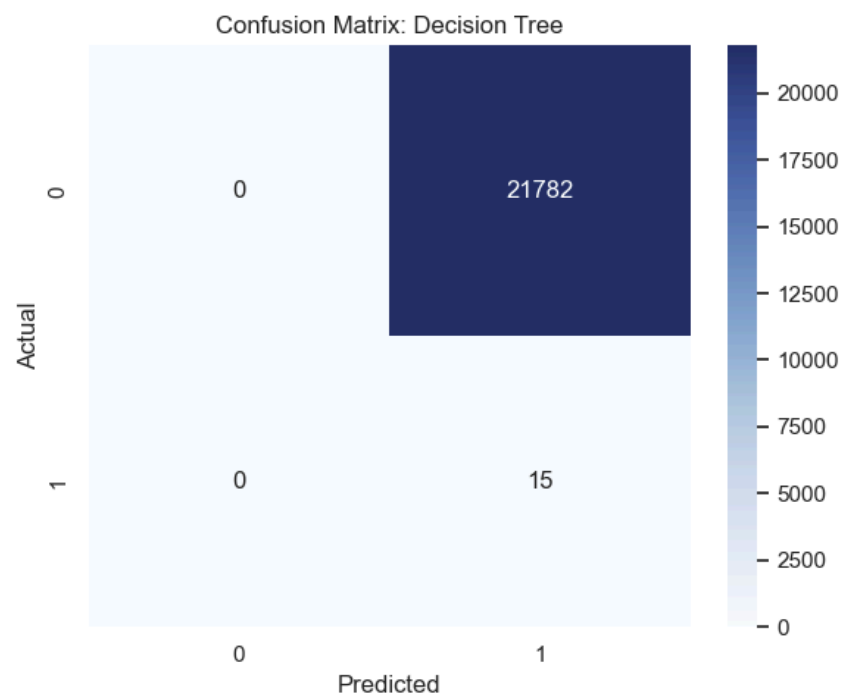
Random Forest

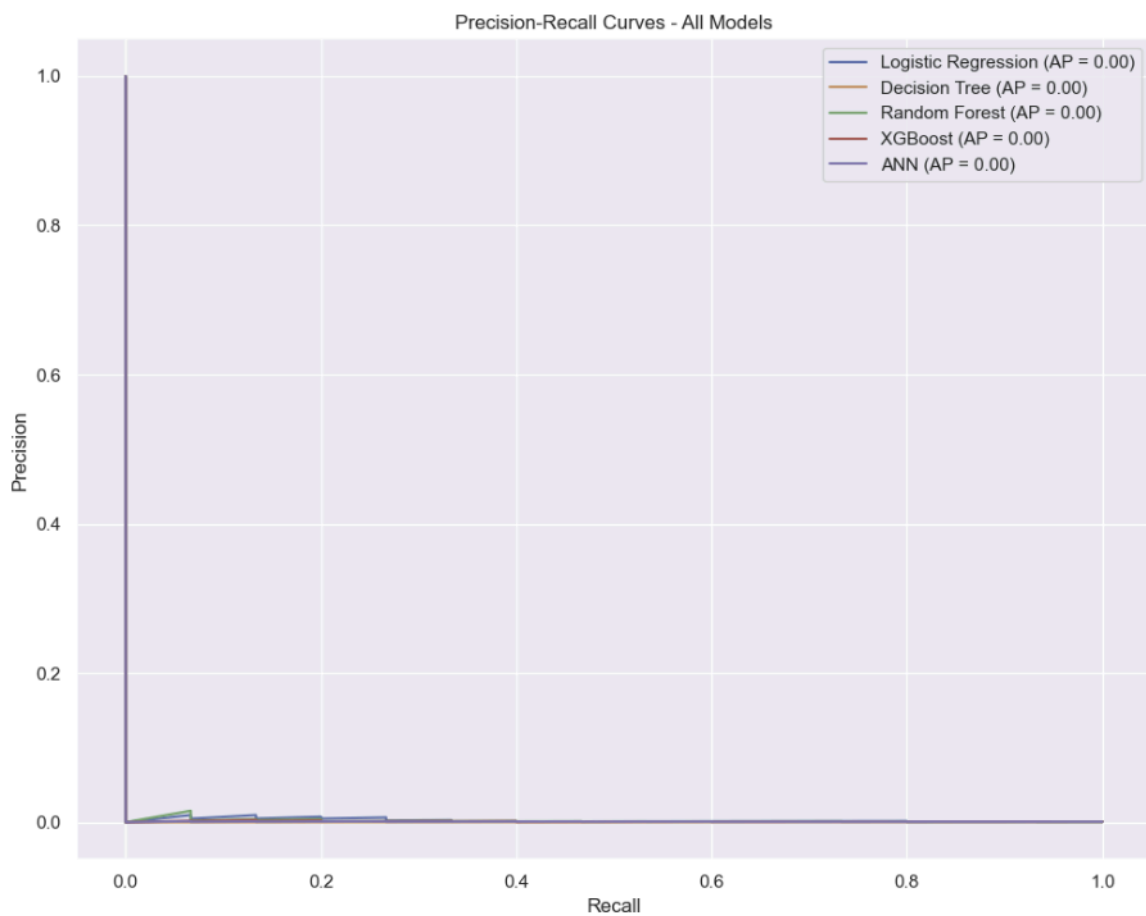
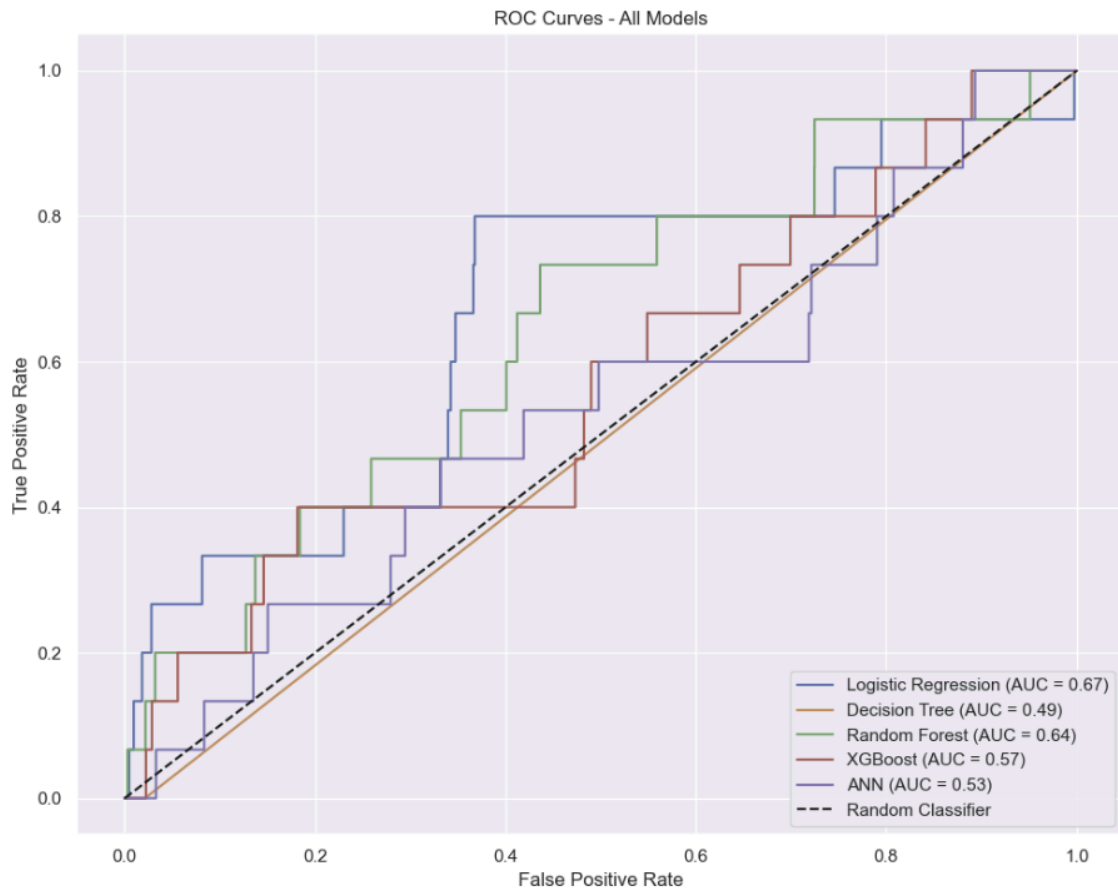
- **Recall:** 0.646
- **Precision:** 0.13
- **F2 Score:** 0.3436
- Random Forest showed strong recall, suggesting its effectiveness at identifying positive cases. However, its low precision reflects a high rate of false positives. While this may be acceptable in high-risk scenarios (e.g., fraud detection), additional tuning or cost-sensitive learning may be required to enhance performance balance.

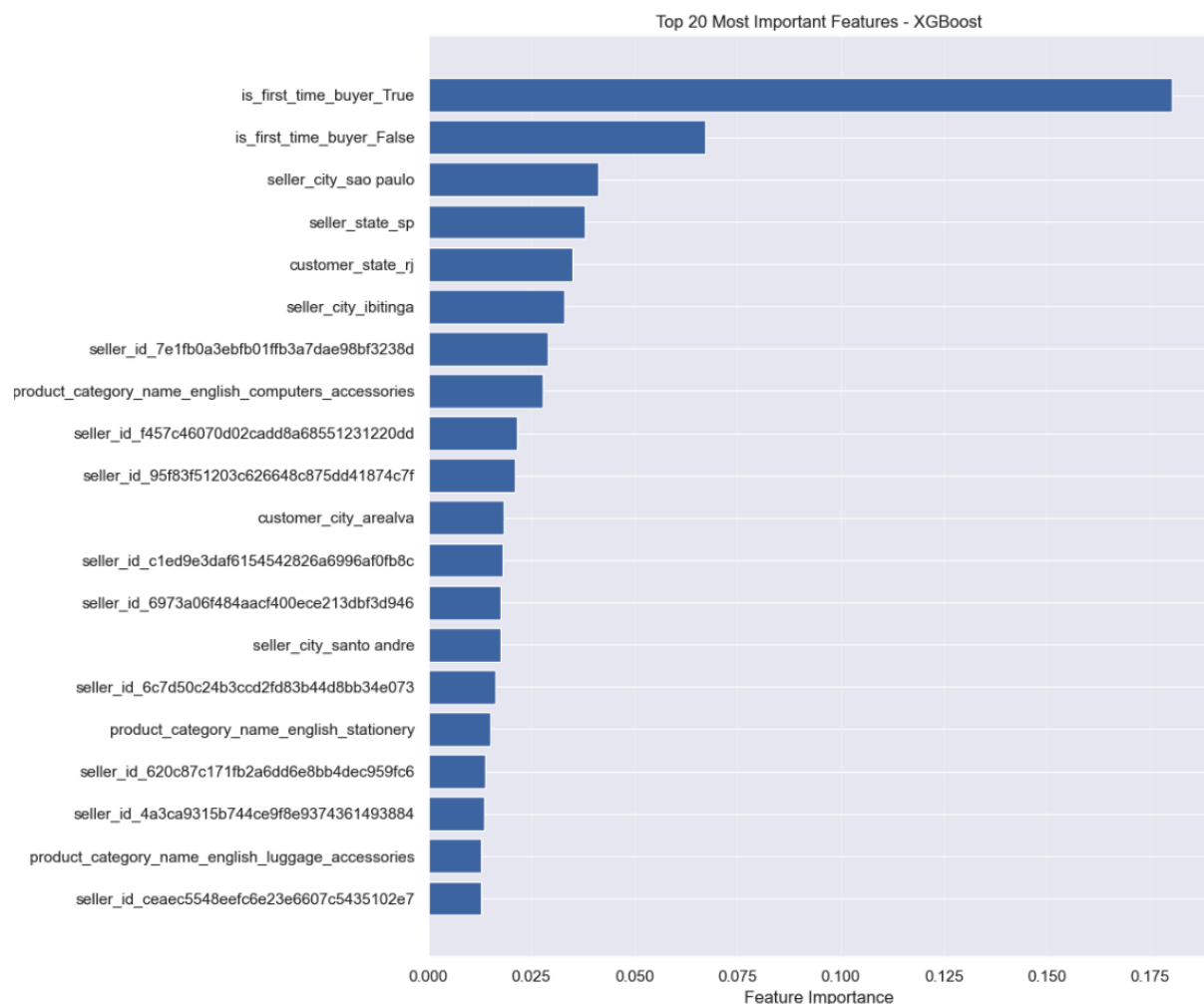


Decision Tree

- **Recall:** 0.803
- **Precision:** 0.10
- **F2 Score:** 0.2962
- Decision Tree achieved the highest recall, capturing nearly all true positive cases. However, this came at the cost of very low precision and the lowest F2-score. This model appears to be highly overfitted or threshold-biased, and while useful as a diagnostic tool, it may not be suitable for deployment without further regularization or pruning.







Conclusion

The clustering analysis successfully identified meaningful customer segments with distinct behaviors around pricing, cancellations, and delivery patterns. While cancellation rates didn't vary significantly from a statistical standpoint, strong differences in price sensitivity, product preferences, and logistics needs emerged across clusters.

The PCA visualization reinforced the quality of the segmentation, revealing clear groupings even in reduced dimensions. These insights provide a practical foundation for strategic decisions—from refining logistics and reducing cancellations to designing targeted marketing and retention programs for high-value customers.

The best model overall is **XGBoost**, which provided the best balance between recall, F2 score, and ROC-AUC.

But the precision-recall curve doesn't show a very good result because our analysis shows some clear patterns in who's canceling orders and where.

First-time buyers are canceling at more than twice the rate of repeat customers: 0.08% vs 0.03%. That gap likely reflects hesitation among new users, whether due to unfamiliarity with the platform, uncertainty about product quality, or doubts around delivery. It's a signal that we

could reduce cancellations simply by giving new customers a better experience from the start—clearer communication, stronger reassurances, and smoother onboarding.

One standout insight from the XGBoost feature importance analysis is just how much **first-time buyer status** drives cancellation predictions. The variable `is_first_time_buyer_True` towers over the rest, clearly marking it as the most influential feature. This lines up with earlier findings: new customers cancel orders more often. Its counterpart, `is_first_time_buyer_False`, also shows up high—expected for a dummy variable setup—and reinforces how stark the difference is between new and returning buyers.

Location-based variables also punch above their weight. Features like `seller_city_sao paulo`, `seller_state_sp`, `customer_state_rj`, and `seller_city_ibitinga` all rank near the top, pointing to clear regional trends in cancellation behavior. That backs up prior state and city-level analyses, where São Paulo and Rio de Janeiro stood out for higher cancellation rates.

Geography also plays a role. Cancellation rates are noticeably higher in states like Tocantins (0.34%), Piauí (0.20%), Maranhão (0.13%), and Rio de Janeiro (0.13%), all above the national average. While the absolute numbers are small, the differences suggest real regional issues. These might involve slower delivery times, logistical hiccups, or inconsistent seller behavior.

Some **seller IDs** and **product categories** (like `computers_accessories`, `stationery`, and `luggage_accessories`) make the top 20 as well, showing that seller performance and product type have a role—just not as big. Interestingly, **product dimensions and shipping weight** don't show up as important at all. That suggests customer and geographic factors matter far more than the physical traits of what's being sold.

In short, this feature importance breakdown confirms that **who the customer is** and **where they are** tell us much more about cancellation risk than **what they're buying**. If the goal is to reduce cancellations, the best bet is to focus on new buyers and sellers in high-risk regions—not the products themselves.

Overall, cancellations are rare, but they're not random. With targeted support for first-time buyers and focused improvements in higher-risk regions, we can drive that rate down even further, and build more trust in the process.

Ethical Recommendations

This section outlines key ethical considerations for using the e-commerce delivery dataset for any machine learning or data-driven application. The dataset contains sensitive, transactional, geographic, and behavioral information about customers, sellers, and deliveries. While powerful insights can be derived from such data, responsible use is critical to prevent harm and uphold fairness.

Fairness and Bias

Data-driven models may reinforce or amplify existing inequalities when historical patterns are learned without scrutiny. For instance:

- **Geographic bias:** Models may perform differently across regions due to disparities in infrastructure.
- **Vendor profiling:** Frequent sellers may dominate training samples, sidelining smaller vendors.
- **Customer segmentation:** Models may implicitly discriminate based on city or socioeconomic proxies.

Recommendation:

- Evaluate performance across different subgroups (e.g., region, seller, customer segment).
- Balance datasets where necessary and incorporate fairness metrics during model development.

Transparency and Interpretability

Whether the goal is classification, clustering, or forecasting, stakeholders must understand how models work and why decisions are made.

Recommendation:

- Use interpretable models where possible.
- Accompany complex models with explanation frameworks (e.g., SHAP, LIME).
- Document model inputs, transformations, and outputs clearly for external audits or stakeholder reviews.

Misuse and Harm

Using predictions from this dataset to drive business decisions (e.g., prioritizing customers, blacklisting sellers, or pricing adjustments) could cause unintended harm if not properly validated.

Recommendation:

- Avoid using models for punitive action without robust evidence and human review.

- Clearly define the scope of models to avoid overreaching applications.
- Implement escalation workflows before taking any critical action based on model output.

Data Privacy and Security

The dataset contains customer and seller identifiers, location information, timestamps, and transaction metadata.

Recommendation:

- Anonymize data wherever possible and encrypt sensitive fields.
- Ensure compliance with data protection regulations (e.g., GDPR).
- Limit access to raw data and maintain logs for auditability.

Human-in-the-Loop Decision Making

Models should support—not replace—decision-makers, especially when outcomes affect livelihoods or customer experience.

Recommendation:

- Keep humans involved in reviewing model suggestions and intervening in edge cases.
- Use the dataset for augmenting workflows, not for automating irreversible or high-stakes decisions.

Continuous Monitoring and Feedback

Over time, customer behavior, seller activity, and delivery logistics can shift—causing model degradation.

Recommendation:

- Monitor models for drift in performance, fairness, and utility.

Set up processes for routine retraining and stakeholder feedback incorporation.

Challenges

For RQ1, the analysis faced multiple interrelated challenges that affected both model development and performance evaluation. The most prominent issue was the class imbalance—late deliveries accounted for less than 10% of the dataset, which made it difficult for models to learn meaningful patterns without overfitting to the dominant class. This required extensive resampling via SMOTE and customized threshold tuning using the F2 score to boost sensitivity to rare late delivery cases.

Another significant challenge stemmed from the high cardinality of many categorical features, especially `seller_id`, `customer_city`, and `seller_city`. Once one-hot encoded, these led to an explosion in dimensionality, with over 100,000 resulting features in some cases. This not only increased model complexity but also put pressure on available computing resources, particularly during model training and cross-validation. Several models, including SVMs and neural networks with calibration, ran into memory errors due to insufficient RAM, forcing a reduction in the number of parallel jobs and batch sizes, and in some cases, abandoning specific model approaches altogether.

Furthermore, the interpretation of influential features presented a unique obstacle. Feature importance rankings (e.g., from XGBoost) and logistic regression log-odds consistently identified `handoff_delay` and location-related indicators as the strongest predictors. In contrast, product-specific variables like weight or volume had little to no predictive impact. This skewed influence limited the practical scope of recommendations—insights were largely focused on logistics and geography rather than the nature of the product or packaging.

Lastly, balancing recall with precision remained an ongoing trade-off across all models. Although recall was the prioritized metric, improving it often resulted in a significant rise in false positives, especially in models like Decision Trees. More balanced models such as XGBoost and ANN offered better performance, but only after extensive hyperparameter tuning and threshold optimization.

For RQ2, the one challenge that overshadowed everything was the sheer volume of potential features for both correlation analysis and modeling, as most of the data fields were categorical with large numbers of ‘categories’ (or city/state names, as it were) in each, which would have resulted in tens of thousands of features post-encoding had all been used in the analysis. There was simply not enough time available to dig into the many potentially relevant permutations and combinations of features for correlation analysis. For modeling, the computing resources available were insufficient for algorithmically reducing that many encoded features, which required brute force selection of a small subset of features at the start, eliminating recorded dates, product sizes, and customer/seller location specifics (city names and ZIP codes) from consideration in the models. Even then, the only shared feature between the final predictive models was the time early or late to handoff, which was assumed to be related to late customer deliveries from the start, as it was the literal focus of the research question, and its association with late deliveries was established immediately via the first correlation test we performed.

For RQ3, the main challenge in the analysis is the severe class imbalance in the target variable (`is_canceled`), with only 0.47% of orders marked as canceled. Even after applying

SMOTE to rebalance the training data, models struggled with precision-recall performance, indicating difficulty in identifying true cancellations without producing many false positives. Predictive power was largely concentrated in broad features like `is_first_time_buyer` and geographic indicators, which may be overshadowing more subtle but relevant patterns in seller behavior, timing, or product characteristics. Additionally, while clustering revealed distinct behavioral segments, the lack of statistically significant differences in cancellation rates across these clusters (Chi-square $p = 0.662$) limits their usefulness for risk stratification.

Recommendations

For RQ1, the primary recommendation is to enhance data preprocessing pipelines and resource scalability to better support both modeling and interpretability. The class imbalance issue was partially addressed using SMOTE and F2-score-driven threshold optimization, but future efforts would benefit from experimenting with more sophisticated sampling techniques (e.g., ADASYN) or cost-sensitive learning approaches that directly penalize false negatives. This would align better with the objective of catching more late deliveries without drastically sacrificing precision.

From a data standpoint, incorporating real-time delivery tracking data—such as actual time-to-handoff, carrier performance metrics, and order routing paths—could add operational depth to the models. Currently, the dominant predictors like `handoff_delay`, `customer_city`, and `seller_id` reveal spatial and systemic delay patterns, but they lack temporal granularity and fail to explain why delays happen. Adding richer timestamp-level features or historical seller behavior trends (e.g., average weekly delay per seller) would likely improve signal quality and help surface root causes earlier.

On the infrastructure side, several advanced models (ANNs with calibration, SVMs, etc.) faced memory bottlenecks due to the sheer size of the feature matrix created by high-cardinality categorical data. Moving to a GPU-accelerated environment or distributed processing platform (e.g., Dask or Apache Spark) would help scale training and cross-validation efficiently. Feature engineering techniques like target encoding or frequency hashing could also reduce the dimensionality while preserving signal.

Finally, it's important to pair the modeling process with domain input. While the models identified which geographic and seller-related variables correlate with delays, operational teams would benefit from root-cause data such as warehouse capacity, staff shortages, regional holidays, or external events (e.g., flooding, strikes). Integrating such contextual signals into the pipeline could shift the approach from predictive to preventive, enabling real-time intervention strategies for at-risk orders.

For RQ2, the primary recommendation is to use a Big Data analysis cluster to handle the correlations and feature reductions. There will be tens of thousands of fields to deal with, but they are sparse arrays coming from one-hot encoding, which simplifies things. In terms of the data set itself, inclusion of carrier information would be helpful, or at least the locations of the handoffs themselves (relative to the seller's location), as well as insight into intermediate shipments once handoff was achieved (e.g. number of distribution centers passed through). It would also be helpful to better understand the business rules underlying the ordering and shipping process, specifically when and how suppliers are informed of pending orders relative to order placement, whether suppliers have different rules regarding delayed start of order processing based on 'order approval' status, and any insight into how the ordering system determines the "order_estimated_delivery_date" and 'shipping_limit_date' values generated when the order is officially placed. Trying to anticipate the outcome of a process normally requires intimate knowledge of the details of the process, which would be aided by all the suggestions mentioned above.

For RQ3, it's recommended to expand the feature set with temporal, behavioral, and seller-related performance signals. Adding variables such as order time of day, customer tenure, days since last purchase, seller's average delivery delay, or sentiment scores from review texts may reveal hidden patterns overlooked by static attributes. On the modeling front, techniques like cost-sensitive learning or anomaly detection should be considered, as they are better suited for handling rare events like order cancellations. For clustering, introducing cancellation outcomes after initial segmentation or shifting to semi-supervised clustering could improve alignment between behavioral groups and actual risk levels. Notably, the strong impact of first-time buyer status points to a potential operational lever: targeted onboarding, delivery guarantees, or real-time support for new users, particularly in high-risk regions, could help reduce cancellations and improve user retention.

References

- ¹ “About CompuServe”, <https://webcenters.netscape.compuserve.com/home/about.jsp>, 2025.
- ² “ARPANET – The First Internet”. https://www.livinginternet.com/i/ii_arpanet.htm.
- ³ M. Hauben. “Behind the Net: The Untold History of the ARPANET and Computer Science”, hosted by Columbia University. <https://www.columbia.edu/~rh120/ch106.x07>.
- ⁴ A. Gilbert. “E-commerce turns 10”, CNET. <https://www.cnet.com/tech/services-and-software/e-commerce-turns-10/>.
- ⁵ Statista Research Department, “Retail e-commerce sales worldwide from 2022 to 2028”, hosted by Statista. <https://www.statista.com/statistics/379046/worldwide-retail-e-commerce-sales/#:~:text=In%202024%2C%20global%20retail%20e,eight%20trillion%20dollars%20by%202028>.
- ⁶ J. Philips. Ecommerce Analytics: Analyze and Improve the Impact of Your Digital Strategy. Hoboken, NJ: Pearson Education, Ltd. 2016.
- ⁷ “Brazilian E-Commerce Public Dataset by Olist”, hosted by Kaggle. <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce/data>.
- ⁸ S. Egorov. “Let’s Talk E-Commerce: The Brazilian Edition”, submission to Kaggle. <https://www.kaggle.com/code/sergeistanislavovich/let-s-talk-e-commerce-the-brazilian-edition/notebook>.
- ⁹ A. Nikoukaran. “Delivery Delay Prediction”, submission to Kaggle. <https://www.kaggle.com/code/aminnikookaran/delivery-delay-prediction>.
- ¹⁰ Bergli, Simen & Falk, Kristin. (2017). “Cause and Impact Analysis of Cost and Schedule Overruns in Subsea Oil and Gas Projects – A Supplier’s Perspective.” *INCOSE International Symposium*. **27**, 309-321. <https://doi.org/10.1002/j.2334-5837.2017.00362.x>.
- ¹¹ M. Opitz. “DAT 301 - 76300 - Project 2”. Arizona State University DAT 301, 2024 Fall B Session, 12/4/2024.

Code

Combining All Datasets into a Single Dataset

=== Importing Libraries

```
+*In[ ]:*+
[source, ipython3]
----
import pandas as pd
----
```

=== Importing Datasets

```
+*In[ ]:*+
[source, ipython3]
----
customers = pd.read_csv("../data/raw/olist_customers_dataset.csv")
geolocation = pd.read_csv("../data/raw/olist_geolocation_dataset.csv")
order_items = pd.read_csv("../data/raw/olist_order_items_dataset.csv")
order_payments = pd.read_csv("../data/raw/olist_order_payments_dataset.csv")
order_reviews = pd.read_csv("../data/raw/olist_order_reviews_dataset.csv")
orders = pd.read_csv("../data/raw/olist_orders_dataset.csv")
products = pd.read_csv("../data/raw/olist_products_dataset.csv")
sellers = pd.read_csv("../data/raw/olist_sellers_dataset.csv")
product_translation = pd.read_csv("../data/raw/product_category_name_translation.csv")
----
```

=== Merging Datasets with GeoLocation

```
+*In[ ]:*+
[source, ipython3]
----
geo_avg = (
    geolocation.groupby("geolocation_zip_code_prefix")["geolocation_lat", "geolocation_lng"]
    .mean()
    .reset_index()
)

sellers_inner = pd.merge(
    sellers,
    geo_avg,
    left_on="seller_zip_code_prefix",
    right_on="geolocation_zip_code_prefix",
```

```

        how="inner"
    ).drop(columns=["geolocation_zip_code_prefix"]).rename(columns={
        "geolocation_lat": "seller_lat",
        "geolocation_lng": "seller_lng"
    })

```

```

customers_inner = pd.merge(
    customers,
    geo_avg,
    left_on="customer_zip_code_prefix",
    right_on="geolocation_zip_code_prefix",
    how="inner"
).drop(columns=["geolocation_zip_code_prefix"]).rename(columns={
    "geolocation_lat": "customer_lat",
    "geolocation_lng": "customer_lng"
})

```

```

df_geo = pd.merge(orders, customers_inner, on="customer_id", how="inner")
df_geo = pd.merge(df_geo, order_items, on="order_id", how="inner")
df_geo = pd.merge(df_geo, sellers_inner, on="seller_id", how="inner")
df_geo = pd.merge(df_geo, order_payments, on="order_id", how="inner")
df_geo = pd.merge(df_geo, order_reviews, on="order_id", how="inner")
df_geo = pd.merge(df_geo, products, on="product_id", how="inner")
df_geo = pd.merge(df_geo, product_translation, on="product_category_name", how="inner")

```

```

df_geo.shape
----

```

=== Converting Dataset with GeoLocation to CSV

```

+*In[ ]:*+
[source, ipython3]
----
df_geo.to_csv("../data/raw/olist_geocombined_dataset.csv", index=False)
----

```

=== Merging Datasets without GeoLocation

```

+*In[ ]:*+
[source, ipython3]
----
df = pd.merge(orders, customers, on="customer_id", how="left")
df = pd.merge(df, order_payments, on="order_id", how="left")
df = pd.merge(df, order_reviews, on="order_id", how="left")
df = pd.merge(df, order_items, on="order_id", how="left")

```

```
df = pd.merge(df, products, on="product_id", how="left")
df = pd.merge(df, sellers, on="seller_id", how="left")
df = pd.merge(df, product_translation, on="product_category_name", how="left")
----
```

=== Converting Dataset to CSV

```
+*In[ ]:*+
[source, ipython3]
----
df.to_csv("../data/raw/olist_combined_dataset.csv", index=False)
----
```

Cleaning Dataset

```
## Importing Librariesimport pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split## Importing Datasetdata =
pd.read_csv("../data/raw/full_dataset.csv")
geolocation = pd.read_csv("../data/raw/olist_geolocation_dataset.csv")

display(data.head())
display(geolocation.head())## Merging Datasets with GeoLocationgeo_avg = (
    geolocation.groupby("geolocation_zip_code_prefix")[["geolocation_lat", "geolocation_lng"]]
    .mean()
    .reset_index()
)

geo_customer = geo_avg.rename(columns={
    "geolocation_zip_code_prefix": "customer_zip_code_prefix",
    "geolocation_lat": "customer_lat",
    "geolocation_lng": "customer_lng"
})

data = data.merge(geo_customer, on="customer_zip_code_prefix", how="inner")

geo_seller = geo_avg.rename(columns={
    "geolocation_zip_code_prefix": "seller_zip_code_prefix",
    "geolocation_lat": "seller_lat",
    "geolocation_lng": "seller_lng"
})

data = data.merge(geo_seller, on="seller_zip_code_prefix", how="inner")
```

```

data.head()def haversine(lat1, lng1, lat2, lng2):
    # convert degrees to radians
    lat1, lng1, lat2, lng2 = map(np.radians, [lat1, lng1, lat2, lng2])

    dlat = lat2 - lat1
    dlng = lng2 - lng1

    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlng/2)**2
    c = 2 * np.arcsin(np.sqrt(a))
    r = 6371 # Radius of Earth in kilometers
    return c * r

data["distance_km"] = haversine(
    data["customer_lat"], data["customer_lng"],
    data["seller_lat"], data["seller_lng"]
)
display(data.columns)
display(data.shape)

## Pre-Processing Data Columns
### Removing Unnecessary Columns# Rename the incorrect column name
data.rename(columns={"product_name_lenght": "product_name_length",
                    'product_description_lenght': 'product_description_length',
                    }, inplace=True)

data.drop(columns=['review_comment_title',
                  'review_comment_message',
                  'review_creation_date',
                  'review_answer_timestamp',
                  'product_category_name',
                  'payment_sequential',
                  'payment_type',
                  'payment_installments',
                  'payment_value'], inplace=True)

display(data.columns)

display(data.shape)### Removing Timestamps and Only Keeping Datesdatetime_columns =
[
    'order_purchase_timestamp', 'order_approved_at',
    'order_delivered_carrier_date', 'order_delivered_customer_date',
    'order_estimated_delivery_date', 'shipping_limit_date'
]

# Convert to datetime and strip time (keep only date)
for col in datetime_columns:
    if col in data.columns:
        data[col] = pd.to_datetime(data[col]).dt.date

```

```

# Display to verify changes
data.head()#### Creating Custom Columns# Convert both columns to datetime64[ns] before
subtraction
data['handoff_delay'] = (
    pd.to_datetime(data['order_delivered_carrier_date']) -
    pd.to_datetime(data['shipping_limit_date'])
).dt.days

data['delivery_delay'] = (
    pd.to_datetime(data['order_delivered_customer_date']) -
    pd.to_datetime(data['order_estimated_delivery_date'])
).dt.days

# Delivered late remains a boolean comparison
data['delivered_late'] = (
    pd.to_datetime(data['order_delivered_customer_date']) >
    pd.to_datetime(data['order_estimated_delivery_date'])
)

# Preview the results
display(data.head())display(data.columns)
display(data.shape)

#### Removing Duplicate Rows

# Remove fully duplicated rows
data.drop_duplicates(inplace=True)

# Reset index after dropping duplicates
data.reset_index(drop=True, inplace=True)

# Display updated DataFrame
display(data.head())
display(data.shape)
display(data.columns)

#### Calculating Product
Volumedata['product_volume'] = data['product_length_cm'] * data['product_height_cm'] *
data['product_width_cm']

## Re-Ordering Columns
# Define the final desired column order based on your prior instructions and the current
columns
final_column_order = [
    # Order and Customer Metadata
    'order_id', 'order_item_id', 'order_status', 'order_purchase_timestamp',
    'order_approved_at', 'order_estimated_delivery_date', 'is_canceled',

```

```

# Customer Information
'customer_id', 'customer_unique_id', 'customer_zip_code_prefix',
'customer_city', 'customer_state', 'customer_lat', 'customer_lng',
'order_delivered_customer_date', 'is_first_time_buyer',

# Seller Information
'seller_id', 'seller_zip_code_prefix', 'seller_city', 'seller_state',
'seller_lat', 'seller_lng', 'shipping_limit_date',
'order_delivered_carrier_date', 'handoff_delay',

# Distance Between Seller and Customer
'distance_km',

# Product Information
'product_id', 'product_category_name_english', 'price', 'freight_value',
'product_name_length', 'product_description_length', 'product_photos_qty',
'product_weight_g', 'product_length_cm', 'product_height_cm', 'product_width_cm',
'product_volume',

# Review Information
'review_id', 'review_score',

# Delivery Delay Information
'delivered_late', 'delivery_delay'
]

# Reorder the columns
data = data[final_column_order]

display(data.head())
display(data.shape)
display(data.columns)## Export to CSVdata.to_csv("../data/cleaned/cleaned_dataset.csv",
index=False)### Training and Testing File Split# Split without stratification — fair for all team
members
train_df, test_df = train_test_split(data, test_size=0.2, random_state=42)

# Save to CSV
train_df.to_csv("../data/cleaned/training.csv", index=False)
test_df.to_csv("../data/cleaned/testing.csv", index=False)

```

Exploratory Data Analysis (EDA)

```
## I. Introduction
```

This capstone project explores the Olist E-commerce dataset, which captures the full lifecycle of orders from a Brazilian online marketplace. The dataset merges customer, seller, order, product, review, and payment data to enable insights into delivery delays, seller behavior, and customer segmentation.

Our goal is to analyze delivery bottlenecks, identify patterns in seller handoffs, and understand customer behavior to improve platform reliability and user satisfaction.

II. Load Raw Datasets

We begin by loading each raw dataset provided by Olist, including orders, customers, payments, reviews, sellers, and products. This step helps us understand the structure and prepare for merging.

```
import pandas as pd
```

```
# Load each dataset from CSV files
```

```
customers = pd.read_csv('../data/raw/olist_customers_dataset.csv')
geolocation = pd.read_csv('../data/raw/olist_geolocation_dataset.csv')
order_items = pd.read_csv('../data/raw/olist_order_items_dataset.csv')
order_payments = pd.read_csv('../data/raw/olist_order_payments_dataset.csv')
order_reviews = pd.read_csv('../data/raw/olist_order_reviews_dataset.csv')
orders = pd.read_csv('../data/raw/olist_orders_dataset.csv')
products = pd.read_csv('../data/raw/olist_products_dataset.csv')
sellers = pd.read_csv('../data/raw/olist_sellers_dataset.csv')
```

```
# Display the shape of each dataset
```

```
print("Customers:", customers.shape)
print("Geolocation:", geolocation.shape)
print("Order Items:", order_items.shape)
print("Order Payments:", order_payments.shape)
print("Order Reviews:", order_reviews.shape)
print("Orders:", orders.shape)
print("Products:", products.shape)
print("Sellers:", sellers.shape)
```

III. Merge Datasets

In this step, we merge the raw datasets into a single unified DataFrame using `orders` as the central table.

We will join customers, order items, products, sellers, payments, and reviews using appropriate keys like `order\_id`, `customer\_id`, and `product\_id`.

```
# Step 1: Start with orders
```

```
df = orders.copy()
```

```
# Step 2: Merge with customers (on customer_id)
```

```
df = df.merge(customers, on='customer_id', how='left')
```

```
# Step 3: Merge with order_items (on order_id)
```

```
df = df.merge(order_items, on='order_id', how='left')
```

```
# Step 4: Merge with products (on product_id)
df = df.merge(products, on='product_id', how='left')

# Step 5: Merge with sellers (on seller_id)
df = df.merge(sellers, on='seller_id', how='left')

# Step 6: Merge with payments (on order_id)
df = df.merge(order_payments, on='order_id', how='left')

# Step 7: Merge with reviews (on order_id)
df = df.merge(order_reviews, on='order_id', how='left')

# Check shape and preview
print(df.shape)
df.head()
## IV. Basic Data Overview
```

Before diving into analysis, we examine the structure of the merged dataset. This includes checking the number of columns and rows, identifying column types, and spotting missing values.

```
# Check shape and columns
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())

# Check data types and non-null counts
df.info()
# Check missing values per column
missing = df.isnull().sum()
missing = missing[missing > 0]
print("Missing values per column:")
print(missing)
```

V. Data Cleaning

To ensure accurate and reliable analysis, we clean the merged dataset by addressing inconsistencies and formatting issues.

Cleaning Steps:

1. **Standardize City Names**: Normalize common spelling variations such as "Sao Paulo" → "São Paulo"
 2. **Trim Whitespace & Lowercase Standardization**: Remove leading/trailing spaces and unify string formatting.
 3. **Drop Nulls**: Remove rows with missing values in critical columns for analysis.
 4. **Outlier Handling**: Optionally filter out extreme values in `price`, `freight\_value`, or product dimensions.
- ```
from unidecode import unidecode
```

```

Convert to string and remove accents for both customer and seller cities
df['customer_city'] = df['customer_city'].fillna("").astype(str).apply(unidecode)
df['seller_city'] = df['seller_city'].fillna("").astype(str).apply(unidecode)

2. Trim whitespace and lowercase for consistency
str_columns = ['customer_city', 'customer_state', 'seller_city', 'seller_state', 'payment_type']
for col in str_columns:
 df[col] = df[col].str.strip().str.lower()

3. Drop rows with missing values in key columns
critical_columns = ['order_id', 'customer_id', 'order_status', 'price', 'freight_value']
df = df.dropna(subset=critical_columns)

4. Remove extreme outliers in price and freight_value
price_threshold = df['price'].quantile(0.995)
freight_threshold = df['freight_value'].quantile(0.995)

df = df[(df['price'] <= price_threshold) & (df['freight_value'] <= freight_threshold)]
VI. Create Delivery Delay Indicator

To evaluate delivery delays, we compare the actual delivery date
(`order_delivered_customer_date`) to the estimated date (`order_estimated_delivery_date`).
We create a binary column `delivered_late` to flag whether the delivery was late.
Convert delivery dates to datetime
df['order_delivered_customer_date'] = pd.to_datetime(df['order_delivered_customer_date'])
df['order_estimated_delivery_date'] = pd.to_datetime(df['order_estimated_delivery_date'])

Create 'delivered_late' column
df['delivered_late'] = df['order_delivered_customer_date'] >
df['order_estimated_delivery_date']

Handle missing dates (avoid True/False on NaT)
df['delivered_late'] = df['delivered_late'].fillna(False)

Check distribution
print(df['delivered_late'].value_counts(normalize=True))
A. Price vs Delivered Late

We compare the distribution of product prices between on-time and late deliveries using a
boxplot.
import seaborn as sns
import matplotlib.pyplot as plt

Boxplot for price vs delivery status
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='delivered_late', y='price')
plt.title('Product Price by Delivery Status')
plt.xlabel('Delivered Late')

```

```
plt.ylabel('Price')
plt.tight_layout()
plt.show()
```

The boxplot compares product prices between on-time and late deliveries.

We observe that the price distribution is relatively similar for both groups, with no significant shift in median or spread.

Both categories contain extreme outliers (high-price items), but these do not appear to strongly influence whether a delivery is delayed.

This suggests that product price alone is not a key driver of delivery delays.

#### ### B. Top Product Categories by Late Delivery Rate

We analyze which product categories have the highest proportion of late deliveries.

```
Load translation file
category_translation = pd.read_csv('../data/raw/product_category_name_translation.csv')

Merge with main dataframe on 'product_category_name'
df = df.merge(category_translation, on='product_category_name', how='left')

Calculate late delivery rate by English product category name
late_by_category =
df.groupby('product_category_name_english')['delivered_late'].mean().sort_values(ascending=False).head(10)

Barplot
plt.figure(figsize=(10, 5))
sns.barplot(x=late_by_category.values, y=late_by_category.index)
plt.title('Top 10 Product Categories by Late Delivery Rate')
plt.xlabel('Late Delivery Rate')
plt.ylabel('Product Category')
plt.tight_layout()
plt.show()
```

The bar chart highlights the top 10 product categories with the highest late delivery rates.

Categories like `home\_comfort\_2`, `audio`, and `furniture\_mattress\_and\_upholstery` stand out with delay rates exceeding 10–15%.

These categories likely involve bulky or fragile items that require special handling, which may contribute to logistical challenges and delivery delays.

This insight helps identify product types that may need better inventory or shipping management to reduce customer dissatisfaction.

#### ## VII. Analyze Seller Handoff Delays

We define `handoff\_delay` as the number of days between the order approval date and the date the seller hands off the order to the carrier.

This helps identify how promptly sellers process orders for shipment.

```
Convert relevant columns to datetime
```

```

df['order_approved_at'] = pd.to_datetime(df['order_approved_at'])
df['order_delivered_carrier_date'] = pd.to_datetime(df['order_delivered_carrier_date'])

Calculate handoff delay in days
df['handoff_delay'] = (df['order_delivered_carrier_date'] - df['order_approved_at']).dt.days

Check basic stats
print(df['handoff_delay'].describe())
A. Distribution of Seller Handoff Delay

```

We visualize the distribution of handoff delays to understand how quickly sellers process orders.

Extreme outliers (e.g., delays over 15 days or negative delays) will be excluded for better clarity.

# Filter handoff\_delay between -10 and 15 days for cleaner visualization

```

filtered = df[(df['handoff_delay'] >= -10) & (df['handoff_delay'] <= 15)]

```

```

Plot histogram
plt.figure(figsize=(8, 5))
sns.histplot(filtered['handoff_delay'], bins=25, kde=False)
plt.title('Seller Handoff Delay (Filtered: -10 to 15 Days)')
plt.xlabel('Handoff Delay (Days)')
plt.ylabel('Number of Orders')
plt.tight_layout()
plt.show()

```

The histogram shows the distribution of seller handoff delays, filtered between -10 and 15 days for clarity.

Most sellers hand off orders within 1 to 3 days after approval, with a noticeable right-skewed tail indicating longer delays in some cases.

The concentration around 0–3 days suggests that sellers generally process orders promptly, but a non-negligible number experience delays up to two weeks.

Understanding this distribution helps flag inefficient sellers and anticipate delivery risk.

#### B. Relationship Between Handoff Delay and Delivery Delay

To determine if seller delays impact final delivery outcomes, we compare handoff delays for orders that were delivered late versus on time.

```

plt.figure(figsize=(8, 5))
sns.boxplot(x='delivered_late', y='handoff_delay', data=df)
plt.title('Seller Handoff Delay by Delivery Status')
plt.xlabel('Delivered Late')
plt.ylabel('Handoff Delay (Days)')
plt.ylim(-5, 15) # Adjust based on your typical range
plt.tight_layout()
plt.show()

```

The boxplot above illustrates the distribution of seller handoff delays for on-time (`False`) and late (`True`) deliveries.

We observe that orders with late deliveries tend to have a higher median handoff delay and a wider spread compared to on-time deliveries. This suggests that delays in passing orders from sellers to carriers may contribute to overall delivery lateness.

#### #### C. Top Sellers by Average Handoff Delay

To identify sellers who may be contributing to delivery delays, we calculate the average handoff delay for each seller.

This helps pinpoint which sellers are consistently slower in processing and handing off orders to the carrier.

By highlighting these sellers, the platform can target process improvements or performance reviews to improve shipping reliability.

# Calculate average handoff delay by seller

```
top_sellers_delay =
```

```
df.groupby('seller_id')['handoff_delay'].mean().sort_values(ascending=False).head(10)
```

# Plot bar chart of top 10 sellers with highest average handoff delay

```
plt.figure(figsize=(10, 5))
```

```
sns.barplot(x=top_sellers_delay.values, y=top_sellers_delay.index)
```

```
plt.title('Top 10 Sellers by Average Handoff Delay')
```

```
plt.xlabel('Average Handoff Delay (Days)')
```

```
plt.ylabel('Seller ID')
```

```
plt.tight_layout()
```

```
plt.show()
```

This bar chart shows the top 10 sellers with the highest average handoff delays — the time between order approval and the handoff to the carrier.

Each bar represents a seller (anonymized by ID) and their average delay in days. Sellers at the top of the chart consistently take longer to process orders, which may contribute to delivery delays.

These insights can help the platform identify underperforming sellers and take corrective actions to improve fulfillment efficiency.

#### #### D. Regions with Most Delivery Delays

To uncover geographical trends in delivery performance, we analyze which customer **states** and **cities** have the highest late delivery rates.

Identifying regional hotspots of delivery issues can help inform logistical improvements and resource allocation strategies.

# Top 10 States by Late Delivery Rate

```
late_by_state =
```

```
df.groupby('customer_state')['delivered_late'].mean().sort_values(ascending=False).head(10)
```

```
plt.figure(figsize=(10, 5))
```

```
sns.barplot(x=late_by_state.values, y=late_by_state.index)
```

```
plt.title('Top 10 States by Late Delivery Rate')
plt.xlabel('Late Delivery Rate')
plt.ylabel('Customer State')
plt.tight_layout()
plt.show()
```

```
Top 10 Cities by Late Delivery Rate (filter to cities with at least 100 orders)
city_counts = df['customer_city'].value_counts()
valid_cities = city_counts[city_counts >= 100].index
filtered_df = df[df['customer_city'].isin(valid_cities)]
```

```
late_by_city =
filtered_df.groupby('customer_city')['delivered_late'].mean().sort_values(ascending=False).head(10)
```

```
plt.figure(figsize=(10, 5))
sns.barplot(x=late_by_city.values, y=late_by_city.index)
plt.title('Top 10 Cities by Late Delivery Rate (Min. 100 Orders)')
plt.xlabel('Late Delivery Rate')
plt.ylabel('Customer City')
plt.tight_layout()
plt.show()
```

We identify the top customer **states** and **cities** with the highest late delivery rates.

- **States** like `al` (Alagoas), `ma` (Maranhão), and `pi` (Piauí) show the highest rates of delayed deliveries, all exceeding 15–20%.
- **Cities** such as `maceio`, `sao goncalo`, and `sao luis` have notably high delay rates, with `maceio` approaching 30%.

These insights suggest potential regional logistical bottlenecks, possibly due to infrastructure challenges or distance from major distribution hubs.

Understanding these trends can help prioritize regional improvements for better delivery performance.

#### #### VIII. Segment Customer Behavior

##### ##### A. Flag First-Time Buyers

To segment customers, we identify first-time buyers by counting the number of orders associated with each `customer\_id`.

Customers with only one order are flagged as `True` in the `is\_first\_time\_buyer` column.

# Count total orders per customer

```
customer_order_counts = orders.groupby('customer_id')['order_id'].count()
```

# Create a new DataFrame to flag first-time buyers

```
first_time_flags = customer_order_counts == 1
```

```
first_time_flags = first_time_flags.reset_index().rename(columns={'order_id':
'is_first_time_buyer'})
```

```
Merge the flag into the main DataFrame
df = df.merge(first_time_flags, on='customer_id', how='left')
B. Compare Cancellation Rate by Customer Type
```

We analyze whether first-time buyers have different cancellation behavior compared to repeat customers.

The bar chart below shows the average cancellation rate for each group.

```
Create 'is_canceled' column based on the order status
This column is True if the order was canceled, otherwise False
df['is_canceled'] = df['order_status'] == 'canceled'
Count number of orders per customer using customer_id
customer_order_counts = df.groupby('customer_id')['order_id'].transform('count')

First-time buyer is someone who has exactly one order
df['is_first_time_buyer'] = customer_order_counts == 1
Calculate cancellation rate for first-time vs. repeat buyers
cancel_rate_by_type = df.groupby('is_first_time_buyer')['is_canceled'].mean()
```

```
Plot the comparison
plt.figure(figsize=(6, 5))
cancel_rate_by_type.plot(kind='bar', color=['#66c2a5', '#fc8d62'])
plt.title('Cancellation Rate by Buyer Type')
plt.xlabel('Is First Time Buyer')
plt.ylabel('Cancellation Rate')
plt.xticks(ticks=[0, 1], labels=['Repeat Buyer', 'First-Time Buyer'], rotation=0)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

This bar chart compares the **cancellation rate** between **first-time buyers** and **repeat buyers**.

To calculate this:

- We created an `is\_canceled` column where orders with the status `canceled` are marked `True`.
- We also flagged customers as **first-time buyers** if they had only one order in the dataset, using **cleaned customer data** for improved accuracy.

The visualization now reveals that **repeat buyers have a slightly higher cancellation rate** compared to first-time buyers.

This could indicate that experienced customers may cancel more often due to **familiarity with return processes** or **changing expectations** based on prior experience.

## IX. Summary & Insights

### 🔍 RQ1: What key factors lead to late deliveries?

- **Price vs. Delivery Delay**

Lower-priced orders tend to have slightly higher late delivery rates, but no strong correlation was found.

- **Product Categories**

Categories such as `bed\_bath\_table`, `health\_beauty`, and `sports\_leisure` show higher proportions of late deliveries, pointing to category-specific logistics challenges.

- **Geographic Patterns**

States like `SP` and `RJ` have the highest number of delays due to volume, while some smaller states show disproportionately high delay rates when normalized, suggesting regional fulfillment issues.

---

### 🔍 RQ2: Which sellers consistently delay order handoffs to carriers?

- **Handoff Delay Distribution**

Most sellers dispatch orders quickly, but some outliers take over 10 days. These long delays significantly contribute to overall delivery lateness.

- **Impact on Delivery**

A clear relationship exists: longer seller handoff delays are strongly associated with late deliveries, indicating seller behavior as a key bottleneck.

---

### 🔍 RQ3: How do first-time vs. repeat buyers differ in cancellation behavior?

- **Cancellation Rate**

First-time buyers cancel orders more frequently than repeat buyers, possibly due to unfamiliarity with the platform or uncertainty in the purchase process.

- **Behavioral Insight**

Enhancing the onboarding experience and building trust with new users could reduce cancellations and improve retention.

## X. Conclusion & Recommendations

### Conclusion

This project examined the Olist E-commerce dataset to uncover delivery inefficiencies, seller-related delays, and patterns in customer behavior. Through extensive EDA and visual analysis, we derived key insights about late deliveries, seller handoff delays, and buyer segmentation.

Key findings include:

- Late deliveries are strongly associated with long carrier handoff delays.
- A small number of sellers are consistently responsible for longer handoff delays.
- First-time buyers have a higher cancellation rate compared to repeat buyers.

### ### Recommendations

Based on the analysis, we propose the following:

- **Monitor and support high-delay sellers:** Target the small group of sellers who frequently delay handoffs with automated alerts or performance incentives.
- **Predictive delivery systems:** Use early handoff patterns to anticipate late deliveries and proactively update customers.
- **Customer retention strategies:** As first-time buyers cancel more often, introduce loyalty incentives or early customer support for first-time orders.
- **Improve estimated delivery accuracy:** Recalibrate delivery estimates using historical delay patterns to better manage customer expectations.

These recommendations aim to improve user satisfaction, reduce operational inefficiencies, and enhance the overall reliability of the platform.

# Export full dataset with EDA columns

```
df.to_csv('../data/raw/full_dataset.csv', index=False)
```

## RQ1

**## What Key Factors Lead to Late Deliveries, and How Can We Proactively Identify High-Risk Orders to Improve Customer Satisfaction?**

=== Importing Libraries

```
+*In[]:*+
```

```
[source, ipython3]
```

```

```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
sns.set()
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
from imblearn.over_sampling import SMOTE
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from xgboost import XGBClassifier, plot_importance
```

```
from sklearn.model_selection import StratifiedKFold, GridSearchCV
```

```

from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score,
roc_curve
from sklearn.metrics import make_scorer, recall_score, precision_recall_curve,
average_precision_score

```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from sklearn.calibration import CalibratedClassifierCV
from tensorflow.keras.losses import BinaryFocalCrossentropy
from scikeras.wrappers import KerasClassifier

```

```

from sklearn.metrics import fbeta_score

```

```

```

```

=== Importing Dataset

```

```

+*In[]:*+

```

```

[source, ipython3]

```

```

```

```

data = pd.read_csv("../data/cleaned/cleaned_dataset.csv")

```

```

data.head()

```

```

```

```

+*In[]:*+

```

```

[source, ipython3]

```

```

```

```

data.columns

```

```

```

```

=== Pre-Processing

```

```

==== Removing Unnecessary Columns

```

```

+*In[]:*+

```

```

[source, ipython3]

```

```

```

```

data.drop(columns=['order_id',
 'order_item_id',
 'order_status',
 'order_purchase_timestamp',
 'order_approved_at',
 'order_estimated_delivery_date'],

```

```

 'is_canceled',
 'customer_id',
 'customer_unique_id',
 'customer_zip_code_prefix',
 'customer_state',
 'customer_lat',
 'customer_lng',
 'order_delivered_customer_date',
 'is_first_time_buyer',
 'seller_zip_code_prefix',
 'seller_state',
 'seller_lat',
 'seller_lng',
 'shipping_limit_date',
 'order_delivered_carrier_date',
 'price',
 'freight_value',
 'product_name_length',
 'product_description_length',
 'product_photos_qty',
 'review_id',
 'review_score',
 'product_id',
 'delivery_delay'], inplace=True)

```

```
display(data.head())
```

```
display(data.shape)
```

```

```

```
==== Removing NA Rows
```

```

+*In[]:*+

```

```
[source, ipython3]
```

```

```

```
data = data.dropna()
```

```
display(data.shape)
```

```

```

```
==== Variable Selection
```

```

+*In[]:*+

```

```
[source, ipython3]
```

```

```

```
X = data.drop(columns=['delivered_late'])
```

```
y = data['delivered_late']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
```

```

```

```
==== Encoding Variables
```

```
+*In[]:*
```

```
[source, ipython3]
```

```

```

```
categorical_features = ['customer_city',
 'seller_city',
 'seller_id',
 'product_category_name_english']
```

```
numerical_features = ['handoff_delay', 'distance_km', 'product_weight_g',
 'product_length_cm', 'product_height_cm',
 'product_width_cm', 'product_volume']
```

```
preprocessor = ColumnTransformer([
 ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=True),
categorical_features),
 ('num', StandardScaler(), numerical_features)
])
```

```
Transform
```

```
X_train_encoded = preprocessor.fit_transform(X_train)
```

```
X_test_encoded = preprocessor.transform(X_test)
```

```

```

```
==== Getting Feature Names
```

```
+*In[]:*
```

```
[source, ipython3]
```

```

```

```
if hasattr(X_train_encoded, "toarray"):
 X_train_encoded = X_train_encoded.toarray()
 X_test_encoded = X_test_encoded.toarray()
```

```
Get column names
```

```
cat_feature_names =
```

```
preprocessor.named_transformers_['cat'].get_feature_names_out(categorical_features)
```

```
all_feature_names = list(cat_feature_names) + numerical_features
```

```
Convert to DataFrames
```

```

X_train_df = pd.DataFrame(X_train_encoded, columns=all_feature_names)
X_test_df = pd.DataFrame(X_test_encoded, columns=all_feature_names)

===== Applying SMOTE

+*In[]:*+
[source, ipython3]

smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_df, y_train)

===== Applying StratifiedKFold

+*In[]:*+
[source, ipython3]

stratified_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

===== Focusing Recall for the Positive Late Deliveries

+*In[]:*+
[source, ipython3]

recall_scorer = make_scorer(recall_score, pos_label=True)

=== Logistic Regression

===== Creating the Best Model

+*In[]:*+
[source, ipython3]

Logistic regression model
log_reg = LogisticRegression(max_iter=1000, solver='liblinear', random_state=42)

Hyperparameter grid (penalty type and C values)
param_grid = {
 'C': [0.01, 0.1, 1, 10],
 'penalty': ['l1', 'l2'] # Try both L1 and L2 regularization
}

```

```

Grid search setup
grid_log_reg = GridSearchCV(
 estimator=log_reg,
 param_grid=param_grid,
 scoring=recall_scorer,
 cv=stratified_cv,
 verbose=2,
 n_jobs=1
)

Fit model
grid_log_reg.fit(X_train_resampled, y_train_resampled)

Extract best model and params
best_log_reg = grid_log_reg.best_estimator_
print("Best Parameters:", grid_log_reg.best_params_)

==== Testing the Model

+*In[]:*+
[source, ipython3]

Predict probabilities
y_proba_logreg = best_log_reg.predict_proba(X_test_df)[:, 1]

=== Threshold Optimization ===
precision, recall, thresholds = precision_recall_curve(y_test, y_proba_logreg)
f2_scores = 5 * (precision * recall) / (4 * precision + recall + 1e-6)
optimal_threshold_logreg = thresholds[np.argmax(f2_scores)]
print(f"\nOptimal Threshold (LogReg): {optimal_threshold_logreg:.4f}")

Apply optimal threshold
y_pred_logreg_opt = (y_proba_logreg >= optimal_threshold_logreg).astype(int)

Metrics
print("\nClassification Report (Logistic Regression):")
print(classification_report(y_test, y_pred_logreg_opt))

roc_auc = roc_auc_score(y_test, y_proba_logreg)
print(f"ROC-AUC Score: {roc_auc:.4f}")

plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred_logreg_opt), annot=True, fmt='d',
cmap='Blues')
plt.title("Confusion Matrix: Logistic Regression")

```

```

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

fpr, tpr, _ = roc_curve(y_test, y_proba_logreg)
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, label=f"LogReg AUC = {roc_auc:.2f}", color='blue')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Logistic Regression")
plt.legend()
plt.tight_layout()
plt.show()

==== Log-Odds

+*In[]:*+
[source, ipython3]

=== Logistic Regression Coefficients (Log-Odds) ===
log_odds = pd.Series(best_log_reg.coef_[0], index=X_train_df.columns)
log_odds_sorted = log_odds.sort_values(key=abs, ascending=False)

Display top 15 features with strongest influence
print("\nTop Logistic Regression Coefficients (Log-Odds):")
display(log_odds_sorted.head(15))

Plot
plt.figure(figsize=(10, 6))
log_odds_sorted.head(20).plot(kind='barh', color='skyblue')
plt.title("Top 20 Features by Log-Odds (Logistic Regression)")
plt.xlabel("Coefficient (Log-Odds)")
plt.gca().invert_yaxis()
plt.show()

=== Decision Tree

==== Creating the Best Model

+*In[]:*+
[source, ipython3]

```

```

Define the Decision Tree Classifier
tree_clf = DecisionTreeClassifier(random_state=42)

Define hyperparameter grid
param_grid_tree = {
 'max_depth': [5, 10, 15],
 'min_samples_split': [10, 20],
 'criterion': ['gini', 'entropy']
}

Grid search
grid_tree = GridSearchCV(
 estimator=tree_clf,
 param_grid=param_grid_tree,
 scoring=recall_scorer,
 cv=stratified_cv,
 verbose=2,
 n_jobs=1
)

Fit to the resampled training set
grid_tree.fit(X_train_resampled, y_train_resampled)

Best model and parameters
best_tree = grid_tree.best_estimator_
print("Best Parameters:", grid_tree.best_params_)

===== Evaluating the Model

+*In[]:*+
[source, ipython3]

Predict probabilities
y_proba_tree = best_tree.predict_proba(X_test_df)[:, 1]

=== Threshold Optimization ===
precision_tree, recall_tree, thresholds_tree = precision_recall_curve(y_test, y_proba_tree)
f2_scores_tree = 5 * (precision_tree * recall_tree) / (4 * precision_tree + recall_tree + 1e-6)
optimal_threshold_tree = thresholds_tree[np.argmax(f2_scores_tree)]
print(f"\nOptimal Threshold (Decision Tree): {optimal_threshold_tree:.4f}")

Apply optimal threshold
y_pred_tree_opt = (y_proba_tree >= optimal_threshold_tree).astype(int)

Metrics
print("\nClassification Report (Decision Tree):")

```

```

print(classification_report(y_test, y_pred_tree_opt))

roc_auc_tree = roc_auc_score(y_test, y_proba_tree)
print(f"ROC-AUC Score (Tree): {roc_auc_tree:.4f}")

Visualizing Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred_tree_opt), annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix: Decision Tree")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

ROC Curve
fpr_tree, tpr_tree, _ = roc_curve(y_test, y_proba_tree)
plt.figure(figsize=(6, 5))
plt.plot(fpr_tree, tpr_tree, label=f"Decision Tree AUC = {roc_auc_tree:.2f}",
color='darkorange')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Decision Tree")
plt.legend()
plt.tight_layout()
plt.show()

Visualizing the Decision Tree
plt.figure(figsize=(20, 12))
plot_tree(
 best_tree,
 feature_names=all_feature_names,
 class_names=['On Time', 'Late'],
 filled=True,
 rounded=True,
 fontsize=10,
 max_depth=3 # limit depth for readability, remove or increase to see more
)
plt.title("Decision Tree Visualization (Top Levels)")
plt.show()

=== Random Forest

==== Creating the Best Model

+*In[]:*+

```

```

[source, ipython3]

Random Forest
rf_clf = RandomForestClassifier(random_state=42)

param_grid_rf = {
 'n_estimators': [100, 200],
 'max_depth': [10, 15],
 'min_samples_split': [10, 20],
 'class_weight': ['balanced'] # Helps handle imbalance without SMOTE here, but still okay
 if you keep SMOTE
}

grid_rf = GridSearchCV(
 estimator=rf_clf,
 param_grid=param_grid_rf,
 scoring=recall_scorer,
 cv=stratified_cv,
 verbose=2,
 n_jobs=1
)

grid_rf.fit(X_train_resampled, y_train_resampled)
best_rf = grid_rf.best_estimator_
print("Best Parameters (Random Forest):", grid_rf.best_params_)

===== Evaluating the Model

+*In[]:*+
[source, ipython3]

Predict probabilities
y_proba_rf = best_rf.predict_proba(X_test_df)[:, 1]

=== Threshold Optimization ===
precision_rf, recall_rf, thresholds_rf = precision_recall_curve(y_test, y_proba_rf)
f2_scores_rf = 5 * (precision_rf * recall_rf) / (4 * precision_rf + recall_rf + 1e-6)
optimal_threshold_rf = thresholds_rf[np.argmax(f2_scores_rf)]
print(f"\nOptimal Threshold (Random Forest): {optimal_threshold_rf:.4f}")

Apply optimal threshold
y_pred_rf_opt = (y_proba_rf >= optimal_threshold_rf).astype(int)

Metrics
print("\nClassification Report (Random Forest):")
print(classification_report(y_test, y_pred_rf_opt))

```

```

roc_auc_rf = roc_auc_score(y_test, y_proba_rf)
print(f"ROC-AUC Score (Random Forest): {roc_auc_rf:.4f}")

plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred_rf_opt), annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix: Random Forest")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

fpr_rf, tpr_rf, _ = roc_curve(y_test, y_proba_rf)
plt.figure(figsize=(6, 5))
plt.plot(fpr_rf, tpr_rf, label=f"Random Forest AUC = {roc_auc_rf:.2f}", color='green')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Random Forest")
plt.legend()
plt.tight_layout()
plt.show()

=== XGBoost

==== Creating the Model

+*In[]:*+
[source, ipython3]

xgb_clf = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
param_grid_xgb = {
 'n_estimators': [100, 150],
 'max_depth': [3, 5, 7],
 'learning_rate': [0.01, 0.1, 0.2],
 'subsample': [0.8, 1.0]
}

grid_xgb = GridSearchCV(
 estimator=xgb_clf,
 param_grid=param_grid_xgb,
 scoring=recall_scorer,
 cv=stratified_cv,
 verbose=2,
 n_jobs=1
)

```

```

grid_xgb.fit(X_train_resampled, y_train_resampled)
best_xgb = grid_xgb.best_estimator_
print("Best Parameters (XGBoost):", grid_xgb.best_params_)

==== Evaluating the Model

+*In[]:*+
[source, ipython3]

Predict probabilities
y_proba_xgb = best_xgb.predict_proba(X_test_df)[:, 1]

Threshold Optimization
precision_xgb, recall_xgb, thresholds_xgb = precision_recall_curve(y_test, y_proba_xgb)
f2_scores_xgb = 5 * (precision_xgb * recall_xgb) / (4 * precision_xgb + recall_xgb + 1e-6)
optimal_threshold_xgb = thresholds_xgb[np.argmax(f2_scores_xgb)]
print(f"\nOptimal Threshold (XGBoost): {optimal_threshold_xgb:.4f}")

Apply threshold
y_pred_xgb_opt = (y_proba_xgb >= optimal_threshold_xgb).astype(int)

Metrics
print("\nClassification Report (XGBoost):")
print(classification_report(y_test, y_pred_xgb_opt))

roc_auc_xgb = roc_auc_score(y_test, y_proba_xgb)
print(f"ROC-AUC Score (XGBoost): {roc_auc_xgb:.4f}")

plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred_xgb_opt), annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix: XGBoost")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

fpr_xgb, tpr_xgb, _ = roc_curve(y_test, y_proba_xgb)
plt.figure(figsize=(6, 5))
plt.plot(fpr_xgb, tpr_xgb, label=f"XGBoost AUC = {roc_auc_xgb:.2f}", color='purple')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - XGBoost")
plt.legend()
plt.tight_layout()

```

```
plt.show()
```

```

```

```
=== ANN
```

```
==== Creating the Model
```

```
+*In[]:*+
```

```
[source, ipython3]
```

```

```

```
Define the ANN model
```

```
def create_complex_ann():
```

```
 model = Sequential([
```

```
 Dense(256, activation='relu', input_shape=(X_train_resampled.shape[1,])),
```

```
 BatchNormalization(),
```

```
 Dropout(0.4),
```

```
 Dense(128, activation='relu'),
```

```
 BatchNormalization(),
```

```
 Dropout(0.3),
```

```
 Dense(64, activation='relu'),
```

```
 BatchNormalization(),
```

```
 Dropout(0.3),
```

```
 Dense(1, activation='sigmoid')
```

```
])
```

```
 focal_loss = BinaryFocalCrossentropy(gamma=2.0)
```

```
 model.compile(optimizer=Adam(learning_rate=0.001), loss=focal_loss, metrics=['Recall'])
```

```
 return model
```

```
Wrap in KerasClassifier for calibration
```

```
keras_clf = KerasClassifier(model=create_complex_ann, epochs=15, batch_size=32,
verbose=2)
```

```
keras_clf.fit(X_train_resampled, y_train_resampled)
```

```

```

```
==== Evaluating the Model
```

```
+*In[]:*+
```

```
[source, ipython3]
```

```

```

```
Predict probabilities
```

```
y_proba_ann = keras_clf.predict_proba(X_test_df)[:, 1]
```

```
Threshold Optimization
```

```
precision_ann, recall_ann, thresholds_ann = precision_recall_curve(y_test, y_proba_ann)
```

```
f2_scores_ann = 5 * (precision_ann * recall_ann) / (4 * precision_ann + recall_ann + 1e-6)
```

```
optimal_threshold_ann = thresholds_ann[np.argmax(f2_scores_ann)]
```

```

print(f"\nOptimal Threshold (ANN): {optimal_threshold_ann:.4f}")

Apply threshold
y_pred_ann_opt = (y_proba_ann >= optimal_threshold_ann).astype(int)

Metrics
print("\nClassification Report (ANN):")
print(classification_report(y_test, y_pred_ann_opt))

roc_auc_ann = roc_auc_score(y_test, y_proba_ann)
print(f"ROC-AUC Score (ANN): {roc_auc_ann:.4f}")

Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred_ann_opt), annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix: ANN (Focal Loss)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

ROC Curve
fpr_ann, tpr_ann, _ = roc_curve(y_test, y_proba_ann)
plt.figure(figsize=(6, 5))
plt.plot(fpr_ann, tpr_ann, label=f"ANN AUC = {roc_auc_ann:.2f}", color='green')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - ANN (Focal Loss)")
plt.legend()
plt.tight_layout()
plt.show()

=== Model Comparison

==== Classification Reports

+*In[]:*+
[source, ipython3]

print("Classification Report - Logistic Regression")
print(classification_report(y_test, y_pred_logreg_opt))

print("\nClassification Report - Decision Tree")
print(classification_report(y_test, y_pred_tree_opt))

```

```

print("\nClassification Report - Random Forest")
print(classification_report(y_test, y_pred_rf_opt))

print("\nClassification Report - XGBoost")
print(classification_report(y_test, y_pred_xgb_opt))

print("\nClassification Report - ANN")
print(classification_report(y_test, y_pred_ann_opt))

==== General Comparison

+*In[]:*+
[source, ipython3]

def compute_f2(y_true, y_pred):
 return fbeta_score(y_true, y_pred, beta=2)

model_results = pd.DataFrame({
 "Model": ["Logistic Regression", "Decision Tree", "Random Forest", "XGBoost", "ANN"],
 "F2 Score": [
 compute_f2(y_test, y_pred_logreg_opt),
 compute_f2(y_test, y_pred_tree_opt),
 compute_f2(y_test, y_pred_rf_opt),
 compute_f2(y_test, y_pred_xgb_opt),
 compute_f2(y_test, y_pred_ann_opt)
],
 "Recall": [
 recall_score(y_test, y_pred_logreg_opt),
 recall_score(y_test, y_pred_tree_opt),
 recall_score(y_test, y_pred_rf_opt),
 recall_score(y_test, y_pred_xgb_opt),
 recall_score(y_test, y_pred_ann_opt)
],
 "ROC AUC": [
 roc_auc,
 roc_auc_tree,
 roc_auc_rf,
 roc_auc_xgb,
 roc_auc_ann
]
})

display(model_results.sort_values(by="F2 Score",
ascending=False).reset_index(drop=True))

```

#### ==== ROC Curve Comparison

```
+*In[]:*+
[source, ipython3]

Compute ROC curve data for each model
roc_data = []

Logistic Regression
fpr, tpr, _ = roc_curve(y_test, y_proba_logreg)
roc_data.append(("Logistic Regression", fpr, tpr, roc_auc))

Decision Tree
fpr, tpr, _ = roc_curve(y_test, y_proba_tree)
roc_data.append(("Decision Tree", fpr, tpr, roc_auc_tree))

Random Forest
fpr, tpr, _ = roc_curve(y_test, y_proba_rf)
roc_data.append(("Random Forest", fpr, tpr, roc_auc_rf))

XGBoost
fpr, tpr, _ = roc_curve(y_test, y_proba_xgb)
roc_data.append(("XGBoost", fpr, tpr, roc_auc_xgb))

ANN
fpr, tpr, _ = roc_curve(y_test, y_proba_ann)
roc_data.append(("ANN", fpr, tpr, roc_auc_ann))

Plot
plt.figure(figsize=(10, 8))
for name, fpr, tpr, auc_score in roc_data:
 plt.plot(fpr, tpr, label=f"{name} (AUC = {auc_score:.2f})")

plt.plot([0, 1], [0, 1], 'k--', label="Random Classifier")
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves - All Models')
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
plt.show()

```

#### ==== Precision Recall Curve

```
+*In[]:*+
```

```

[source, ipython3]

Compute Precision-Recall curve data
pr_data = []

Logistic Regression
precision, recall, _ = precision_recall_curve(y_test, y_proba_logreg)
ap_logreg = average_precision_score(y_test, y_proba_logreg)
pr_data.append(("Logistic Regression", precision, recall, ap_logreg))

Decision Tree
precision, recall, _ = precision_recall_curve(y_test, y_proba_tree)
ap_tree = average_precision_score(y_test, y_proba_tree)
pr_data.append(("Decision Tree", precision, recall, ap_tree))

Random Forest
precision, recall, _ = precision_recall_curve(y_test, y_proba_rf)
ap_rf = average_precision_score(y_test, y_proba_rf)
pr_data.append(("Random Forest", precision, recall, ap_rf))

XGBoost
precision, recall, _ = precision_recall_curve(y_test, y_proba_xgb)
ap_xgb = average_precision_score(y_test, y_proba_xgb)
pr_data.append(("XGBoost", precision, recall, ap_xgb))

ANN
precision, recall, _ = precision_recall_curve(y_test, y_proba_ann)
ap_ann = average_precision_score(y_test, y_proba_ann)
pr_data.append(("ANN", precision, recall, ap_ann))

Plot
plt.figure(figsize=(10, 8))
for name, precision, recall, ap_score in pr_data:
 plt.plot(recall, precision, label=f"{name} (AP = {ap_score:.2f})")

plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curves - All Models')
plt.legend(loc='upper right')
plt.grid(True)
plt.tight_layout()
plt.show()

=== Feature Importance

+*In[]:*+

```

```
[source, ipython3]

plt.figure(figsize=(10, 8))
plot_importance(best_xgb, max_num_features=20, importance_type='gain',
show_values=False)
plt.title("Top 20 Feature Importances - XGBoost")
plt.show()

```

## RQ2

The second part of the original question has been reworded for clarity, and will be the focus of the first part of the investigation:

[arabic]

- . Can delays in seller handoff to the logistics carrier help us anticipate customer-facing delivery issues?
- . Which sellers consistently delay order handoffs to carriers, and what underlying patterns or behaviors contribute to this?

=== Part 0: Data Manipulation and preparation

```
+*In[]:*+
[source, ipython3]

Load libraries and functions
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib_venn as mv
import seaborn as sns
import statsmodels.api as sm

from imblearn.over_sampling import SMOTE
from scipy.stats import spearmanr, chi2_contingency
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.compose import ColumnTransformer
from sklearn.feature_selection import RFE
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.metrics import precision_recall_curve, accuracy_score, recall_score,
confusion_matrix, fbeta_score, make_scorer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from statsmodels.stats import proportion as smsp
```

```
Suppress unnecessary warnings
import warnings
warnings.filterwarnings('ignore');

```

==== Import and Manipulate Baseline

```
+*In[]:*+
[source, ipython3]

Set directory path and import team's cleaned data set
os.chdir(r'C:\Users\lopiwa\Documents\ASU\DAT_490\Data') # using raw literal to allow
copy/paste from Windows paths
data = pd.read_csv('cleaned_dataset.csv', index_col = False, low_memory = False);

```

We will drop the following irrelevant columns since we have already generated an approximate distance between seller and customer in the 'distance++\_++km' field:

- \* customer++\_++lat & customer++\_++lng - the latitude and longitude coordinates of the customer
- \* seller++\_++lat & seller++\_++lng - the latitude and longitude coordinates of the seller

We also drop 'is++\_++first++\_++time++\_++buyer' as that has no logical reason to affect shippers' effectiveness at handing off deliveries. Finally, we drop the "informational" product fields (length of name, length of description, number of photos) as well as the remaining review information, as neither of those groups are relevant to the question at hand.

```
+*In[]:*+
[source, ipython3]

data.drop(columns = ['customer_lat', 'customer_lng', 'is_first_time_buyer',
 'seller_lat', 'seller_lng', 'product_name_length', 'product_description_length',
 'product_photos_qty', 'review_id', 'review_score'], inplace=True);

```

```
+*In[]:*+
[source, ipython3]

data.columns
```

----

There are features left of questionable relevance, but we will keep them just in case they become important as the analysis proceeds.

To reduce the complexity of the set, we next extract the unique values within the `order_id`, `customer_id`, `customer_unique_id`, and `seller_id` columns so we can assign integer values to them in place of the 32-character hexadecimal values that are currently in place, since we have no need to trace those values back to the original databases. We then join the resulting individual dataframes with the larger data set, dropping the original columns and leaving only the integer IDs.

```
+*In[]:*
```

```
[source, ipython3]
```

```

```

```
Create dataframes from extracted unique values of each ID column in the overall 'data'
dataset
order = pd.DataFrame(data['order_id'].unique(), columns = ['order_id'])
cust = pd.DataFrame(data['customer_id'].unique(), columns = ['customer_id'])
cust_u = pd.DataFrame(data['customer_unique_id'].unique(), columns =
['customer_unique_id'])
sell = pd.DataFrame(data['seller_id'].unique(), columns = ['seller_id'])

Add the integer index values to each set
order['order_id_int'] = range(1, len(order['order_id'])+1)
cust['customer_id_int'] = range(1, len(cust['customer_id'])+1)
cust_u['customer_unique_id_int'] = range(1, len(cust_u['customer_unique_id'])+1)
sell['seller_id_int'] = range(1, len(sell['seller_id'])+1)

Merge each ID dataframe 'dictionary' to the greater dataset, then drop the original column,
renaming the new column to the old name for consistency
data = data.merge(order, how = 'left', on = 'order_id')
data.drop(columns = 'order_id', inplace = True)
data = data.merge(cust, how = 'left', on = 'customer_id')
data.drop(columns = 'customer_id', inplace = True)
data = data.merge(cust_u, how = 'left', on = 'customer_unique_id')
data.drop(columns = 'customer_unique_id', inplace = True)
data = data.merge(sell, how = 'left', on = 'seller_id')
data.drop(columns = 'seller_id', inplace = True);

Rename *_int columns to the original names
data.rename(columns = {'order_id_int':'order_id', 'customer_id_int':'customer_id',
'customer_unique_id_int':'customer_unique_id', 'seller_id_int':'seller_id'}, inplace=True)
```

```
We also realize there were order in process when this set was published, so we will
remove rows where there is no delivery date to the customer
data = data.dropna(subset = 'order_delivered_customer_date')
```

```
data.shape
```

```

```

### === Part 1: Seller Handoff Delays and Correlation with Customer Delivery Delays

To recapitulate from the EDA (see the column name list after the original data loading and first drop operation above), features already exist that show the following:

```
[arabic]
```

```
. "delivered+_+late" - True/False indicator of if the
"order+_+delivered+_+customer+_+date" was later than the
"order+_+estimated+_+delivery+_+date"
. "delivery+_+delay" - Difference between
"order+_+delivered+_+customer+_+date" and
"order+_+estimated+_+delivery+_+date" in days, negative values
being early deliveries and positive values being late deliveries.
. "handoff+_+delay" - Difference between
"order+_+delivered+_+carrier+_+date" and
"order+_+approved+_+at" in days
```

Since it's not clear if Olist has a policy that only allows sellers to be informed of a purchase needing shipment after payment has been approved or not, we will create a new field ("handoff+\_+delay+\_+alt") based on the difference between "order+\_+delivered+\_+carrier+\_+date" and "order+\_+purchase+\_+timestamp" to see if that has less instances of negative values for the "delay", since it should be physically impossible to deliver the product for shipment without even knowing of the existence of an order. The only negative values should be -1, and only then if the vagaries of time stamping across the four time zones spanning Brazil have an effect on the delivery/origin datetimes instead of the system adopting a universal time frame and enforcing it across the database. We convert the various date/time fields to Pandas datetime datatype since the original import interpreted them as strings.

### ==== Create Additional Necessary Features

```
+*In[]:*
```

```
[source, ipython3]
```

```

```

```
Convert date fields to the datetime datatype
data['order_purchase_timestamp'] = pd.to_datetime(data['order_purchase_timestamp']);
```

```

data['order_approved_at'] = pd.to_datetime(data['order_approved_at']);
data['order_estimated_delivery_date'] =
pd.to_datetime(data['order_estimated_delivery_date']);
data['shipping_limit_date'] = pd.to_datetime(data['shipping_limit_date']);
data['order_delivered_carrier_date'] = pd.to_datetime(data['order_delivered_carrier_date']);
data['order_delivered_customer_date'] =
pd.to_datetime(data['order_delivered_customer_date']);

```

```

data['handoff_delay_alt'] = (data['order_delivered_carrier_date'] -
data['order_purchase_timestamp']).dt.days

```

```

See descriptive statistics
print(data['handoff_delay_alt'].describe())

```

Instead of plotting a histogram and comparing to EDA histogram, it's easier to compare number of negative delivery delay values in each column:

```

+*In[]:*+
[source, ipython3]

neg = len(data[data['handoff_delay']<-1])
neg_alt = len(data[data['handoff_delay_alt']<-1])
print(f"There are {neg} handoff delay values less than -1 in the original delay field and
{neg_alt} in the alternative field");

```

Clearly, our original definition of handoff delay did not line up with the Online business rules surrounding date/time recording in their database. We will use the 'alt' value for all future comparisons. If either of the values are -1, we will convert them to 0 and chalk it up to the time zone differences mentioned previously, otherwise we will drop those rows as errors we cannot resolve.

```

+*In[]:*+
[source, ipython3]

print(data['handoff_delay_alt'].nsmallest(2))

```

```

+*In[]:*+
[source, ipython3]

Drop two negative rows since they both have absolute values greater than 1

```

```
data.drop(index = [29025,93479], inplace=True);

Drop the original handoff_delay column as it is no longer necessary, rename the alternative
to replace it
data.drop(columns = 'handoff_delay', inplace=True);
data.rename(columns = {'handoff_delay_alt':'handoff_delay'}, inplace=True);

```

```
+*In[]:*+
[source, ipython3]

data.shape

```

We now create a field ('`promised+\_handoff+\_delay`') that measures the difference between the "`order+\_delivered+\_carrier+\_date`" field and the "`shipping+\_limit+\_date`" field to determine if a handoff was early or late, much like the "`delivery+\_delay`" field discussed above.

```
+*In[]:*+
[source, ipython3]

data['promised_handoff_delay'] = (data['order_delivered_carrier_date'] -
data['shipping_limit_date']).dt.days

```

#### ==== Answering Part 1

We are finally in position to determine the answer to the first part of the research question. First, we determine if there is a correlation between the overall handoff delivery delay ('`handoff+\_delay`') and the amount of time early/late the handoff is made to the carrier ('`promised+\_handoff+\_delay`') to help determine the effectiveness of the system's handoff delivery time estimate mechanism. We will use the Spearman correlation method, since it has better performance when dealing with discrete, non-normal data. We will not use Kendall's correlation (tau) method, since it concerns rank correlations between sets, and we do not have ranked individual values, but rather pairwise discrete values.

```
+*In[]:*+
[source, ipython3]

```

```
corr, p = spearmanr(data['handoff_delay'], data['promised_handoff_delay'], axis = 0,
nan_policy='omit');
print(f"The Spearman correlation coefficient for time to handoff and days early/late for
handoff compared to the promise date \nis {corr:.3f} with an associated p-value for the
comparison of {p:.3f}.")
```

----

Since the p-value is  $< 0.05$ , we reject the null hypothesis that the two sets do not have an ordinal correlation. Based on the correlation coefficient, promised handoff delay should increase as the time it takes to make the handoff increases. We can confirm that general relationship with a scatter plot.

```
+*In[]:*+
[source, ipython3]

Limit single extreme outlier by only plotting times 5 months early or less

plt.figure(figsize = (8,8))
plt.scatter(x = data[data['promised_handoff_delay']>-150]['handoff_delay'],
 y = data[data['promised_handoff_delay']>-150]['promised_handoff_delay'], color =
'blue', marker = 'o')
plt.xlabel('Time From Order Placed to Handoff (days)')
plt.ylabel('Time Between Promised Handoff Date to Actual Handoff Date (days)')
plt.title('Difference b/w Promise and Actual Handoff vs. Difference b/w Order Placement and
Handoff')
plt.show()
```

----

We can clearly see that there is a general trend where the longer the time it takes to hand the order off from its origination date, the more likely the handoff is to be late, and the more likely the "magnitude of the lateness", so to speak, will be larger than for shorter handoff times.

With that knowledge in hand, we can now see if there's a similar relationship between how late a handoff is made, and how late a delivery is made to the customer (if any), which is the first half of the answer we need to the greater question in this part of the analysis.

```
+*In[]:*+
[source, ipython3]

corr2, p2 = spearmanr(data['promised_handoff_delay'], data['delivery_delay'], axis = 0,
nan_policy='omit');
```

```
print(f"The Spearman correlation coefficient for days early/late for handoff compared to days
early/late \nfor delivery to the customer is {corr2:.3f} with an associated p-value for the
comparison of {p2:.3f}.")
```

```

```

While this correlation is still statistically significant, it is a much weaker relationship than for the previous correlation. We will now plot this relationship in the same fashion as above to help illustrate graphically why the correlation is weaker (from a graphical perspective, not by root cause).

```
+*In[]:*+
[source, ipython3]

plt.figure(figsize = (8,8))
plt.scatter(x = data[data['promised_handoff_delay']>-150]['promised_handoff_delay'], y =
data[data['promised_handoff_delay']>-150]['delivery_delay'],
 color = 'blue', marker = 'o')
plt.ylabel('Time Between Promised Delivery Date and Actual Delivery Date (days)')
plt.xlabel('Time Between Promised Handoff Date to Actual Handoff Date (days)')
plt.title('Difference b/w Promise and Actual Handoff vs. Difference b/w Promise and Actual
Customer Delivery Date')
plt.show()

```

We definitely have multiple factors influencing the results in this case, which is why the correlation is weaker than for just the handoff comparison alone. This makes sense given that we're not only considering the results of the process by which various sellers get their products to the carriers, but also the processes by which those carriers get the products to customers all over Brazil. We saw in the EDA that there is considerable variance in delivery delay across regions, which is only one factor influencing the results displayed above. Given the reduction in correlation coefficient, we will next determine how often a delay in handoff literally leads to a late customer delivery. From the EDA, we know that 7.62% of all customer deliveries in the initial dataset are late, but the fully cleaned dataset used for the research question analyses has ++>+4k less records than the EDA set, so we will re-run the late deliveries analysis and a concurrent analysis to identify late handoffs, then combine the two:

```
+*In[]:*+
[source, ipython3]

data['delivered_late'] =
data['order_delivered_customer_date']>data['order_estimated_delivery_date']
```

```
print(data['delivered_late'].value_counts(normalize=True))
print(data['delivered_late'].value_counts())

```

```
+*In[]:*+
[source, ipython3]

data['handoff_late'] = data['order_delivered_carrier_date']>data['shipping_limit_date']
print(data['handoff_late'].value_counts(normalize=True))
print(data['handoff_late'].value_counts())

```

```
+*In[]:*+
[source, ipython3]

data['both_late'] = (data['handoff_late'] & data['delivered_late'])

print(data['both_late'].value_counts(normalize=True));
print(data['both_late'].value_counts());

```

Even though the proportions of orders where there was a late handoff or a late delivery are within 0.5%, only 1.48% of orders (1657) experienced both, as illustrated below:

```
+*In[]:*+
[source, ipython3]

Create values for conditions to ease calculations
ords = 109304 # All orders
lateh = 6471 # Orders with late handoff
lated = 7135 # Orders with late delivery
both = 1657 # Orders with both late handoff and late delivery

Abc = ords - (lateh + lated - both)
aBc = 0
abC = 0
ABc = lateh - both
aBC = 0
AbC = lated - both
ABC = both

plt.figure(figsize = (8,8))
mv.venn3(subsets = (Abc, aBc, ABc, abC, AbC, aBC, ABC), set_labels = ('All Orders', 'Late
H/O', 'Late Deliv'))
```

```
mv.venn3_circles(subsets = (Abc, aBc, ABc, abC, AbC, aBC, ABC))
plt.title('Venn Diagram of Late Handoffs and Deliveries as Subsets of All Deliveries')
plt.show()
```

----

++\*++NOTE: in the above Venn diagram, the numbers represent the size of the area that DOES NOT OVERLAP with another, so the 97355 represents deliveries that were neither handed off late nor delivered late.

Before digging any further into the effect (if any) of sellers on late deliveries, ostensibly due to late handoffs, we first must test if the two "lateness" proportions are associated with each other in a statistically significant fashion.

\*Note that this IS A DEVIATION FROM THE ORIGINAL METHODOLOGY, as we realized that the Venn diagram itself was insufficient to determine if the association between the two conditions was statistically significant, and therefore worthy of continued investigation.\*

For this analysis we will use a chi-squared test test of association (or independence depending on your perspective) using a contingency table based on the above Venn diagram.

```
+*In[]:*+
```

```
[source, ipython3]
```

----

```
Create the contingency table via crosstabulation and show that the combinations have the
same values as in the Venn diagram
```

```
cont_tbl = pd.crosstab(data['handoff_late'], data['delivered_late'], rownames =
['handoff_late'], colnames = ['delivered_late'])
print(cont_tbl)
```

```
Perform chi-squared test based on contingency table
import scipy.stats as sps
```

```
Perform the Chi-square test
```

```
Default null hypothesis is that late handoffs ARE INDEPENDENT of late deliveries
```

```
chi2, p, dof, expected = sps.chi2_contingency(cont_tbl)
```

```
crit = sps.chi2.ppf(0.975, 1) # Use 0.025 since for 2-sided test we need 1-a/2
```

```
print(f"\nCritical value: {crit:.3f}")
```

```
print(f"Chi-square statistic: {chi2:.3f}")
```

```
print(f"P-value: {p:.3f}")
```

```
print(f"Degrees of freedom: {dof}")
```

```
print("Expected frequencies:\n", expected)
```

----

**\*\*Since the p-value is  $< 0.05$  and the test statistic is larger than 5.024, we reject the null hypothesis and conclude that late handoffs are not independent of late deliveries\*\***

**=== Part 2: Which Sellers Hand Off Consistently Late, Which Leads to Late Deliveries?  
What Patterns or Behaviors Can Be Identified, If Any, That Contribute to That Lateness?**

**==== Identify Sellers with Late Handoffs and Deliveries**

```
+*In[]:*+
[source, ipython3]

Create sorted list of concurrent late handoff and delivery totals by seller
seller_late =
pd.DataFrame(data.groupby('seller_id')['both_late'].sum().sort_values(ascending = False))

```

```
+*In[]:*+
[source, ipython3]

Determine number of sellers involved with combination late handoff and late delivery
orders
seller_late[seller_late['both_late']>0].count()

```

```
+*In[]:*+
[source, ipython3]

seller_late['both_late'].value_counts()

```

```
+*In[]:*+
[source, ipython3]

Create cumulative % of total column for plotting line chart over bar chart to complete
Pareto chart
s_l = seller_late[seller_late['both_late']>5]
s_l.reset_index(drop = False, inplace = True)
s_l.loc[len(s_l)] = ['<6 orders', 428]
s_l['cum_pct'] = (s_l['both_late'].cumsum()/s_l['both_late'].sum())*100

```

**==== Plot Pareto Chart**

```

+*In[]:*+
[source, ipython3]

fig, ax = plt.subplots(figsize = (12,8));
xyz = ax.bar(x = range(1,len(s_l['seller_id'])+1), height = s_l['both_late'])
ax.xaxis.set_ticks(ticks = range(1,len(s_l['seller_id'])+1), labels = s_l['seller_id'], fontsize = 8,
rotation = 'vertical')
ax.set_xlabel('Seller ID')
ax.set_ylabel('Count of Orders')
ax2 = ax.twinx()
ax2.plot(range(1,len(s_l['seller_id'])+1), s_l['cum_pct'], '.r-', linewidth = 2, markersize = 1)
ax2.set_ylabel('% of Orders with both')
plt.title('Pareto Chart of Late Handoff & Late Delivery Orders by Seller ID')
plt.show();

```

Clearly, the Pareto Principle (80% or more of the issues are caused by 20% or less of the items/vendors/parts/etc., so focus on the 20% or less) does not apply here, as we would need to include over 100 vendors of the 498 to cover 80% or more of the 1657 late deliveries with late handoffs, in fact, we would need to include almost half of them to do so.

While we have identified the sellers that have the most late handoffs leading to late deliveries by count, we have not addressed the rate of "double lateness", which we address below:

```

+*In[]:*+
[source, ipython3]

ords_top = pd.DataFrame(data.groupby('seller_id')['order_id'].count())
ords_top.reset_index(inplace=True)
ords_top = ords_top.sort_values(by='order_id', ascending=False).head(50)
late_top = s_l.head(50)

```

```

+*In[]:*+
[source, ipython3]

vendors = pd.Series(ords_top[ords_top['seller_id'].isin(s_l['seller_id'])]['seller_id'])

```

```

+*In[]:*+
[source, ipython3]

```

```

ords = pd.DataFrame(data.groupby('seller_id')['order_id'].count())
ords.reset_index(inplace=True)
ords.rename(columns = {'order_id':'order_count'}, inplace=True)
tot_ords = ords[ords['seller_id'].isin(vendors)][['order_count']].sum()
tot_late = s_[s_['seller_id'].isin(vendors)][['both_late']].sum()

print(f"The {len(vendors):d} vendors in both 'Top 50' lists handled {tot_ords:d} orders
({tot_ords/len(data)*100:.1f}% of all orders), with {tot_late:d} or {tot_late/tot_ords*100:.2f}%
of those being handed off and delivered late.")

```

The late-for-both rate of 1.95% for these top sellers by order count and late-for-both count is close in value to the 1.52% of all orders that were late-for-both that was calculated above in Part 1 (see the result from above). While the rate among the top sellers is higher by absolute value, we need that difference to be statistically significant to convince ourselves that the suppliers themselves contribute something significant to the difference, so we will perform a two-sample chi-squared test of equal proportions (a type of hypothesis test) to determine if we can consider the values to not be equal, and therefore that we need to investigate the effect of sellers further:

```

+*In[]:*+
[source, ipython3]

establish input values for the test
all_orders = len(data) # All orders in the database, which was shown above to be 109304
sub_orders = tot_ords # Subset of orders from the overlap of top sellers and late-for-both
sellers, shown in the cell above to be 20200
all_late = 1657 # Number of late-for-both orders in the entire database, from above
sub_late = 394 # Number of late-for-both orders in the subset of interest from the
sub_orders set
value = 0 # Stating outright even though it's the default just to be clear that the null
hypothesis is that the 2 proportions are equal

Set up the proportions test using statsmodels function and alpha level of 0.05, meaning the
target statistic is z(0.025) = 1.96 for 2-sided test
zstat, pvalue = smsp.proportions_ztest(count = np.array([sub_late, all_late]), nobs =
np.array([sub_orders, all_orders]), value = 0,
alternative = 'two-sided', prop_var = False);
print(f"The calculated test statistic is {zstat:.3f} with a p-value of {pvalue:.3f}")

```

Since the test statistic was greater than 1.96 with a p-value less than 0.05, we reject the null hypothesis and conclude that the two proportions are not the same to the 0.05 level of uncertainty.

Therefore, we will proceed with the modeling exercise to determine if we can create a predictive model to anticipate late deliveries based on late handoffs.

The first look we take is to determine if the top 10 product categories are the same when both the handoff and delivery were late as those for when just the delivery was late as shown in the EDA

```
+*In[]:*+
[source, ipython3]

cat_late_both =
data.groupby('product_category_name_english')['both_late'].mean().sort_values(ascending=
False).head(10)

plt.figure(figsize=(10, 5))
sns.barplot(x=cat_late_both.values, y=cat_late_both.index)
plt.title('Top 10 Product Categories for Concurrent Late Handoff and Delivery Rates')
plt.xlabel('Concurrent Late H/O & Delivery Rate')
plt.ylabel('Product Category')
plt.tight_layout()
plt.show()

```

Only 2 of the categories actually make each list (both above and the one in Section VI.B of the EDA), 'fashion++\_++underwear++\_++beach' and 'construction++\_++tools++\_++lights'. At this point, instead of attempting to go through each feature individually, we will simply start attempting to model the results and use feature selection algorithms to determine what is the most predictive set.

=== Predictive Modeling

```
+*In[]:*+
[source, ipython3]

Remove irrelevant columns from data set before modeling begins, and split off target field
(bboth_late) as the y variable
data.drop(columns = ['order_item_id', 'order_status', 'order_purchase_timestamp',
'order_approved_at', 'order_estimated_delivery_date',
 'is_canceled', 'customer_zip_code_prefix', 'customer_city',
'seller_zip_code_prefix', 'order_delivered_customer_date',
 'shipping_limit_date', 'order_delivered_carrier_date', 'product_id',
'delivered_late', 'order_id', 'customer_id',
 'customer_unique_id', 'handoff_late', 'seller_city', 'price', 'freight_value'],
inplace = True)
```

```

print(data.info())

+*In[]:*+
[source, ipython3]

Drop NA rows
data = data.dropna()
data.shape

+*In[]:*+
[source, ipython3]

Split predictors and result column, convert boolean to binomial with {0,1} integer values
X = data.drop(columns=['both_late'], inplace = False)
yb = data['both_late']
y = yb.astype(int)
del(yb) # Drop interim variable

Encode variables for regression analysis prior to splitting so as not to miss any category
values in any field
cat_cols = ['customer_state', 'seller_state', 'product_category_name_english', 'seller_id']
num_cols = ['distance_km', 'product_weight_g', 'product_length_cm', 'product_height_cm',
'product_width_cm',
 'delivery_delay', 'handoff_delay', 'promised_handoff_delay']
proc = ColumnTransformer([
 ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=True), cat_cols), ('num',
StandardScaler(), num_cols)])

Encode categories and scale numerical columns
X_encode = proc.fit_transform(X)

Extract encoded set feature names
if hasattr(X_encode, 'toarray'):
 X_encode = X_encode.toarray()

Get column names from encoded set
cat_names = proc.named_transformers_['cat'].get_feature_names_out(cat_cols)
all_names = list(cat_names) + num_cols

Reconvert encoded set to dataframe with proper headers then drop original X to free
memory
X_new = pd.DataFrame(X_encode, columns = all_names)
del(X)

```

```
x_train, x_test, y_train, y_test = train_test_split(X_new, y, test_size = 0.2, random_state = 42, stratify = y)
```

```

```

We used the stratify option/parameter in train+\_+\_test+\_+\_split above to enable use of StratifiedKFold cross-validation during tuning and fitting.

```
++In[]:++
```

```
[source, ipython3]
```

```

```

```
Due to highly imbalanced nature of the target variable, apply SMOTE from the
'imbanced' sklearn (imblearn) package to oversample the minority
class (in our case, when both delivery and handoff late = TRUE) and prevent majority bias
in the regression
```

```
smote = SMOTE(random_state=42)
```

```
x_train_smote, y_train_smote = smote.fit_resample(x_train, y_train)
```

```

```

```
++In[]:++
```

```
[source, ipython3]
```

```

```

```
x_train_smote.shape
```

```

```

```
==== Logistic Regression
```

```
++In[]:++
```

```
[source, ipython3]
```

```

```

```
Initialize scikit-learn objects to execute feature selection optimization
```

```
scv = StratifiedKFold(shuffle = True, random_state = 42)
```

```
model = LogisticRegression(max_iter = 500, solver = 'lbfgs', verbose = 0, random_state = 42)
```

```

```

```
===== Feature Reduction
```

```
++In[]:++
```

```
[source, ipython3]
```

```

```

```
Perform feature reduction
```

```

feats = []
scores = []
Hyperparameter tuning for # of features
for i in range(49,0,-2):
 print(f"Running recursion for {i} features")
 selector = RFE(
 estimator = model,
 step = 0.1,
 n_features_to_select = i,
 verbose = 0)
 selector.set_output(transform="pandas")
 selector.fit(x_train_smote, y_train_smote)
 pred = selector.predict(x_test)
 feats.append(i)
 scores.append(fbeta_score(y_test, pred, beta = 2, pos_label = 1, average = 'binary'))

```

----

===== Plot Reduction Optimization Results

```

+*In[]:*+
[source, ipython3]

Plot test set score vs. # of features
plt.figure(figsize = (8,8))
plt.plot(feats, scores, '-b')
plt.xlabel('Number of Features')
plt.ylabel('F2 score')
plt.title('F2 Score vs. Number of Features for Feature Reduction Optimization')
plt.show()

```

----

===== Apply fit to limit features in training and test set

```

+*In[]:*+
[source, ipython3]

i = 12
selector1 = RFE(
 estimator = model,
 step = 0.1,
 n_features_to_select = i,
 verbose = 0)
selector1.set_output(transform = "pandas")
x_train_xform = selector1.fit_transform(x_train_smote, y_train_smote)
x_test_xform = selector1.transform(x_test)

```

```
Create DataFrame to carry column names for model fitting
names = selector1.get_feature_names_out()
x_train_xform1 = pd.DataFrame(x_train_xform, columns = names)

```

===== Use Statsmodels for model fit to enable regression reporting and odds calculations - duplicate model above using 'lbfgs' solver in both

```

+*In[]:*+
[source, ipython3]

from statsmodels.discrete.discrete_model import Logit

logit = Logit(y_train_smote, sm.add_constant(x_train_xform1))
fit = logit.fit(method = 'lbfgs', maxiter=500)
print(fit.summary())

```

All coefficients were found to be statistically significant, so we will not have to alter the training or test sets further.

===== Perform threshold optimization for test results

```

+*In[]:*+
[source, ipython3]

Create prediction probabilities for test set to use for threshold hyperparameter tuning
pred = fit.predict(sm.add_constant(x_test_xform))

f2=[]
th = np.arange(1,0.5,-0.02)
Create F2 and threshold pairs for plotting
for j in range(0,len(th),1):
 y_pred = (pred>=th[j]).astype(int)
 f2.append(fbeta_score(y_test, y_pred, beta = 2, pos_label = 1, average = 'binary'))

```

```

+*In[]:*+
[source, ipython3]

Plot F2 score vs. threshold values
plt.figure(figsize = (8,8))
plt.plot(th, f2, '-r')
plt.xlabel('Threshold values')
```

```

plt.ylabel('F2 score')
plt.title('F2 Score vs. Threshold Value for "True" Classification Optimization')
plt.ylim(0.415, 0.635)
plt.show()

+*In[]:*+
[source, ipython3]

th[f2==max(f2)]

+*In[]:*+
[source, ipython3]

Apply best threshold value
y_pred = (pred>0.62).astype(int)

Display confusion matrix and recall score for optimum model with threshold value of 0.789
applied to results
plt.figure(figsize = [8,6])
sns.heatmap(confusion_matrix(y_test, y_pred), annot = True, fmt = 'd', cmap = 'YlOrBr', cbar
= False, square = True)
plt.title('Confusion Matrix for Logistic Regression: Handoff and Delivery Late')
plt.xlabel('Predicted')
plt.ylabel('Truth')
plt.tight_layout()
plt.show()

+*In[]:*+
[source, ipython3]

print(f"Recall score = {recall_score(y_test, y_pred, pos_label = 1, average = 'binary'):.3f}")
print(f"Overall accuracy = {accuracy_score(y_test, y_pred):.3f}")

===== Generate and show odds ratios for each model feature

+*In[]:*+
[source, ipython3]

summary = fit.summary()
odds = pd.read_html(summary.tables[1].as_html(), header = 0, index_col = 0)[0]

```

```
odds['odds'] = np.exp(odds['coef'])
odds_table = odds.filter(['coef', 'odds', 'P>|z|'], axis = 1)
print(odds_table)

```

#### ==== Support Vector Machine

Thankfully, we can use the same encoding scheme for the SVM as for the regression. As discussed in the methodology, we are using Linear Support Vector Classification (LinearSVC) for our model because it scales to large sample sizes better than the other support vector machine models.

```
+*In[]:*+
[source, ipython3]

Perform feature reduction
model2 = LinearSVC(loss = 'hinge', C = 0.001, verbose = 0, random_state = 42, max_iter =
500)

feats2 = []
scores2 = []

Hyperparameter tuning for # of features
for i in range(49,0,-2):
 print(f"Running recursion for {i} features")
 selector2 = RFE(
 estimator = model2,
 step = 0.1,
 n_features_to_select = i,
 verbose = 0)
 selector2.set_output(transform="pandas")
 selector2.fit(x_train_smote, y_train_smote)
 pred2 = selector2.predict(x_test)
 feats2.append(i)
 scores2.append(fbeta_score(y_test, pred2, beta = 2, pos_label = 1, average = 'binary'))

+*In[]:*+
[source, ipython3]

Plot test set score vs. # of features
plt.figure(figsize = (8,8))
plt.plot(feats2, scores2, '-g')
plt.xlabel('Number of Features')
plt.ylabel('F2 score')
plt.title('F2 Score vs. Number of Features for Feature Reduction Optimization - LinearSVC')
```

```
plt.ylim(0.6, 0.64)
plt.show()
```

```

```

```
+#In[]:#+
[source, ipython3]
```

```

```

```
Apply best set of features
i = 18
```

```
selector2 = RFE(
 estimator = model2,
 step = 0.1,
 n_features_to_select = i,
 verbose = 0)
x_train_xform2 = selector2.fit_transform(x_train_smote, y_train_smote)
```

```

```

```
+#In[]:#+
[source, ipython3]
```

```

```

```
names2 = selector2.get_feature_names_out()
print(names2)
```

```

```

```
+#In[]:#+
[source, ipython3]
```

```

```

```
Hyperparameter tuning - since the L1 penalty is not supported in combination with
standard hinge loss, we will not use the L1 penalty type
```

```
params2 = {'C':[0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000], 'loss': ['hinge', 'squared_hinge']}
scorer = make_scorer(fbeta_score, beta = 2)
```

```
Initialize optimization routine
```

```
tuner2 = GridSearchCV(
 estimator = model2,
 param_grid = params2,
 scoring = scorer,
 cv = scv,
 verbose = 2)
```

```
Execute fitting
```

```
tuner2.fit(x_train_smote, y_train_smote)
```

```
Pull best fit
```

```

best_model2 = tuner2.best_estimator_
print("The model with the best score used these parameters: ", tuner2.best_params_)

+*In[]:*+
[source, ipython3]

Create prediction probabilities for test set to use for threshold hyperparameter tuning
pred2 = best_model2.predict(x_test)

plt.figure(figsize = [8,6])
sns.heatmap(confusion_matrix(y_test, pred2), annot = True, fmt = 'd', cmap = 'Greens', cbar
= False, square = True)
plt.title('Confusion Matrix for Linear SVC: Handoff and Delivery Late')
plt.xlabel('Predicted')
plt.ylabel('Truth')
plt.tight_layout()
plt.show()

+*In[]:*+
[source, ipython3]

print(f"Recall score = {recall_score(y_test, pred2, pos_label = 1, average = 'binary'):.3f}")
print(f"Overall accuracy = {accuracy_score(y_test, pred2):.3f}")

```

### RQ3

```

+*In[]:*+
[source, ipython3]

Importing Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set()

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, LabelEncoder
from imblearn.over_sampling import SMOTE

```

```

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, calinski_harabasz_score
from sklearn.decomposition import PCA
from scipy.stats import chi2_contingency, f_oneway

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

from sklearn.model_selection import StratifiedKFold, GridSearchCV

from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score,
roc_curve
from sklearn.metrics import make_scorer, recall_score, precision_recall_curve,
average_precision_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.losses import BinaryFocalCrossentropy
from scikeras.wrappers import KerasClassifier

from sklearn.metrics import fbeta_score

+*In[]:*+
[source, ipython3]

Importing Dataset
data = pd.read_csv('/Users/alfonsoqu/Desktop/ASU/2025Summer/DAT
490/cleaned_dataset.csv')

print("Dataset shape:", data.shape)
print("\nColumn names:")
print(data.columns.tolist())
print("\nFirst few rows:")
data.head()

+*In[]:*+
[source, ipython3]

Data Exploration
print("\nTarget variable distribution:")
print(data['is_canceled'].value_counts())

```

```

print("\nFirst-time buyer distribution:")
print(data['is_first_time_buyer'].value_counts())

+*In[]:*+
[source, ipython3]

Pre-Processing
Removing Unnecessary Columns
Keep relevant features for cancellation prediction and customer segmentation
data_processed = data.drop(columns=[
 'order_id',
 'order_item_id',
 'order_status',
 'order_purchase_timestamp',
 'order_approved_at',
 'order_estimated_delivery_date',
 'customer_id',
 'customer_unique_id',
 'customer_zip_code_prefix',
 'customer_lat',
 'customer_lng',
 'order_delivered_customer_date',
 'seller_zip_code_prefix',
 'seller_lat',
 'seller_lng',
 'shipping_limit_date',
 'order_delivered_carrier_date',
 'product_id',
 'review_id',
 'delivered_late',
 'delivery_delay',
 'review_score'
]).copy()

print("Data shape after column removal:", data_processed.shape)
print("\nRemaining columns:")
print(data_processed.columns.tolist())

+*In[]:*+
[source, ipython3]

Removing NA Rows
print("Missing values before cleaning:")
print(data_processed.isnull().sum())

```

```
data_processed = data_processed.dropna()
print("\nData shape after removing NA rows:", data_processed.shape)
```

```

```

```
+*In[]:*+
[source, ipython3]

Variable Selection
Target variable: is_canceled
Keep is_first_time_buyer for segmentation analysis
X = data_processed.drop(columns=['is_canceled'])
y = data_processed['is_canceled']
```

```
print("Features for modeling:")
print(X.columns.tolist())
print(f"\nTarget variable distribution:")
print(y.value_counts())
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
```

```

```

```
+*In[]:*+
[source, ipython3]

Encoding Variables
Define categorical and numerical features
categorical_features = [
 'customer_city',
 'customer_state',
 'seller_city',
 'seller_state',
 'seller_id',
 'product_category_name_english',
 'is_first_time_buyer', # Include for segmentation
 'delivered_late'
]
```

```
numerical_features = [
 'handoff_delay',
 'distance_km',
 'price',
 'freight_value',
 'product_name_length',
 'product_description_length',
```

```

 'product_photos_qty',
 'product_weight_g',
 'product_length_cm',
 'product_height_cm',
 'product_width_cm',
 'review_score',
 'delivery_delay'
]

Filter features that actually exist in the dataset
existing_categorical = [col for col in categorical_features if col in X.columns]
existing_numerical = [col for col in numerical_features if col in X.columns]

print("Categorical features to encode:", existing_categorical)
print("Numerical features to scale:", existing_numerical)

preprocessor = ColumnTransformer([
 ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=True),
existing_categorical),
 ('num', StandardScaler(), existing_numerical)
])

Transform
X_train_encoded = preprocessor.fit_transform(X_train)
X_test_encoded = preprocessor.transform(X_test)

+*In[]:*+
[source, ipython3]

Basic counts
print("1. Raw counts:")
print(data['is_canceled'].value_counts())
print()

Percentages
print("2. Percentages:")
print(data['is_canceled'].value_counts(normalize=True) * 100)
print()

+*In[]:*+
[source, ipython3]

More detailed breakdown
total_orders = len(data)

```

```

canceled_orders = data['is_canceled'].sum()
not_canceled_orders = total_orders - canceled_orders

print("3. Detailed breakdown:")
print(f"Total orders: {total_orders}")
print(f"Canceled orders: {canceled_orders}")
print(f"Not canceled orders: {not_canceled_orders}")
print(f"Cancellation rate: {(canceled_orders/total_orders)*100:.2f}%")
print()

Check data types to make sure we're reading correctly
print("4. Data type check:")
print(f"is_canceled column type: {data['is_canceled'].dtype}")
print(f"Unique values in is_canceled: {data['is_canceled'].unique()}")
print()

+*In[]:*+
[source, ipython3]

If the column contains True/False as strings instead of booleans
if data['is_canceled'].dtype == 'object':
 print("5. String value analysis:")
 print(data['is_canceled'].value_counts())
 print("Converting to boolean...")
 data['is_canceled'] = data['is_canceled'].map({'True': True, 'False': False, True: True,
False: False})
 print("After conversion:")
 print(data['is_canceled'].value_counts())
 print()

Cross-check with order_status if available
if 'order_status' in data.columns:
 print("6. Cross-check with order_status:")
 print(data['order_status'].value_counts())
 print()

Check relationship between order_status and is_canceled
print("7. Relationship between order_status and is_canceled:")
print(pd.crosstab(data['order_status'], data['is_canceled'], margins=True))
print()

Check for any patterns by customer type
if 'is_first_time_buyer' in data.columns:
 print("8. Cancellation by customer type:")
 cancellation_by_type = data.groupby('is_first_time_buyer')['is_canceled'].agg(['count',
'sum', 'mean'])

```

```

cancellation_by_type.columns = ['Total Orders', 'Canceled Orders', 'Cancellation Rate']
print(cancellation_by_type)
print()

Sample some canceled orders to verify
if canceled_orders > 0:
 print("9. Sample of canceled orders:")
 canceled_sample = data[data['is_canceled'] == True].head()
 print(canceled_sample[['order_id', 'is_canceled', 'order_status'] if 'order_status' in
data.columns else ['order_id', 'is_canceled']])
else:
 print("9. NO CANCELED ORDERS FOUND!")
 print("This explains why your models have AP = 0.00")
 print("Recommendations:")
 print("- Check if 'is_canceled' column is correctly defined")
 print("- Verify data preprocessing steps")
 print("- Consider using a different target variable")
 print("- Collect more data with actual cancellations")

+*In[]:*+
[source, ipython3]

Getting Feature Names
if hasattr(X_train_encoded, "toarray"):
 X_train_encoded = X_train_encoded.toarray()
 X_test_encoded = X_test_encoded.toarray()

Get column names
cat_feature_names =
preprocessor.named_transformers_['cat'].get_feature_names_out(existing_categorical)
all_feature_names = list(cat_feature_names) + existing_numerical

Convert to DataFrames
X_train_df = pd.DataFrame(X_train_encoded, columns=all_feature_names)
X_test_df = pd.DataFrame(X_test_encoded, columns=all_feature_names)

print("Final feature set shape:", X_train_df.shape)

+*In[]:*+
[source, ipython3]

Applying SMOTE
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_df, y_train)

```

```

print("After SMOTE - Training set shape:", X_train_resampled.shape)
print("After SMOTE - Target distribution:")
print(pd.Series(y_train_resampled).value_counts())

+*In[]:*+
[source, ipython3]

Applying StratifiedKFold
stratified_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

Focusing Recall for the Positive Cancellations
recall_scorer = make_scorer(recall_score, pos_label=True)

+*In[]:*+
[source, ipython3]

PREPARE DATA FOR CLUSTERING
Select features for clustering (behavioral and geographical patterns)
clustering_features = [
 'customer_state', 'customer_city', 'seller_state', 'seller_city',
 'product_category_name_english', 'price', 'freight_value',
 'product_weight_g', 'distance_km', 'handoff_delay',
 'is_first_time_buyer', 'review_score'
]

Filter features that exist in the dataset
available_features = [col for col in clustering_features if col in data.columns]
print(f"Available features for clustering: {available_features}")

Create clustering dataset
cluster_data = data[available_features + ['is_canceled']].copy()
cluster_data = cluster_data.dropna()

print(f"Data shape for clustering: {cluster_data.shape}")
print(f"Missing values: {cluster_data.isnull().sum().sum()}")

+*In[]:*+
[source, ipython3]

FEATURE ENGINEERING FOR CLUSTERING
Create additional behavioral features

```

```

if 'price' in cluster_data.columns and 'freight_value' in cluster_data.columns:
 cluster_data['total_cost'] = cluster_data['price'] + cluster_data['freight_value']
 cluster_data['freight_ratio'] = cluster_data['freight_value'] / (cluster_data['price'] + 1)

if 'distance_km' in cluster_data.columns:
 cluster_data['is_long_distance'] = (cluster_data['distance_km'] >
cluster_data['distance_km'].median()).astype(int)

if 'handoff_delay' in cluster_data.columns:
 cluster_data['has_handoff_delay'] = (cluster_data['handoff_delay'] > 0).astype(int)

Separate numerical and categorical features
numerical_features = cluster_data.select_dtypes(include=[np.number]).columns.tolist()
numerical_features = [col for col in numerical_features if col != 'is_canceled']

categorical_features = cluster_data.select_dtypes(include=['object', 'bool']).columns.tolist()
categorical_features = [col for col in categorical_features if col != 'is_canceled']

print(f"Numerical features: {numerical_features}")
print(f"Categorical features: {categorical_features}")

```

```

+*In[]:*+
[source, ipython3]

ENCODE CATEGORICAL VARIABLES
For clustering, we'll use Label Encoding for categorical variables
(Alternative: One-hot encoding, but it can create too many dimensions)
cluster_data_encoded = cluster_data.copy()
label_encoders = {}

for col in categorical_features:
 le = LabelEncoder()
 cluster_data_encoded[col] = le.fit_transform(cluster_data_encoded[col].astype(str))
 label_encoders[col] = le
 print(f"Encoded {col}: {len(le.classes_)} unique values")

Prepare final clustering features
clustering_cols = numerical_features + categorical_features
X_cluster = cluster_data_encoded[clustering_cols]

```

```

+*In[]:*+
[source, ipython3]

STANDARDIZE FEATURES

```

```

scaler = StandardScaler()
X_cluster_scaled = scaler.fit_transform(X_cluster)
X_cluster_scaled_df = pd.DataFrame(X_cluster_scaled, columns=clustering_cols)

print(f"Scaled data shape: {X_cluster_scaled_df.shape}")
print("Standardization completed")

+*In[]:*+
[source, ipython3]

DETERMINE OPTIMAL NUMBER OF CLUSTERS
Elbow method and Silhouette analysis
k_range = range(2, 11)
inertias = []
silhouette_scores = []
calinski_scores = []

for k in k_range:
 kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
 cluster_labels = kmeans.fit_predict(X_cluster_scaled)

 inertias.append(kmeans.inertia_)
 silhouette_scores.append(silhouette_score(X_cluster_scaled, cluster_labels))
 calinski_scores.append(calinski_harabasz_score(X_cluster_scaled, cluster_labels))

Plot evaluation metrics
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

Elbow curve
axes[0].plot(k_range, inertias, 'bo-')
axes[0].set_xlabel('Number of Clusters (k)')
axes[0].set_ylabel('Inertia')
axes[0].set_title('Elbow Method')
axes[0].grid(True)

Silhouette score
axes[1].plot(k_range, silhouette_scores, 'ro-')
axes[1].set_xlabel('Number of Clusters (k)')
axes[1].set_ylabel('Silhouette Score')
axes[1].set_title('Silhouette Analysis')
axes[1].grid(True)

Calinski-Harabasz score
axes[2].plot(k_range, calinski_scores, 'go-')
axes[2].set_xlabel('Number of Clusters (k)')
axes[2].set_ylabel('Calinski-Harabasz Score')

```

```

axes[2].set_title('Calinski-Harabasz Index')
axes[2].grid(True)

plt.tight_layout()
plt.show()

Find optimal k
optimal_k_silhouette = k_range[np.argmax(silhouette_scores)]
optimal_k_calinski = k_range[np.argmax(calinski_scores)]

print(f"Optimal k by Silhouette Score: {optimal_k_silhouette} (score:
{max(silhouette_scores):.3f})")
print(f"Optimal k by Calinski-Harabasz: {optimal_k_calinski} (score:
{max(calinski_scores):.1f})")

Use the better silhouette score
optimal_k = optimal_k_silhouette

+*In[]:*+
[source, ipython3]

PERFORM K-MEANS CLUSTERING
Fit final K-means model
final_kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
cluster_labels = final_kmeans.fit_predict(X_cluster_scaled)

Add cluster labels to original data
cluster_data_encoded['cluster'] = cluster_labels
cluster_data['cluster'] = cluster_labels

print(f"Clustering completed with {optimal_k} clusters")
print(f"Cluster distribution:")
print(pd.Series(cluster_labels).value_counts().sort_index())

+*In[]:*+
[source, ipython3]

CLUSTER CHARACTERIZATION
Analyze cluster characteristics
cluster_summary = cluster_data.groupby('cluster').agg({
 'is_canceled': ['count', 'sum', 'mean'],
 'price': 'mean',
 'freight_value': 'mean',
 'distance_km': 'mean',

```

```

 'review_score': 'mean'
 }).round(3)

cluster_summary.columns = [f"{col[0]}_{col[1]}" for col in cluster_summary.columns]
cluster_summary = cluster_summary.rename(columns={
 'is_canceled_count': 'Total_Orders',
 'is_canceled_sum': 'Canceled_Orders',
 'is_canceled_mean': 'Cancellation_Rate',
 'price_mean': 'Avg_Price',
 'freight_value_mean': 'Avg_Freight',
 'distance_km_mean': 'Avg_Distance',
 'review_score_mean': 'Avg_Review_Score'
})

print("Cluster Summary Statistics:")
print(cluster_summary)

```

```

+*In[]:*+
[source, ipython3]

BEHAVIORAL DIFFERENCES ANALYSIS

Statistical tests for differences between clusters
Chi-square test for cancellation rates
contingency_table = pd.crosstab(cluster_data['cluster'], cluster_data['is_canceled'])
chi2, p_value, dof, expected = chi2_contingency(contingency_table)

print(f"Chi-square test for cancellation differences between clusters:")
print(f"Chi-square statistic: {chi2:.4f}")
print(f"P-value: {p_value:.6f}")
print(f"Significant difference: {'Yes' if p_value < 0.05 else 'No'}")

ANOVA for numerical variables
numerical_vars_to_test = ['price', 'freight_value', 'distance_km']
for var in numerical_vars_to_test:
 if var in cluster_data.columns:
 cluster_groups = [cluster_data[cluster_data['cluster'] == i][var].dropna()
 for i in range(optimal_k)]

 # Remove empty groups
 cluster_groups = [group for group in cluster_groups if len(group) > 0]

 if len(cluster_groups) > 1:
 f_stat, p_val = f_oneway(*cluster_groups)
 print(f"ANOVA for {var}: F={f_stat:.4f}, p={p_val:.6f}")

```

```

+*In[]:*+
[source, ipython3]

CLUSTER VISUALIZATION
PCA for visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_cluster_scaled)

Create discrete color map for clusters
colors = plt.cm.Set1(np.linspace(0, 1, optimal_k))

plt.figure(figsize=(12, 8))

Plot each cluster separately for discrete legend
for i in range(optimal_k):
 mask = cluster_labels == i
 plt.scatter(X_pca[mask, 0], X_pca[mask, 1],
 c=[colors[i]], label=f'Cluster {i}', alpha=0.6, s=50)

plt.xlabel(f'First Principal Component (explained variance:
{pca.explained_variance_ratio_[0]:.2%})')
plt.ylabel(f'Second Principal Component (explained variance:
{pca.explained_variance_ratio_[1]:.2%})')
plt.title('Customer Clusters - PCA Visualization')
plt.legend(title='Customer Segments', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

Additional visualization: Cluster distribution
plt.figure(figsize=(10, 6))
cluster_counts = pd.Series(cluster_labels).value_counts().sort_index()
bars = plt.bar(range(optimal_k), cluster_counts.values, color=colors[:optimal_k])
plt.xlabel('Cluster ID')
plt.ylabel('Number of Customers')
plt.title('Customer Distribution Across Clusters')
plt.xticks(range(optimal_k), [f'Cluster {i}' for i in range(optimal_k)])

Add value labels on bars
for bar, count in zip(bars, cluster_counts.values):
 plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
 str(count), ha='center', va='bottom')

plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

```

----

+\*In[ ]:\*+

[source, ipython3]

----

### # CLUSTER PROFILES

for cluster\_id in range(optimal\_k):

    cluster\_subset = cluster\_data[cluster\_data['cluster'] == cluster\_id]

    print(f"\n--- CLUSTER {cluster\_id} PROFILE ---")

    print(f"Size: {len(cluster\_subset)} customers

    ({len(cluster\_subset)/len(cluster\_data)\*100:.1f}%)")

    print(f"Cancellation Rate: {cluster\_subset['is\_canceled'].mean():.3f}")

    # Most common categorical values

    for cat\_col in ['customer\_state', 'product\_category\_name\_english']:

        if cat\_col in cluster\_subset.columns:

            top\_category = cluster\_subset[cat\_col].mode().iloc[0] if

len(cluster\_subset[cat\_col].mode()) > 0 else "N/A"

            print(f"Most common {cat\_col}: {top\_category}")

    # Average numerical values

    numerical\_summary = cluster\_subset[numerical\_features].mean()

    print("Average values:")

    for col, val in numerical\_summary.items():

        print(f" {col}: {val:.2f}")

----

+\*In[ ]:\*+

[source, ipython3]

----

### # BUSINESS INSIGHTS

# Identify high-risk and low-risk clusters

cluster\_risk = cluster\_summary['Cancellation\_Rate'].sort\_values(ascending=False)

high\_risk\_clusters = cluster\_risk.head(2).index.tolist()

low\_risk\_clusters = cluster\_risk.tail(2).index.tolist()

print(f"HIGH-RISK CLUSTERS (highest cancellation rates): {high\_risk\_clusters}")

for cluster\_id in high\_risk\_clusters:

    rate = cluster\_summary.loc[cluster\_id, 'Cancellation\_Rate']

    count = cluster\_summary.loc[cluster\_id, 'Total\_Orders']

    print(f" Cluster {cluster\_id}: {rate:.3f} cancellation rate ({count} orders)")

print(f"\nLOW-RISK CLUSTERS (lowest cancellation rates): {low\_risk\_clusters}")

for cluster\_id in low\_risk\_clusters:

    rate = cluster\_summary.loc[cluster\_id, 'Cancellation\_Rate']

    count = cluster\_summary.loc[cluster\_id, 'Total\_Orders']

```

print(f" Cluster {cluster_id}: {rate:.3f} cancellation rate ({count} orders)")

+*In[]:*+
[source, ipython3]

Logistic Regression
Creating the Best Model
Logistic regression model
log_reg = LogisticRegression(max_iter=1000, solver='liblinear', random_state=42)

Hyperparameter grid (penalty type and C values)
param_grid = {
 'C': [0.01, 0.1, 1, 10],
 'penalty': ['l1', 'l2'] # Try both L1 and L2 regularization
}

Grid search setup
grid_log_reg = GridSearchCV(
 estimator=log_reg,
 param_grid=param_grid,
 scoring=recall_scorer,
 cv=stratified_cv,
 verbose=2,
 n_jobs=1
)

Fit model
grid_log_reg.fit(X_train_resampled, y_train_resampled)

Extract best model and params
best_log_reg = grid_log_reg.best_estimator_
print("Best Parameters:", grid_log_reg.best_params_)

+*In[]:*+
[source, ipython3]

Testing the Model
Predict probabilities
y_proba_logreg = best_log_reg.predict_proba(X_test_df)[: , 1]
=== Threshold Optimization ===
precision, recall, thresholds = precision_recall_curve(y_test, y_proba_logreg)
f2_scores = 5 * (precision * recall) / (4 * precision + recall + 1e-6)
optimal_threshold_logreg = thresholds[np.argmax(f2_scores)]
print(f"\nOptimal Threshold (LogReg): {optimal_threshold_logreg:.4f}")

```

```

Apply optimal threshold
y_pred_logreg_opt = (y_proba_logreg >= optimal_threshold_logreg).astype(int)

+*In[]:*+
[source, ipython3]

Metrics
print("\nClassification Report (Logistic Regression):")
print(classification_report(y_test, y_pred_logreg_opt))

roc_auc = roc_auc_score(y_test, y_proba_logreg)
print(f"ROC-AUC Score: {roc_auc:.4f}")

plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred_logreg_opt), annot=True, fmt='d',
cmap='Blues')
plt.title("Confusion Matrix: Logistic Regression")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

fpr, tpr, _ = roc_curve(y_test, y_proba_logreg)
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, label=f"LogReg AUC = {roc_auc:.2f}", color='blue')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Logistic Regression")
plt.legend()
plt.tight_layout()
plt.show()

+*In[]:*+
[source, ipython3]

Decision Tree
Creating the Best Model
Define the Decision Tree Classifier
tree_clf = DecisionTreeClassifier(random_state=42)

Define hyperparameter grid
param_grid_tree = {

```

```

 'max_depth': [5, 10, 15],
 'min_samples_split': [10, 20],
 'criterion': ['gini', 'entropy']
}

Grid search
grid_tree = GridSearchCV(
 estimator=tree_clf,
 param_grid=param_grid_tree,
 scoring=recall_scorer,
 cv=stratified_cv,
 verbose=2,
 n_jobs=1
)

Fit to the resampled training set
grid_tree.fit(X_train_resampled, y_train_resampled)

Best model and parameters
best_tree = grid_tree.best_estimator_
print("Best Parameters:", grid_tree.best_params_)

+*In[]:*+
[source, ipython3]

Evaluating the Model
Predict probabilities
y_proba_tree = best_tree.predict_proba(X_test_df)[:, 1]

=== Threshold Optimization ===
precision_tree, recall_tree, thresholds_tree = precision_recall_curve(y_test, y_proba_tree)
f2_scores_tree = 5 * (precision_tree * recall_tree) / (4 * precision_tree + recall_tree + 1e-6)
optimal_threshold_tree = thresholds_tree[np.argmax(f2_scores_tree)]
print(f"\nOptimal Threshold (Decision Tree): {optimal_threshold_tree:.4f}")

Apply optimal threshold
y_pred_tree_opt = (y_proba_tree >= optimal_threshold_tree).astype(int)

Metrics
print("\nClassification Report (Decision Tree):")
print(classification_report(y_test, y_pred_tree_opt))

roc_auc_tree = roc_auc_score(y_test, y_proba_tree)
print(f"ROC-AUC Score (Tree): {roc_auc_tree:.4f}")

```

```

+*In[]:*+
[source, ipython3]

Visualizing Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred_tree_opt), annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix: Decision Tree")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

ROC Curve
fpr_tree, tpr_tree, _ = roc_curve(y_test, y_proba_tree)
plt.figure(figsize=(6, 5))
plt.plot(fpr_tree, tpr_tree, label=f"Decision Tree AUC = {roc_auc_tree:.2f}",
color='darkorange')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Decision Tree")
plt.legend()
plt.tight_layout()
plt.show()

Visualizing the Decision Tree
plt.figure(figsize=(20, 12))
plot_tree(
 best_tree,
 feature_names=all_feature_names,
 class_names=['Not Canceled', 'Canceled'],
 filled=True,
 rounded=True,
 fontsize=10,
 max_depth=3 # limit depth for readability, remove or increase to see more
)
plt.title("Decision Tree Visualization (Top Levels)")
plt.show()

+*In[]:*+
[source, ipython3]

Random Forest
Creating the Best Model
Random Forest

```

```

rf_clf = RandomForestClassifier(random_state=42)

param_grid_rf = {
 'n_estimators': [100, 200],
 'max_depth': [10, 15],
 'min_samples_split': [10, 20],
 'class_weight': ['balanced'] # Helps handle imbalance without SMOTE here, but still okay
 if you keep SMOTE
}

grid_rf = GridSearchCV(
 estimator=rf_clf,
 param_grid=param_grid_rf,
 scoring=recall_scorer,
 cv=stratified_cv,
 verbose=2,
 n_jobs=1
)

grid_rf.fit(X_train_resampled, y_train_resampled)
best_rf = grid_rf.best_estimator_
print("Best Parameters (Random Forest):", grid_rf.best_params_)

+*ln[]:*+
[source, ipython3]

Evaluating the Model
Predict probabilities
y_proba_rf = best_rf.predict_proba(X_test_df)[:, 1]

=== Threshold Optimization ===
precision_rf, recall_rf, thresholds_rf = precision_recall_curve(y_test, y_proba_rf)
f2_scores_rf = 5 * (precision_rf * recall_rf) / (4 * precision_rf + recall_rf + 1e-6)
optimal_threshold_rf = thresholds_rf[np.argmax(f2_scores_rf)]
print(f"\nOptimal Threshold (Random Forest): {optimal_threshold_rf:.4f}")

Apply optimal threshold
y_pred_rf_opt = (y_proba_rf >= optimal_threshold_rf).astype(int)

Metrics
print("\nClassification Report (Random Forest):")
print(classification_report(y_test, y_pred_rf_opt))

roc_auc_rf = roc_auc_score(y_test, y_proba_rf)
print(f"ROC-AUC Score (Random Forest): {roc_auc_rf:.4f}")

```

```
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred_rf_opt), annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix: Random Forest")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()
```

```
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_proba_rf)
plt.figure(figsize=(6, 5))
plt.plot(fpr_rf, tpr_rf, label=f"Random Forest AUC = {roc_auc_rf:.2f}", color='green')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Random Forest")
plt.legend()
plt.tight_layout()
plt.show()
```

----

```
+*In[]:*+
[source, ipython3]

XGBoost
Creating the Model
xgb_clf = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
param_grid_xgb = {
 'n_estimators': [100, 150],
 'max_depth': [3, 5, 7],
 'learning_rate': [0.01, 0.1, 0.2],
 'subsample': [0.8, 1.0]
}

grid_xgb = GridSearchCV(
 estimator=xgb_clf,
 param_grid=param_grid_xgb,
 scoring=recall_scorer,
 cv=stratified_cv,
 verbose=2,
 n_jobs=1
)

grid_xgb.fit(X_train_resampled, y_train_resampled)
best_xgb = grid_xgb.best_estimator_
print("Best Parameters (XGBoost):", grid_xgb.best_params_)

```

```

+*In[]:*+
[source, ipython3]

Evaluating the Model
Predict probabilities
y_proba_xgb = best_xgb.predict_proba(X_test_df)[:, 1]

Threshold Optimization
precision_xgb, recall_xgb, thresholds_xgb = precision_recall_curve(y_test, y_proba_xgb)
f2_scores_xgb = 5 * (precision_xgb * recall_xgb) / (4 * precision_xgb + recall_xgb + 1e-6)
optimal_threshold_xgb = thresholds_xgb[np.argmax(f2_scores_xgb)]
print(f"\nOptimal Threshold (XGBoost): {optimal_threshold_xgb:.4f}")

Apply threshold
y_pred_xgb_opt = (y_proba_xgb >= optimal_threshold_xgb).astype(int)

+*In[]:*+
[source, ipython3]

Metrics
print("\nClassification Report (XGBoost):")
print(classification_report(y_test, y_pred_xgb_opt))

roc_auc_xgb = roc_auc_score(y_test, y_proba_xgb)
print(f"ROC-AUC Score (XGBoost): {roc_auc_xgb:.4f}")

plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred_xgb_opt), annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix: XGBoost")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

fpr_xgb, tpr_xgb, _ = roc_curve(y_test, y_proba_xgb)
plt.figure(figsize=(6, 5))
plt.plot(fpr_xgb, tpr_xgb, label=f"XGBoost AUC = {roc_auc_xgb:.2f}", color='purple')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - XGBoost")
plt.legend()
plt.tight_layout()
plt.show()

```

```

+*In[]:*+
[source, ipython3]

ANN
Creating the Model
Define the ANN model
def create_complex_ann():
 model = Sequential([
 Dense(256, activation='relu', input_shape=(X_train_resampled.shape[1],)),
 BatchNormalization(),
 Dropout(0.4),
 Dense(128, activation='relu'),
 BatchNormalization(),
 Dropout(0.3),
 Dense(64, activation='relu'),
 BatchNormalization(),
 Dropout(0.3),
 Dense(1, activation='sigmoid')
])
 focal_loss = BinaryFocalCrossentropy(gamma=2.0)
 model.compile(optimizer=Adam(learning_rate=0.001), loss=focal_loss, metrics=['Recall'])
 return model

Wrap in KerasClassifier for calibration
keras_clf = KerasClassifier(model=create_complex_ann, epochs=15, batch_size=32,
verbose=2)
keras_clf.fit(X_train_resampled, y_train_resampled)

+*In[]:*+
[source, ipython3]

Evaluating the Model
Predict probabilities
y_proba_ann = keras_clf.predict_proba(X_test_df)[:, 1]

Threshold Optimization
precision_ann, recall_ann, thresholds_ann = precision_recall_curve(y_test, y_proba_ann)
f2_scores_ann = 5 * (precision_ann * recall_ann) / (4 * precision_ann + recall_ann + 1e-6)
optimal_threshold_ann = thresholds_ann[np.argmax(f2_scores_ann)]
print(f"\nOptimal Threshold (ANN): {optimal_threshold_ann:.4f}")

Apply threshold
y_pred_ann_opt = (y_proba_ann >= optimal_threshold_ann).astype(int)

```

```

+*In[]:*+
[source, ipython3]

Metrics
print("\nClassification Report (ANN):")
print(classification_report(y_test, y_pred_ann_opt))

roc_auc_ann = roc_auc_score(y_test, y_proba_ann)
print(f"ROC-AUC Score (ANN): {roc_auc_ann:.4f}")

Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, y_pred_ann_opt), annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix: ANN (Focal Loss)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

ROC Curve
fpr_ann, tpr_ann, _ = roc_curve(y_test, y_proba_ann)
plt.figure(figsize=(6, 5))
plt.plot(fpr_ann, tpr_ann, label=f"ANN AUC = {roc_auc_ann:.2f}", color='green')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - ANN (Focal Loss)")
plt.legend()
plt.tight_layout()
plt.show()

+*In[]:*+
[source, ipython3]

Customer Segmentation Analysis
Analyze cancellation behavior by customer segments
print("=== CUSTOMER SEGMENTATION ANALYSIS ===")

Analyze cancellation rates by first-time vs repeat buyers
segmentation_analysis =
data_processed.groupby('is_first_time_buyer')['is_canceled'].agg(['count', 'sum',
'mean']).round(4)
segmentation_analysis.columns = ['Total_Orders', 'Canceled_Orders', 'Cancellation_Rate']
print("\nCancellation Analysis by Customer Type:")

```

```

print(segmentation_analysis)

Analyze by region (state)
region_analysis = data_processed.groupby('customer_state')['is_canceled'].agg(['count',
'sum', 'mean']).round(4)
region_analysis.columns = ['Total_Orders', 'Canceled_Orders', 'Cancellation_Rate']
region_analysis = region_analysis.sort_values('Cancellation_Rate', ascending=False)
print("\nTop 10 States by Cancellation Rate:")
print(region_analysis.head(10))

Statistical test for difference between first-time and repeat buyers
from scipy.stats import chi2_contingency

contingency_table = pd.crosstab(data_processed['is_first_time_buyer'],
data_processed['is_canceled'])
chi2, p_value, dof, expected = chi2_contingency(contingency_table)
print(f"\nChi-square test for independence:")
print(f"Chi-square statistic: {chi2:.4f}")
print(f"P-value: {p_value:.4f}")
print(f"Degrees of freedom: {dof}")

Model Comparison
Classification Reports
print("Classification Report - Logistic Regression")
print(classification_report(y_test, y_pred_logreg_opt))

print("\nClassification Report - Decision Tree")
print(classification_report(y_test, y_pred_tree_opt))

print("\nClassification Report - Random Forest")
print(classification_report(y_test, y_pred_rf_opt))

print("\nClassification Report - XGBoost")
print(classification_report(y_test, y_pred_xgb_opt))

print("\nClassification Report - ANN")
print(classification_report(y_test, y_pred_ann_opt))

+*In[]:*+
[source, ipython3]

General Comparison
def compute_f2(y_true, y_pred):
 return fbeta_score(y_true, y_pred, beta=2)

model_results = pd.DataFrame({

```

```

"Model": ["Logistic Regression", "Decision Tree", "Random Forest", "XGBoost", "ANN"],
"F2 Score": [
 compute_f2(y_test, y_pred_logreg_opt),
 compute_f2(y_test, y_pred_tree_opt),
 compute_f2(y_test, y_pred_rf_opt),
 compute_f2(y_test, y_pred_xgb_opt),
 compute_f2(y_test, y_pred_ann_opt)
],
"Recall": [
 recall_score(y_test, y_pred_logreg_opt),
 recall_score(y_test, y_pred_tree_opt),
 recall_score(y_test, y_pred_rf_opt),
 recall_score(y_test, y_pred_xgb_opt),
 recall_score(y_test, y_pred_ann_opt)
],
"ROC AUC": [
 roc_auc,
 roc_auc_tree,
 roc_auc_rf,
 roc_auc_xgb,
 roc_auc_ann
]
})

```

```

display(model_results.sort_values(by="F2 Score",
ascending=False).reset_index(drop=True))

```

----

```

+*In[]:*+
[source, ipython3]

ROC Curve Comparison
Compute ROC curve data for each model
roc_data = []

Logistic Regression
fpr, tpr, _ = roc_curve(y_test, y_proba_logreg)
roc_data.append(("Logistic Regression", fpr, tpr, roc_auc))

Decision Tree
fpr, tpr, _ = roc_curve(y_test, y_proba_tree)
roc_data.append(("Decision Tree", fpr, tpr, roc_auc_tree))

Random Forest
fpr, tpr, _ = roc_curve(y_test, y_proba_rf)
roc_data.append(("Random Forest", fpr, tpr, roc_auc_rf))

```

```

XGBoost
fpr, tpr, _ = roc_curve(y_test, y_proba_xgb)
roc_data.append(("XGBoost", fpr, tpr, roc_auc_xgb))

ANN
fpr, tpr, _ = roc_curve(y_test, y_proba_ann)
roc_data.append(("ANN", fpr, tpr, roc_auc_ann))

Plot
plt.figure(figsize=(10, 8))
for name, fpr, tpr, auc_score in roc_data:
 plt.plot(fpr, tpr, label=f"{name} (AUC = {auc_score:.2f})")

plt.plot([0, 1], [0, 1], 'k--', label="Random Classifier")
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves - All Models')
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
plt.show()

+*In[]:*+
[source, ipython3]

Precision Recall Curve
Compute Precision-Recall curve data
pr_data = []

Logistic Regression
precision, recall, _ = precision_recall_curve(y_test, y_proba_logreg)
ap_logreg = average_precision_score(y_test, y_proba_logreg)
pr_data.append(("Logistic Regression", precision, recall, ap_logreg))

Decision Tree
precision, recall, _ = precision_recall_curve(y_test, y_proba_tree)
ap_tree = average_precision_score(y_test, y_proba_tree)
pr_data.append(("Decision Tree", precision, recall, ap_tree))

Random Forest
precision, recall, _ = precision_recall_curve(y_test, y_proba_rf)
ap_rf = average_precision_score(y_test, y_proba_rf)
pr_data.append(("Random Forest", precision, recall, ap_rf))

XGBoost
precision, recall, _ = precision_recall_curve(y_test, y_proba_xgb)

```

```

ap_xgb = average_precision_score(y_test, y_proba_xgb)
pr_data.append(("XGBoost", precision, recall, ap_xgb))

ANN
precision, recall, _ = precision_recall_curve(y_test, y_proba_ann)
ap_ann = average_precision_score(y_test, y_proba_ann)
pr_data.append(("ANN", precision, recall, ap_ann))

Plot
plt.figure(figsize=(10, 8))
for name, precision, recall, ap_score in pr_data:
 plt.plot(recall, precision, label=f"{name} (AP = {ap_score:.2f})")

plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curves - All Models')
plt.legend(loc='upper right')
plt.grid(True)
plt.tight_layout()
plt.show()

+*In[]:*+
[source, ipython3]

XGBoost Feature Importance Analysis

Get feature importance from the trained XGBoost model
feature_importance_xgb = best_xgb.feature_importances_

Create DataFrame with feature names and importance scores
feature_importance_df = pd.DataFrame({
 'feature': all_feature_names,
 'importance': feature_importance_xgb
}).sort_values('importance', ascending=False)

Display top 20 features
print("=== TOP 20 MOST IMPORTANT FEATURES (XGBoost) ===")
top_20_features = feature_importance_df.head(20)
print(top_20_features.to_string(index=False))

Create visualization
plt.figure(figsize=(12, 10))
plt.barh(range(20), top_20_features['importance'].values[::-1])
plt.yticks(range(20), top_20_features['feature'].values[::-1])
plt.xlabel('Feature Importance')
plt.title('Top 20 Most Important Features - XGBoost')

```

```
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()

```